Communication and Data Efficient Algorithms for Vertical Federated Learning

Submitted in partial fulfillment of the requirements for

the degree of

Doctor of Philosophy

in

Electrical and Computer Engineering

Pedro Valdeira

B.Sc., Aerospace Engineering, Instituto Superior Técnico
M.Sc., Aerospace Engineering, Instituto Superior Técnico
M.S., Electrical and Computer Engineering, Carnegie Mellon University

Carnegie Mellon University
Pittsburgh, PA

August 2025

Acknowledgments

The research in this thesis was supported by the Fundação para a Ciência e a Tecnologia through the Carnegie Mellon Portugal Program under grant SFRH/BD/150738/2020.

I am deeply grateful to everyone who has contributed to my Ph.D. journey, both professionally and personally.

First, I would like to thank my advisors: Prof. Yuejie Chi, my CMU advisor and thesis committee chair, Prof. João Xavier, my IST advisor and thesis committee member, and Prof. Cláudia Soares, my co-advisor. I am grateful for the support, mentorship, and technical guidance that you have provided me throughout my Ph.D., as well as for giving me the freedom to explore my ideas. I have learned a lot from you.

I would also like to thank the other members of my thesis committee, Prof. Soummya Kar, Dr. Shiqiang Wang, Prof. Zhize Li, and Prof. João Pedro Gomes, for their insightful feedback and interesting discussions. Your expertise has helped me improve the quality of this thesis, and I appreciate the time and effort that you have dedicated to this process.

I would further like to thank Dr. Shiqiang Wang for his mentorship and guidance as a collaborator. I greatly appreciate your encouragement and advice, and I am grateful for the opportunity to work with and learn from you.

Moreover, I am very grateful for the great colleagues and friends that I have met throughout this journey. In particular, I would like to acknowledge the members of my research group at CMU—Prof. Maxime Ferreira Da Costa, Prof. Harlin Lee, Prof. Vincent Monardo, Dr. Tian Tong, Dr. Boyue Li, Prof. Laixi Shi, Dr. Shicong Cen, Diogo Cardoso, Jiin Woo, Harry Dong, Prof. Zhize Li, Lingjing Kong, Zixin Wen, He Wang, Xingyu Xu, Tong Yang, Dr. Sudeep Salgia, and Timofey Efimov—as well as the members of the Signal and Image Processing Group, at IST, and of my research group at NOVA. I would also like to thank my friends at Porter Hall, my friends in the CMU Portugal program, and, more broadly, all my CMU friends for your companionship.

Finally, I express my heartfelt gratitude to my family—especially my parents and sister—and longtime friends, for your unwavering and unconditional support over the years.

Pedro Valdeira

Abstract

Federated learning (FL) is a collaborative machine learning paradigm where multiple clients jointly train a model without sharing their raw local data. FL can be categorized as *horizontal* FL, where clients hold different subsets of samples, or *vertical* FL (VFL), where clients hold different features of a shared set of samples. There have been significant recent advances in FL, yet most research has focused on the horizontal setting. Despite the importance of VFL to domains such as finance and healthcare, there is still a gap between existing work and real-world setups—namely when it comes to dealing with communication and data-availability constraints.

The first part of this thesis focuses on communication efficiency in VFL. In FL, client-server schemes, where the clients communicate only with a central server, are predominant; however, such setups strain the finite bandwidth of the server, often causing latency and becoming a bottleneck in training. To mitigate this, we resort to lossy compression. We introduce error feedback compressed VFL (`EF-VFL`), which employs an error feedback mechanism to stabilize communication-compressed training in VFL. Numerical experiments confirm the improved communication efficiency of `EF-VFL` over state-of-the-art methods.

The second part of this thesis addresses data efficiency in VFL. More precisely, we tackle the problem of leveraging incomplete samples, where some clients lack their feature blocks, during both training and inference. We propose `LASER-VFL`, a simple yet effective method relying on model parameter sharing and a task-sampling mechanism to efficiently train a family of predictors that is capable of handling arbitrary sets of observed feature blocks. `LASER-VFL` significantly improves over the performance of baselines, both in the presence and, remarkably, even in the absence of missing features.

The third part of this thesis explores an alternative approach to mitigate the bandwidth bottleneck in the client-server scheme: the use of direct client-client links. In this semi-decentralized scheme, client-client links enable information flow without straining server bandwidth, while client-server links accelerate convergence in poorly connected client networks. We propose multi-token coordinate descent (`MTCD`), a semi-decentralized VFL method. Token methods avoid the staleness seen in most decentralized optimization methods, at the cost of losing their parallelism. By running multiple tokens in parallel, MTCD allow us to navigate this staleness-parallelism trade-off. `MTCD` recovers the client-server and decentralized schemes as special cases, and, by tuning its reliance on client-server links, it can also span the spectrum of semi-decentralized configurations in between. This flexibility allows `MTCD` to outperform both ends of the spectrum in applications where neither extreme is ideal.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1

Introduction

In many real-world settings, no single entity holds all the existing data that is relevant to train a machine learning model for a given task. Rather, this information is scattered across multiple data owners. Instead of training separate models on their local subsets, these entities can engage in collaborative learning to leverage the full, distributed dataset, and thus improve the performance of the model. However, privacy regulations often prohibit sharing raw data, and the sheer size of the data often makes centralization impractical.

In federated learning (FL), these data owners, known as *clients*, jointly train a model without exposing their local raw data (McMahan et al., 2017). To achieve this, the clients alternate between locally updating the model based on their own data and exchanging task-relevant representations of their data, but never their raw data. This typically relies on a central server that aggregates the updated information sent by each client and distributes the information resulting from this aggregation back to the clients for the next round of local training. We illustrate this in Figure 1.1. This iterative approach enables the clients to collaboratively learn the task of interest while keeping the raw data local.

FL applications typically fall into one of two settings. In *cross-silo* FL, we have a small number of well-resourced clients with reliable connectivity. This includes, for example, hospitals collaborating to train diagnostic models (Rieke et al., 2020) and banks jointly learning risk predictors (Cheng et al., 2020). In contrast, in *cross-device* FL, we have a large number of clients with limited compute and unreliable connectivity. This is the case, for example, of mobile devices collaborating to train next-word prediction models (Hard et al., 2018) and of internet-of-things sensor networks that learn to perform tasks that enable smart transportation (Nguyen et al., 2021).

FL setups can also be categorized based on how the global dataset is partitioned across clients. Consider a design matrix $\boldsymbol{X} \in \mathbb{R}^{N \times d}$, where the nth row of $\boldsymbol{X}$ corresponds to

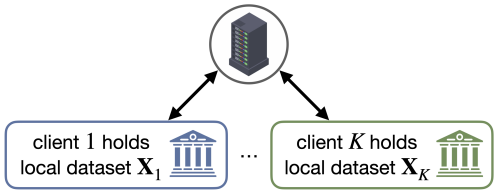
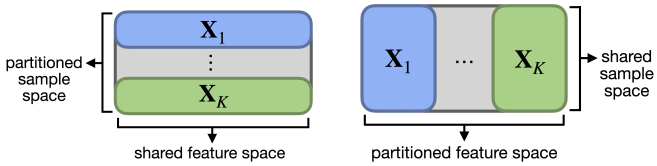


Figure 1.1: An illustration of an example federated learning setup.



(a) Horizontal FL. (b) Vertical FL.

Figure 1.2: An illustration of the categorization of federated learning (FL) based on dataset partitioning. This thesis focuses on the vertical FL setting.

a sample $\boldsymbol{x}_n$ with d features. When the set of clients $\mathcal{K} \coloneqq \{1, \ldots, K\}$ share the same feature space but hold different subsets of samples, their local datasets $\{\boldsymbol{X}_k\}$ correspond to horizontal splits of the design matrix, as seen in Figure 1.2a. This setting is known as *horizontal* FL. In contrast, when the clients share the same sample space but hold different subsets of features, the local datasets correspond to vertical splits of the design matrix, as seen in Figure 1.2b. This setting is known as *vertical* FL. Since FL does not allow for the centralization and redistribution of raw data, the type of partitioning is dictated by how the data naturally arises in each application, rather than by design choice. In this thesis, we focus on the vertical FL (VFL) setup (Liu et al., 2024).

In VFL, the global dataset $\boldsymbol{X}$ is partitioned across the clients by features, with each client $k \in \mathcal{K}$ holding a local dataset $\boldsymbol{X}_k \in \mathbb{R}^{N \times d_k}$, where d_k is the number of features observed by client k. This can correspond, for example, to an online retail company and a social media platform holding different attributes of shared users. Therefore, we have that $\boldsymbol{x}^n = (\boldsymbol{x}_1^n, \ldots, \boldsymbol{x}_K^n)$ for all n, where $\boldsymbol{x}_k^n$ is the block of features of sample n that is observed by client k. Note that, in contrast to horizontal FL, the clients participating in VFL each collect local datasets with distinct types of information (features), and therefore often represent organizations in different sectors. This reduces conflicting interests and thus strengthens their incentive to collaborate.

Cross-silo VFL applications include scenarios where multiple entities hold distinct attributes concerning a shared set of users and seek to collaboratively train a predictor, as in the example above. For example:

- WeBank partners with other companies to jointly build a risk model from data regarding shared customers (Cheng et al., 2020).

- Sun et al. (2021) predict perioperative complication risk by resorting to both internal hospital data, such as medical exams and lab results, and external patient data, such as daily health metrics from wearable devices.

In addition to the cross-silo applications that initially drew attention to VFL, its vertical paradigm also supports important cross-device use cases. For example, consider a network of devices, each collecting sensor- or location-specific data across a city, whose combined readings at a given timestamp form a feature-distributed sample. Due to the large number of such devices, centralizing all raw streams of data is impractical. Thus, by allowing devices to collaboratively learn a task while keeping their data local, VFL is essential to smart-city applications such as traffic monitoring and waste-management optimization (Sharma et al., 2021).

Regardless of the specific application being considered, it is important to highlight one key difference between the horizontal and vertical settings. In horizontal FL, the local datasets of different clients typically come from similar or related distributions, so training a single shared global model is an effective approach.[1] In contrast, in VFL, the local datasets correspond to distinct feature spaces with inherently different distributions. This makes the use of a single shared model unsuitable for VFL, which requires a fundamentally different approach.

To train VFL models without sharing the local datasets, split neural networks (Ceballos et al., 2020) are typically considered. In split neural networks, each client k has a local *representation model* $\boldsymbol{f}_k$, parameterized by $\boldsymbol{\theta}_{\boldsymbol{f}_k}$, which extracts an (often lower-dimensional) representation of the original local data that is well-suited for the target task. These representations are then sent to an entity (a server or one of the clients) which uses them as input to a *fusion model* g, parameterized by $\boldsymbol{\theta}_g$. We illustrate this in Figure 1.3.

We therefore want to train a split neural network *predictor* h that corresponds to the composition of the fusion model g and the representation models $\{\boldsymbol{f}_k\}$. To learn the parameters $\boldsymbol{\theta} \coloneqq (\boldsymbol{\theta}_{\boldsymbol{f}_1}, \ldots, \boldsymbol{\theta}_{\boldsymbol{f}_K}, \boldsymbol{\theta}_g)$ of such a predictor, we can solve the following

[1] Although standard FL methods, such as `FedAvg` (McMahan et al., 2017), learn a single global model, personalized FL approaches tailor different models for different clients (Smith et al., 2017). However, these methods typically still rely on a shared backbone, such as joint pretraining or shared representation layers.

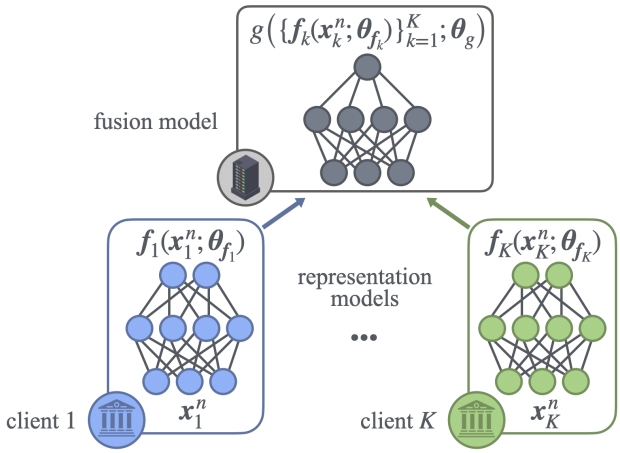


Figure 1.3: An illustration of a split neural network.

optimization problem:

$$\min_{\boldsymbol{\theta}} \quad \left\{ \mathcal{L}(\boldsymbol{\theta}) \coloneqq \frac{1}{N} \sum_{n=1}^{N} \ell_n(h(\boldsymbol{x}^n;\boldsymbol{\theta})) \right\} \quad \text{with} \quad h(\boldsymbol{x}^n;\boldsymbol{\theta}) \coloneqq g\left(\{\boldsymbol{f}_k(\boldsymbol{x}_k^n;\boldsymbol{\theta}_{\boldsymbol{f}_k})\}_{k=1}^K ; \boldsymbol{\theta}_g \right), \tag{1.1}$$

where ℓ_n denotes the loss function for sample n. This loss function incorporates the label of sample n, which we assume is known at least to the entity (client or server) holding the fusion model. For simplicity, the rest of this chapter assumes that the fusion model is hosted on a server.

The optimization problem above is typically tackled using iterative methods. For simplicity, let us focus on the most straightforward method to train split neural networks in VFL, which we will henceforth refer to as standard VFL (`SVFL`): a full-batch approach that is mathematically equivalent to gradient descent.

Recall that the gradient descent update is as follows:

$$\forall t \geq 0\colon \quad \boldsymbol{\theta}^{t+1} \;=\; \boldsymbol{\theta}^t \;-\; \eta\, \nabla \mathcal{L}\left(\boldsymbol{\theta}^t\right),$$

where η is the stepsize. We will now describe an iteration of `SVFL` is as follows.

- **Forward pass:** Each client k performs a forward pass over its representation model and then sends representations $\{\boldsymbol{f}_k(\boldsymbol{x}_k^n;\boldsymbol{\theta}_{\boldsymbol{f}_k})\colon \forall n\}$ to the server (uplink). The server then aggregates the representations of all clients, using them as input to the fusion model g and completes the forward pass.

- **Backward pass:** Then, the server performs a backward pass over the fusion model

and sends the derivative of the loss with respect to the representations back to the clients (downlink), which complete the backward pass locally. This concludes back-propagation, thus allowing for a gradient-based update of the model parameters.

This process is repeated until convergence. We will now go over challenges with `SVFL`.

Communication efficiency challenges. Such iterative methods require numerous rounds of communication between the clients and the server. Given the central role of the server (see Figure 1.1), this strains its limited bandwidth, often leading to increased latency, which can significantly slow down training and become a bottleneck (Liu et al., 2024).

Note that communication efficiency is also a challenge in horizontal FL, as frequent exchanges of model parameters or gradients can lead to a similar bottleneck (Lian et al., 2017). In fact, to address this, a plethora of communication-efficient methods for horizontal FL have been proposed. Some of these techniques have served as inspiration for communication-efficient VFL methods; namely, the use of multiple local updates between rounds of communication (McMahan et al., 2017), asynchronous updates (Xie et al., 2019), and lossy compression (Alistarh et al., 2017) in horizontal FL have inspired VFL methods employing analogous approaches (Liu et al., 2022b; Chen et al., 2020; Castiglia et al., 2022).

However, given that in VFL the communicated objects are not gradients (or model parameters), but rather representations (in the uplink) and derivatives with respect to those representations (in the downlink), these VFL methods are *not* a direct application of their horizontal FL counterparts to the VFL setting. To better understand this, note that, if, for the sake of communication efficiency, the gradient being communicated in horizontal FL is approximated, the approximation error affects the gradient-based update vector directly. In contrast, the error induced by an approximation of the representations sent in VFL will undergo a nonlinear transformation before affecting the update vector. This illustrates how employing techniques to improve communication efficiency in VFL is fundamentally different from employing them in horizontal FL. Therefore, although horizontal FL literature offers valuable insights into communication efficiency in FL, the unique characteristics of the VFL setting demand dedicated research on communication-efficient methods in this setup.

Data efficiency challenges. We observe from Problem (1.1) that, to compute the loss for even a single sample n, we need the blocks of features observed by all K clients, as the loss depends on the output of the predictor, $h(\boldsymbol{x}^n; \boldsymbol{\theta})$, where $\mathbf{x}^n = (\mathbf{x}_1^n, \ldots, \mathbf{x}_K^n)$. Therefore,

this split neural network approach requires the observations of all clients to be present in both the training and test datasets.[2] Most VFL literature thus assumes that every client observes its partition during both training and inference.

However, this assumption rarely holds in practice. For instance, in the earlier example of an online retailer and a social media platform with shared users, each company is also likely to have unique users. One naive way to avoid having to deal with such incomplete samples is to simply disregard them, and instead consider only the samples for which all clients observe their partitions. Yet, wasting incomplete samples during training harms generalization, and not supporting them during inference limits the utility of the model. Moreover, if any client leaves the federation after training, its partition becomes permanently unavailable, rendering the learned model unusable. Therefore, missing feature blocks are a key challenge limiting the applicability of VFL in real-world scenarios.

Chapter organization. The rest of this chapter is organized as follows: from Section 1.1 to Section 1.3, we provide an overview of the contributions of this thesis to efficient vertical federated learning; and in Section 1.4, we go over the organization for the rest of the thesis, introduce some notation that is used throughout it, and present definitions underlying some of the assumptions made throughout the thesis.

1.1 Compressed VFL

Lossy compression is a common technique for reducing communication overhead. When employing compression, rather than transmitting an object (for example, a vector) in full, we first apply a compression operator that maps the original object to a surrogate object of the same dimension that requires fewer bits to be transmitted. The receiver then uses this compressed approximation in place of the original. Thus, lossy compression can help mitigate the aforementioned communication bottleneck in VFL by lowering the cost of each communication at the expense of some information loss.

In the following discussion, we examine communication costs in uncompressed VFL and explore how lossy compression can reduce them. This discussion focuses on uplink communications, given that:

1. **Uplink communications are typically the bottleneck:** In practice, the server can often aggregate the downlink information for the clients into a single object,

[2]This contrasts with horizontal FL, where the local data of a client suffices to perform inference, as well as to compute a mini-batch gradient estimate and thus update the model.

collapsed along the client dimension, and broadcast it to all clients. Because this is a one-to-many transmission, downlink communications then lose the dependency on the number of clients, making the uplink the true bottleneck.

2. **This keeps this discussion simple:** The core ideas behind uplink and downlink compression are similar, yet, in VFL, downlink communications are more nuanced than uplink ones. This is because, in downlink communications, the server can send either the derivative of the loss with respect to client representations (as mentioned above) or, alternatively, the representations themselves and the server model.[3]

The points above will be made clearer in Chapter 2.

Communications in uncompressed VFL. In the standard, uncompressed VFL method described earlier, `SVFL`, each client k sends its representations $\{\boldsymbol{f}_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_{\boldsymbol{f}_k}) \colon \forall n\}$ to the server at each iteration. For simplicity, let us assume that all the representations extracted by different clients share the same dimension E, that is,

$$\forall (k, n) \in \mathcal{K} \times \{1, \dots, N\} \colon \quad \boldsymbol{f}_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_{\boldsymbol{f}_k}) \in \mathbb{R}^E.$$

It follows that, at every iteration, the server receives from each of the K clients an uplink transmission of size E for each of the N samples. As a result, the uplink communication cost per iteration in `SVFL` is given by:

$$(\texttt{SVFL} \text{ uplink cost per iteration}) \quad NEK.$$

To reduce this uplink cost, and thus alleviate the communication bottleneck at the server, we can resort to lossy compression.

Prior work on compressed VFL. Although previous works had already employed lossy compression in VFL (Li et al., 2020b; Khan et al., 2022), Castiglia et al. (2022) were, to the best of our knowledge, the first to propose a communication-compressed VFL method with convergence guarantees: compressed-VFL (`CVFL`). We will now go over the communication cost per iteration of `CVFL`.

The exact communication cost per iteration of `CVFL` depends on the specific compressor chosen. Therefore, for concreteness, this discussion will focus on the top-k sparsification compressor, as an example.

[3]The object sent in the downlink depends on whether the optimization method uses multiple local updates and on whether all clients know the labels.

Top-k sparsification zeroes out all entries in the communicated object except for the k largest ones (in absolute value). To communicate a top-k compressed object, we therefore need only send these k values and their indices, rather than the whole object inputted to the compressor. In the following discussion we ignore the communication of the indices for simplicity.[4] (We define top-k sparsification formally in Chapter 2.)

In `CVFL`, to employ top-k sparsification in the communication-compressed training of a split neural network, each client sparsifies its NE-dimensional set of representations before sending it to the server, keeping only the k largest values to be transmitted. This reduces the uplink communication of each client from NE scalars to just k, substantially reducing the bandwidth requirements on the server. Thus, taking all K clients into account, the uplink communication cost per iteration in `CVFL` is given by:

$$(\texttt{CVFL uplink cost per iteration}) \quad kK.$$

More generally, when compression—not necessarily top-k sparsification, in specific—is applied, the NE term in the `SVFL` uplink cost per iteration is replaced by a smaller quantity whose exact value depends on the chosen compression mechanism.

Challenges in communication-compressed VFL. Although `CVFL` can significantly reduce the uplink cost per iteration of `SVFL`, its direct use of the compressed object as a surrogate for the original object introduces error throughout training by replacing representations with approximations. This compression error leads to:

1. A slower rate of convergence per iteration, compared to that of `SVFL`.

2. Inexact convergence, unless a vanishing amount of compression is employed.

We will now elaborate on these two points, starting with a discussion on the uplink communication complexity of `SVFL`—the baseline to beat.

Recalling that `SVFL` is mathematically equivalent to gradient descent, we know that, for L-smooth functions, it converges in expected gradient squared norm at a rate of $\mathcal{O}(1/T)$. Therefore, it follows from the `SVFL` uplink cost per iteration presented above that the total uplink communication cost for `SVFL` to reach an error level $\delta > 0$ is given by:

$$(\texttt{SVFL uplink communication complexity}) \quad NEK \cdot \mathcal{O}(1/\delta).$$

[4]This is a reasonable approximation, since the indices typically require fewer bits to be communicated than the values in the object being compressed. Further, even if they required the same number of bits, this would only change the communication complexity by a constant factor of 2.

Table 1.1: Comparison of the uplink communication complexity required to achieve error level $\delta > 0$ when employing top-k sparsification and full-batch gradients, between prior art and our method.

Algorithm	SVFL	CVFL (Castiglia et al., 2022)	EF-VFL **(ours)**
Uplink cost	$NEK \cdot \mathcal{O}(1/\delta)$	$\min\{k \cdot \mathcal{O}(1/\sqrt{\delta}),\, NE\} \cdot K \cdot \mathcal{O}(1/\delta^2)$	$kK \cdot \mathcal{O}(1/\delta)$

In contrast, for CVFL to achieve a similar convergence, it requires $\mathcal{O}(1/\delta^2)$ iterations, rather than $\mathcal{O}(1/\delta)$. This is because CVFL does not converge at a $\mathcal{O}(1/T)$ rate, but rather at a rate of $\mathcal{O}(1/\sqrt{T})$ for a $\mathcal{O}(1/\sqrt{T})$ stepsize and a $\mathcal{O}(1/\sqrt{T})$ compression error. Further, to achieve this vanishing compression error, the amount of compression employed must decrease as training progresses. For top-k sparsification, this corresponds to gradually increasing the number of nonzero entries in the compressed object. Therefore, the total communication complexity for CVFL is given by:

$$(\texttt{CVFL uplink communication complexity}) \quad \min\{k \cdot \mathcal{O}(1/\sqrt{\delta}), NE\} \cdot K \cdot \mathcal{O}(1/\delta^2),$$

where the $\min\{k \cdot \mathcal{O}(1/\sqrt{\delta}), NE\}$ term replacing NE captures the decreasing amount of compression throughout training, which corresponds to an increase number of nonzero entries up to a maximum size—the size of the uncompressed object, NE.

In practice, this decreasing amount of compression as training progresses means that, while CVFL does reduce the total uplink communication cost, the uplink communications increase throughout training. This makes CVFL unsuitable for applications with strict per-round communication limitations, such as hard bandwidth constraints. Therefore, existing work lacks a communication-compressed VFL method that preserves the convergence rate of uncompressed VFL methods and does not require a decreasing amount of compression as training progresses.

Our contributions to compressed VFL. Our contribution is motivated by the insight that the compression used in CVFL—namely, compressing the representations at each iteration independently before using them as surrogates for the original, uncompressed representations—does not exploit the fact that these representations are in fact *not* independent. To address this, in Chapter 2, we propose error feedback compressed vertical federated learning (EF-VFL), which leverages the history of past representations and their compressed counterparts to build a more accurate surrogate without any additional communication overhead.

More precisely, `EF-VFL` achieves this by maintaining an auxiliary vector at both sender and receiver and updating it at each step using feedback from the compression of past representations. By refining this auxiliary vector as a surrogate for the true representation, `EF-VFL` harnesses historical information without any additional communication cost. This allows for faster and more stable convergence.

In particular, `EF-VFL` achieves the same, faster convergence rate of the uncompressed `SVFL` method, $\mathcal{O}(1/T)$, without requiring a vanishing stepsize or diminishing compression. This stands in contrast to `CVFL`, which achieves only an $\mathcal{O}(1/\sqrt{T})$ convergence rate under a $\mathcal{O}(1/\sqrt{T})$ stepsize and $\mathcal{O}(1/\sqrt{T})$ compression.

Our main contributions are as follows.

- We propose `EF-VFL`, a method that leverages an error feedback technique to improve the speed and stability of communication-compressed training in VFL.
- We show that, for sufficiently large batch sizes, our method achieves a convergence rate of $\mathcal{O}(1/T)$ for nonconvex objectives, improving over the state-of-the-art $\mathcal{O}(1/\sqrt{T})$ rate of Castiglia et al. (2022) and matching the rate of uncompressed VFL.
- Empirical results demonstrate that our method significantly outperforms the baselines in communication efficiency.

We summarize the comparison in communication complexity with prior art in Table 1.1.

1.2 VFL with missing features

Most VFL methods require all clients to observe their feature block for a given sample in order to compute its loss. As a result, such methods cannot handle incomplete samples—where only a subset of the clients observe their partition—neither during training nor during inference. Yet, the assumption that all feature blocks are observed rarely holds in practice, as seen earlier, in our example of a set of organizations sharing a set of users, which will typically also have unique users. In Figure 1.4a, we illustrate an example dataset where $K = 3$ clients observe their feature blocks with an (independent) probability of 0.5.

A straightforward (yet naive) approach to handling missing feature blocks is to simply discard any incomplete samples, restricting the dataset to the samples where every client observes its partition. This allows us to then employ standard VFL methods, but comes at the cost of wasting a potentially large number of samples. We illustrate this in Figure 1.4b,

(a) Data availability. (b) `SVFL` data use. (c) Local data use (1). (d) Our data use (1).

Figure 1.4: An illustration of an example data availability and the corresponding data usage for three key methods. In Figure 1.4a, we show the availability of a dataset with $K = 3$ blocks of features, each with a 0.5 probability of being observed. In Figure 1.4b, Figure 1.4c, and Figure 1.4d, we illustrate the dataset usage and waste by standard VFL (`SVFL`), a local approach, and our method, respectively. `SVFL` trains a single predictor, while the local approach and our method train different predictors at different clients (we show the data usage for client 1 for both of these methods).

where this approach would disregard $1 - 0.5^3 = 87.5\%$ of the original training and test datasets.

This waste of data is highly undesirable: limiting training to fully observed samples harms generalization, and not supporting incomplete samples during inference reduces the availability and utility of the model. Moreover, since it does not support missing feature blocks during inference, this approach is not robust to any client leaving the federation after training, as its partition would become permanently unavailable.

This vulnerability of VFL to incomplete samples could tempt clients to abandon collaboration altogether, and instead train local predictors using only their own features. Such a local approach allows each client to utilize all its available observations for training and solves the issue of robustness against missing blocks during inference, as each client k can then perform inference independently of other clients. We illustrate this on Figure 1.4c. However, such independent local predictors do not make use of the features observed at other clients.

Prior work on VFL with missing features. To handle missing features in the training data, some approaches expand the dataset, filling the missing features before collaborative training (Kang et al., 2022), while others use their partition of partially-observed samples

for local representation learning but exclude them from collaboration (He et al., 2024). These methods train a joint predictor, as in SVFL, yet they can outperform SVFL by taking advantage of incomplete training samples.

On the other hand, to improve robustness against missing blocks during inference, each client can train a local predictor using only its own features, avoiding collaboration altogether, as mentioned above. This approach can also handle incomplete samples during training, as illustrated in Figure 1.4c. Recent works have proposed less naive approaches to such train local predictors, employing collaborative techniques such as knowledge distillation (Huang et al., 2023) and data augmentation (Gao et al., 2024) to train local predictors without wasting all the data at other clients. These methods are thus robust to missing features during inference, unlike SVFL and the methods mentioned in the paragraph above, which train joint predictors.

Challenges in VFL with missing features. Similarly to SVFL and the naive local approach illustrated in Figure 1.4b and Figure 1.4c, the prior work on VFL with missing features mentioned above still face challenges in the presence of incomplete samples. In particular, joint predictors still require complete samples during inference, and local predictors cannot utilize the feature blocks of other clients, if available. The former harms the availability of the model, as mentioned above, while the latter wastes features, leading to a reduced predictive power. Further, in both cases, these existing works resort to multiple training stages and auxiliary modules, making them more complex than SVFL.

Therefore, there is a gap in the literature in that we lack a VFL method that is robust to missing features during both training and inference without wasting data, leveraging all the available data. That is, without either dropping incomplete samples or ignoring features observed by other clients.

Our contributions to VFL with missing features. To address this, in Chapter 3, we propose LASER-VFL (**L**everaging **A**ll **S**amples **E**fficiently for **R**eal-world **VFL**), a novel method that enables the training of and inference using split neural network-based models in the presence of arbitrary sets of available feature blocks, without wasting data. That is, LASER-VFL uses any and all available blocks of features. To the best of our knowledge, this is the first method to achieve this.

Our approach avoids the multi-stage pipelines that are common in this area, providing a simple yet effective solution that leverages the sharing of model parameters and a task-sampling mechanism to train a family of predictors. By fully utilizing all data, as illustrated

Table 1.2: A preview of our results. We present the F_1-score$\times$100 for a mortality prediction task on the MIMIC-IV clinical dataset.

Method	$p_{\text{miss}} = 0.0$	$p_{\text{miss}} = 0.5$
`Local`	74.9	73.0
`Standard VFL`	91.6	51.8
`LASER-VFL` **(ours)**	**92.0**	**82.0**

in Figure 1.4d,[5] `LASER-VFL` achieves significant improvements in performance, as seen in Table 1.2, where we present a preview of our results for a mortality prediction task using the MIMIC-IV dataset, when the probability of each block of features missing, p_{miss}, is in $\{0.0, 0.5\}$, for both data and test data. Remarkably, our approach even surpasses standard VFL when all samples are fully observed. We attribute this to a dropout-like regularization effect introduced by the task sampling mechanism.

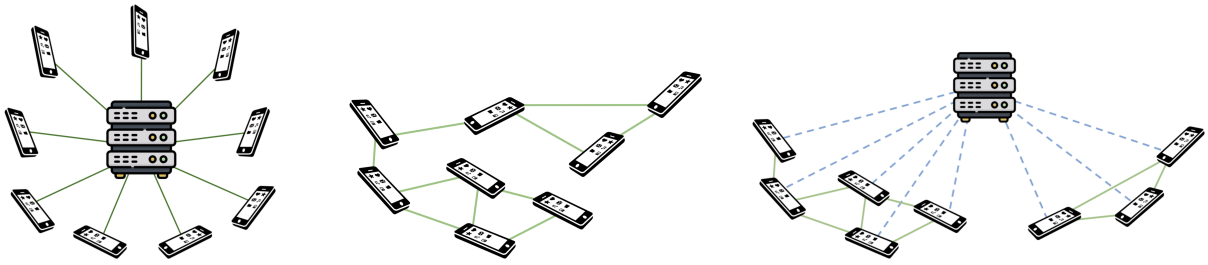
The main contributions of this work are as follows.

- We propose `LASER-VFL`, a simple, hyperparameter-free, and efficient VFL method that is flexible to varying sets of feature blocks during training and inference. To the best of our knowledge, this is the first method to achieve such flexibility without wasting either training or test data.
- We show that `LASER-VFL` converges at a $\mathcal{O}(1/\sqrt{T})$ rate for nonconvex objectives. Furthermore, under the Polyak-Łojasiewicz (PL) inequality, it achieves linear convergence to a neighborhood around the optimum.
- Numerical experiments show that `LASER-VFL` consistently outperforms baselines across multiple datasets and varying data availability patterns. It demonstrates superior robustness to missing features and, notably, when all features are available, it still outperforms even standard VFL.

1.3 Semi-decentralized VFL

FL algorithms typically consider a client-server communication scheme, where the clients communicate only with a central server. That is, the communication graph forms a star

[5]Note that, in `LASER-VFL`, the feature blocks that client 1 does not use for inference correspond to samples for which client 1 does not observe its own feature block. In our running example of an online retailer and a social media company sharing users, this happens when the retailer does not make a prediction for someone who is not one of its users.



(a) Client-server scheme. (b) Fully decentralized scheme. (c) Semi-decentralized scheme.

Figure 1.5: An illustration of different communication schemes that can be considered in federated learning. This section and Chapter 4 explore the semi-decentralized scheme.

topology with the server at its center, as seen in Figure 1.5a. As discussed earlier, this scheme is prone to bandwidth bottlenecks at the server, nonetheless, it remains a good fit for some cross-silo VFL applications where we have a small number of clients with reliable connectivity. However, when we have a large number of clients, namely in the cross-device setting, this strains server bandwidth, causing significant latency (Lian et al., 2017). In this section, rather than considering the use of lossy compression as a way to alleviate this bottleneck, as discussed in Section 1.1, we explore an orthogonal strategy: the use of alternative communication schemes that mitigate server bandwidth strain by also leveraging direct client-client communications.

Decentralized FL. To mitigate this communication bottleneck at the server (or when a server is simply not available), we can resort to a (fully) decentralized approach, where the clients communicate directly with each other, dispensing with the server. In this decentralized scheme, the communication graph is only required to be connected; its topology is otherwise arbitrary, as illustrated in Figure 1.5b.

Decentralized optimization has been widely studied. Earlier work was motivated by applications in wireless sensor networks and multi-agent control (Nedic and Ozdaglar, 2009; Duchi et al., 2012), while many recent efforts have been driven by FL (Koloskova et al., 2020; Li et al., 2020a; Zhao et al., 2022). Although decentralized methods mitigate the bandwidth bottleneck of client-server setups by distributing the communication load across the network (and avoid having a single point of failure at the server), such methods can incur high total communication costs, due to the stale information used during training.

To achieve a lower communication cost than the more common consensus-based decentralized methods (Nedic and Ozdaglar, 2009; Duchi et al., 2012), we can employ token methods (Nedic and Bertsekas, 2001; Hendrikx, 2023). In token methods, a *token* carrying the current model estimate performs a random walk over a communication graph, under-

going local updates. This approach avoids the stale updates of consensus methods, yet it sacrifices their parallelism, which can lead to slower convergence in poorly connected networks (Hendrikx, 2023). Multi-token methods (Ye et al., 2020; Chen et al., 2022) navigate this trade-off by running multiple tokens simultaneously and periodically combining them, drawing from the idea that performing multiple random walks in parallel leads to a linear speed-up in cover time (Alon et al., 2008).

Challenges in traditional communication schemes. As we just discussed, decentralized schemes can mitigate the server communication bottleneck seen in client-server schemes. However, while they are well-suited for applications with well-connected networks or those lacking client-server links, they often converge slowly in sparse and large networks (Nedic et al., 2018), even failing to converge if the network is disconnected.

Therefore, there is a trade-off when choosing between client-server and decentralized schemes: client-server setups allow for the fast spreading of information among clients, but often create a bandwidth bottlenecks at the server, whereas decentralized setups alleviate that bottleneck at the expense of slower convergence on large, sparse graphs.

VFL methods in both the client-server scheme (Liu et al., 2022b; Castiglia et al., 2022) and the decentralized scheme (He et al., 2018; Alghunaim et al., 2021) have been proposed. However, the duality of restricting our choice to these two traditional communication schemes does not allow us to choose a flexible degree of dependency on the server. That is, existing work does not allow us to operate along the spectrum between these two standard communication schemes to suit applications where neither extreme is ideal.

Semi-decentralized FL. To leverage the different strengths that make client-server and decentralized schemes well-suited for different applications, the semi-decentralized (SD) scheme (Lin et al., 2021) employs both client-client and client-server links, as illustrated in Figure 1.5c. In this scheme, client-client communications enable the flow of information without straining the limited bandwidth of the server, while client-server links accelerate convergence in poorly connected networks. In fact, even when the client-client communication graph is disconnected, as long as one client in each cluster can communicate with the server, convergence is still attainable.

Our contributions to semi-decentralized VFL. Motivated by this, in Chapter 4, we address this gap between client-server and decentralized schemes, focusing on the VFL setup. We propose multi-token coordinate descent (`MTCD`), a flexible semi-decentralized VFL method. `MTCD` can exploit both client-server and client-client links.

By selecting appropriate hyperparameters, MTCD recovers the client-sever and decentralized schemes as special cases—in fact, its decentralized instance is itself a novel method of independent interest. Yet, by controlling the degree of dependency on client-server links, MTCD can also explore a spectrum of schemes ranging from client-server to decentralized.

To capture the aforementioned drawbacks of the client-server scheme succinctly, we model the relative impact of using client-server versus client-client links as the ratio of their "costs", which depends on the application. This allows us to demonstrate, both analytically and empirically, that by tuning the degree of dependency on the server, the semi-decentralized instances of MTCD can outperform both client-server and decentralized approaches across a range of applications where neither extreme is ideal.

Our main contributions are as follows.

- We introduce MTCD, a flexible VFL method that subsumes the client-server and decentralized schemes, and further extends it to the SD scheme. To the best of our knowledge, MTCD is the first token method and first SD method for VFL, as well as the first SD multi-token method overall.
- We show that, for sufficiently large batch sizes, MTCD converges at an $\mathcal{O}(1/T)$ rate for nonconvex objectives when the tokens roam over disjoint sets of clients. If the objective is further convex, we achieve the same rate while allowing for overlapping token trajectories.
- We show, both analytically and empirically, that by adjusting the degree of server dependence along the spectrum between client-server and decentralized schemes, MTCD can outperform both schemes across a range of applications.

1.4 Thesis organization, notation, and assumptions

1.4.1 Thesis organization

The rest of this thesis is organized as follows: in Chapter 2, we introduce EF-VFL, a communication-compressed method for VFL, provide convergence guarantees for it, and present empirical results; in Chapter 3, we propose LASER-VFL, a VFL method capable handling missing features during both training and inference, we prove the convergence of LASER-VFL, and empirically observe its performance; in Chapter 4, we propose MTCD, a semi-decentralized multi-token method for VFL, analyze its convergence, and run numerical experiments; and, in Chapter 5, we conclude the thesis and discuss future directions.

1.4.2 Thesis notation

The $\mathcal{L}(\boldsymbol{\theta})$ notation introduced in Problem (1.1) is useful for discussing the training of split neural networks, as it emphasizes that the dataset is fixed, while the model parameters are our optimization variable. Similarly, we now introduce some notation that will be useful to keep more of the split neural network structure in Problem (1.1) while still abstracting away the dependency on the data.

In particular, we define $\boldsymbol{F}_k(\boldsymbol{\theta}_k)$ as the matrix where row n corresponds to the representation extracted by client k for its feature block of sample n, $\boldsymbol{f}_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_{\boldsymbol{f}_k})$, for $k \in \mathcal{K}$. It will also be useful to define

$$\boldsymbol{F}(\boldsymbol{\theta}) \coloneqq (\boldsymbol{F}_0(\boldsymbol{\theta}_0), \boldsymbol{F}_1(\boldsymbol{\theta}_1), \ldots, \boldsymbol{F}_K(\boldsymbol{\theta}_K)),$$

where $\boldsymbol{F}_0(\boldsymbol{\theta}_0) \coloneqq \boldsymbol{\theta}_0$.

We can similarly omit the dependency of the fusion model and of the loss function on the data. Further, given that the loss function for each sample n, ℓ_n, is not parameterized, when tailoring our notation to the context of optimization, we can also abstract the loss away, combining it with the predictor. Thus, we define ϕ as the function that takes as input $\boldsymbol{F}(\boldsymbol{\theta})$ and outputs the average across all samples of the composition of the loss and the prediction for each sample n. This allows us to write:

$$\mathcal{L}(\boldsymbol{\theta}) = \phi(\boldsymbol{F}(\boldsymbol{\theta})).$$

Mini-batch notation. It is sometimes useful to resort to the following notation for the loss for a single sample:

$$\mathcal{L}_n(\boldsymbol{\theta}) \coloneqq \ell_n(h(\boldsymbol{x}^n; \boldsymbol{\theta})),$$

which allows us to write the objective as

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_n(\boldsymbol{\theta}).$$

Further, when using mini-batch approximations of the objective function, we denote by $\mathcal{B} \subseteq \{1, \ldots, N\}$ the set of mini-batch indices, of size $|\mathcal{B}| \coloneqq B$, and define the mini-batch approximation of our objective function as:

$$\mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta}) \coloneqq \frac{1}{B} \sum_{n \in \mathcal{B}} \mathcal{L}_n(\boldsymbol{\theta}).$$

Similarly, we denote the mini-batch counterparts of $\boldsymbol{F}$, $\boldsymbol{F}_k$, $\boldsymbol{X}$, $\boldsymbol{X}_k$, and ϕ as $\boldsymbol{F}_{\mathcal{B}}$, $\boldsymbol{F}_k^{\mathcal{B}}$, $\boldsymbol{X}^{\mathcal{B}^t}$, $\boldsymbol{X}_k^{\mathcal{B}^t}$, and $\phi_{\mathcal{B}}$, respectively.

1.4.3 Thesis assumptions

We resort to the following definitions when making assumptions throughout the thesis.

Definition 1 (Finite infimum)**.** *A function f has a finite infimum $f^\star$ if:*

$$f^\star \coloneqq \inf_{\boldsymbol{x}} f(\boldsymbol{x}) > -\infty. \tag{D1}$$

Definition 2 (L-smoothness)**.** *A function f is L-smooth if it is differentiable and there exists a positive constant L such that:*

$$\forall \boldsymbol{x}, \boldsymbol{y}\colon \quad \|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|. \tag{D2}$$

Definition 3 (Unbiasedness)**.** *A function $\widetilde{f}$ is an unbiased estimate of function f if:*

$$\forall \boldsymbol{x}\colon \quad \mathbb{E}\left[\widetilde{f}(\boldsymbol{x})\right] = f(\boldsymbol{x}). \tag{D3}$$

Definition 4 (Bounded variance)**.** *A function $\widetilde{f}$ has a bounded variance if there exists a positive constant σ such that:*

$$\forall \boldsymbol{x}\colon \quad \mathbb{E}\left\|\widetilde{f}(\boldsymbol{x}) - \mathbb{E}\left[\widetilde{f}(\boldsymbol{x})\right]\right\|^2 \leq \sigma^2. \tag{D4}$$

We now define the Polyak-Łojasiewicz (PL) inequality (Polyak, 1963), which lower-bounds the gradient norm by the suboptimality at every point. Thus, the gradient can only be small near minima and increases proportionally as the function value rises above its minimum, ruling out flat regions away from the optimum and ensuring that any critical point are global minima. Further, since the gradient cannot decrease too fast with the suboptimality, gradient-based methods achieve global linear convergence, all without assuming convexity, as the PL condition does not require a unique minimizer.

Definition 5 (PL inequality)**.** *A function f satisfies the PL inequality if there exists a positive constant μ such that:*

$$\forall \boldsymbol{x}\colon \quad \|\nabla f(\boldsymbol{x})\|^2 \geq 2\mu(f(\boldsymbol{x}) - f^\star). \tag{D5}$$

In addition to strongly convex objectives often seen in classical machine learning, which satisfy the (weaker) PL condition, overparameterized deep networks, whose local minima are all global (Nguyen et al., 2018), obey a (slightly modified) PL inequality over most of the parameter space (Liu et al., 2022a). Further, it has recently been shown that the PL inequality holds for certain in-context learning problems with transformers (Yang et al., 2024).

Chapter 2

Compressed VFL

In this chapter, we address the server-bandwidth bottleneck in client-server VFL. To mitigate this problem, we employ lossy compression, where the sender replaces each object with an approximation that takes fewer bits to transmit, reducing communication cost at the expense of some information loss.

As noted in Section 1.1, in the standard (uncompressed) method for solving Problem (1.1), `SVFL`, the server receives NE scalars from each of the K clients per iteration. Prior work on communication-compressed VFL, `CVFL` (Castiglia et al., 2022), reduced this NE cost to a smaller, compressor-dependent quantity, thus reducing the uplink cost per iteration. Yet, by directly using the compressed representation as a surrogate for the original representation, `CVFL` incurs an approximation error that **1)** prevents exact convergence unless the amount of compression vanishes over training, and **2)** slows down convergence, from the $\mathcal{O}(1/T)$ rate of `SVFL` to a $\mathcal{O}(1/\sqrt{T})$ rate for a $\mathcal{O}(1/\sqrt{T})$ stepsize and $\mathcal{O}(1/\sqrt{T})$ compression. This vanishing compression means that, while `CVFL` lowers total uplink communication, the growing per-round upload cost as training proceeds prevents `CVFL` from meeting strict per-round communication limits. We thus lack a communication-compressed VFL method matching the convergence rate of `SVFL` under nonvanishing compression.

Motivated by the observation that `CVFL` compresses the representations at each iteration independently, without leveraging the fact that these representations are not independent, we introduce error feedback compressed vertical federated learning (`EF-VFL`). `EF-VFL` exploits the history of past representations and their compression to construct a more accurate surrogate, all without incurring additional communication overhead. This allows `EF-VFL` to achieve the same $\mathcal{O}(1/T)$ convergence rate as `SVFL`, without requiring a vanishing stepsize or diminishing compression.

This chapter is based on our previous publication (Valdeira et al., 2025b).

2.1 Related work

Communication-efficient FL. The aforementioned communication bottleneck in FL (Lian et al., 2017) makes communication-efficient methods a particularly active area of research. FL methods often employ multiple local updates between rounds of communication, use only a subset of the clients at a time (McMahan et al., 2017), or even update the global model asynchronously (Xie et al., 2019). Another line of research considers fully decentralized methods (Lian et al., 2017), dispensing with the server and, instead, exploiting direct communications between clients. This can alleviate the strain on the bandwidth ensuing from the centralized role of the server. Another popular technique is lossy compression, which is the focus of this work. In both FL and, more generally, in the broader area of distributed optimization (Chen and Sayed, 2012; Mota et al., 2013), communication-compressed methods have recently received significant attention (Amiri and Gündüz, 2020; Du et al., 2020; Shlezinger et al., 2020; Nassif et al., 2023). Recent works in FL have also studied the use compression mechanisms in combination with privacy mechanisms (Li et al., 2022b; Li and Chi, 2025)

Communication-compressed optimization methods can exploit different families of compressors. A popular choice is the family of *unbiased* compressors (Alistarh et al., 2017), which is appealing in that its properties facilitate the theoretical analysis of the resulting methods. Yet, some widely adopted compressors do not belong to this class, such as top-k sparsification (Alistarh et al., 2018; Stich et al., 2018) and deterministic rounding (Sapio et al., 2021). Thus, the broader family of *contractive* compressors (Beznosikov et al., 2023) has recently attracted a lot of attention. Yet, methods employing contractive compressors for direct compression are often prone to instability or even divergence (Beznosikov et al., 2023). To address this, error-feedback techniques were initially proposed as a heuristic (Seide et al., 2014). More recently, however, there has been significant progress in our theoretical understanding of applying error feedback to gradient compression (Stich et al., 2018; Alistarh et al., 2018; Richtárik et al., 2021). In the horizontal setting, some works have combined error feedback compression with the aforementioned communication-efficient techniques, leading, for example, to communication-compressed fully decentralized methods (Koloskova et al., 2019; Zhao et al., 2022).

Communication-efficient VFL. To mitigate the communication bottleneck in VFL, techniques akin to those used in horizontal FL are often employed. In particular, Liu et al. (2022b) performed multiple local updates between communication rounds, Chen et al. (2020) updated the local models asynchronously, and Ying et al. (2018) proposed a fully

decentralized approach.

In this chapter, we focus on communication-compressed methods tailored to VFL. As mentioned in Chapter 1, most work in communication-compressed methods focuses on gradient compression and thus does not apply directly to VFL, where representation compression is employed. Nevertheless, a few empirical works on VFL have recently employed compression. Namely, Khan et al. (2022) compressed the local data before sending it to the server, where the model is trained, and Li et al. (2020b) proposed an asynchronous method with bidirectional (sparse) gradient compression. Nevertheless, Castiglia et al. (2022), where direct compression is used, is the only work on compressed VFL with theoretical guarantees. For a more a detailed discussion on VFL, see Yang et al. (2023); Liu et al. (2024).

2.2 Preliminaries

We now define the class of contractive compressors, which we consider throughout this chapter.

Definition 6 (Contractive compressor)**.** *A map $\mathcal{C}\colon \mathbb{R}^d \mapsto \mathbb{R}^d$ is a contractive compressor if there exists a constant $\alpha \in (0,1]$ such that,*[1]

$$\forall \boldsymbol{v} \in \mathbb{R}^d\colon \quad \mathbb{E}\|\mathcal{C}(\boldsymbol{v}) - \boldsymbol{v}\|^2 \leq (1-\alpha)\|\boldsymbol{v}\|^2, \tag{D6}$$

where the expectation is taken with respect to the (possible) randomness in $\mathcal{C}$.

As explained in Section 2.1, the class of contractive compressors is a broad class of compressors that encompasses many popular compressors. We now present two examples of such popular contractive compressors: top-k sparsification and stochastic quantization.

- Top-k sparsification (Alistarh et al., 2018; Stich et al., 2018) is a map $\mathrm{top}_k\colon \mathbb{R}^d \mapsto \mathbb{R}^d$ defined as

$$\mathrm{top}_k(\boldsymbol{v}) \coloneqq \boldsymbol{v} \odot \boldsymbol{u}(\boldsymbol{v}), \tag{2.1}$$

  where $\odot$ denotes the Hadamard product and $\boldsymbol{u}(\boldsymbol{v})$ is such that its entry i is 1 if v_i is one of the k largest entries of $\boldsymbol{v}$ in absolute value and 0 otherwise. We have that, for top_k, (D6) holds for $\alpha = k/d$.

[1] In Definition 6, we use a common, simplified notation, omitting the randomness in $\mathcal{C}\colon \mathbb{R}^d \times \Omega \to \mathbb{R}^d$. We assume that the randomness ω in $\mathcal{C}(\boldsymbol{v}, \omega)$ at different applications of $\mathcal{C}$ is independent.

- Stochastic quantization (Alistarh et al., 2017) is a map $\mathrm{qsgd}_s\colon \mathbb{R}^d \mapsto \mathbb{R}^d$, with $s > 1$ quantization levels defined as

$$\mathrm{qsgd}_s(\boldsymbol{v}) \coloneqq \frac{\|\boldsymbol{v}\| \cdot \mathrm{sign}(\boldsymbol{v})}{s\tau} \cdot \left\lfloor s\frac{|\boldsymbol{v}|}{\|\boldsymbol{v}\|} + \xi \right\rfloor, \tag{2.2}$$

  where $\tau = 1 + \min\{d/s^2, \sqrt{d}/s\}$ and $\xi \sim \mathcal{U}([0,1]^d)$, where $\mathcal{U}$ denotes the uniform distribution. In practice, we are interested in values of s such that $s = 2^b$, where b is the number of bits. We have that, for qsgd_s, (D6) holds for $\alpha = 1/\tau$.

2.2.1 Error feedback

When transmitting a converging sequence of vectors $\{\boldsymbol{v}^t\}$, error feedback mechanisms can reduce the compression error compared to direct compression. In communication-compressed optimization, this allows for faster and more stable convergence.

In direct compression, each $\boldsymbol{v}^t$ is compressed independently, with $\mathcal{C}(\boldsymbol{v}^t)$ simply replacing $\boldsymbol{v}^t$ at the receiver. In contrast, in error feedback compression, the receiver employs a surrogate for $\boldsymbol{v}^t$ that incorporates information from previous steps $i = 0, \ldots, t-1$. This is achieved by resorting to an auxiliary vector that is stored in memory and updated at each step, leveraging *feedback* from the compression of $\{\boldsymbol{v}^i\colon i = 0, \ldots, t-1\}$ to refine the surrogate for $\boldsymbol{v}^t$.

Earlier communication-compressed optimization methods employing error feedback mechanisms were motivated by sigma-delta modulation (Inose et al., 1962). In these methods, the auxiliary vector accumulated the compression *error* across steps, adding the accumulated error to the current vector $\boldsymbol{v}^t$ before compressing and transmitting it (Seide et al., 2014; Alistarh et al., 2018; Karimireddy et al., 2019).

More recently, a new type of error feedback mechanism has been proposed, where the auxiliary vector $\boldsymbol{s}^t$ tracks $\boldsymbol{v}^t$ directly, rather than the accumulated compression error. This mechanism uses the (compressed) difference between $\boldsymbol{v}^{t+1}$ and $\boldsymbol{s}^t$, that is, the *error* of the surrogate, as the feedback. This approach was first introduced in (Mishchenko et al., 2024) for unbiased compressors and was later extended to the more general class of contractive compressors in EF21 (Richtárik et al., 2021). More formally, in EF21, the surrogate $\boldsymbol{s}^t$ is initialized as $\boldsymbol{s}^0 = \mathcal{C}(\boldsymbol{v}^0)$ and updated recursively as:

$$\forall t \geq 0\colon \quad \boldsymbol{s}^{t+1} = \boldsymbol{s}^t + \mathcal{C}(\boldsymbol{v}^{t+1} - \boldsymbol{s}^t). \tag{2.3}$$

Note that, unlike $\mathcal{C}(\boldsymbol{v}^t)$, $\boldsymbol{s}^t$ is not necessarily in the range of $\mathcal{C}$. For example, if $\mathcal{C}$ uses

sparsification, then the surrogate at the receiver in direct compression, $\mathcal{C}(\boldsymbol{v}^t)$, must be sparse, whereas $\boldsymbol{s}^t$ does not need to be. By continually updating $\boldsymbol{s}^t$ with the compressed error, we ensure that it tracks $\boldsymbol{v}^t$. Moreover, since these updates are compressed, the communication cost remains the same as in direct compression. To maintain consistency, the surrogate $\boldsymbol{s}^t$ is updated at both the sender (client or server) and the receiver. In this chapter, we adopt an EF21-based error feedback mechanism and henceforth refer to it simply as error feedback.

2.2.2 Problem setup

Throughout most of this chapter, we assume that the loss function ℓ_n, which contains the label of sample $\boldsymbol{x}^n$, is known by all the clients and the server. This assumption, known as "relaxed protocol" (Liu et al., 2024), is sometimes made in VFL (Hu et al., 2019; Castiglia et al., 2022, 2024) and has applications, for example, in credit score prediction. We also address the case of private labels, proposing a modified version of our method for that setting in Section 2.3.1.

We let ∇ denote not only the gradient of scalar-valued functions but, more generally, the derivative of a (possibly multidimensional) map, and make the following assumptions:

- $\mathcal{L}$ is $L_{\mathcal{L}}$-smooth (D2) and has a finite infimum $\mathcal{L}^\star$ (D1) and ϕ is L_ϕ-smooth (D2). We let $L \coloneqq \max\{L_{\mathcal{L}}, L_\phi\}$, and thus have that both $\mathcal{L}$ and ϕ are L-smooth.

- The representations $\{\boldsymbol{F}_k\}_{k=0}^K$ have bounded derivatives (D7), as we now define.

  Definition 7 (Bounded derivative)**.** *Map $\boldsymbol{M}\colon \mathbb{R}^{p_1} \mapsto \mathbb{R}^{p_2 \times p_3}$ has a bounded derivative if there exists a positive constant H such that:*

$$\forall \boldsymbol{x} \in \mathbb{R}^{p_1}\colon \quad \|\nabla \boldsymbol{M}(\boldsymbol{x})\| \leq H, \tag{D7}$$

  where $\|\nabla \boldsymbol{M}(\boldsymbol{x})\|$ is the Euclidean norm of the third-order tensor $\nabla \boldsymbol{M}(\boldsymbol{x})$.

Note that we do *not* assume that our objective function $\mathcal{L}$ has a bounded gradient. We only require the representations maps $\{\boldsymbol{F}_k\}$ to have a bounded derivative. The same assumption is also made in Castiglia et al. (2022).

In this chapter, we let $\boldsymbol{\theta}_k$, for $k \in \mathcal{K}$, and $\boldsymbol{\theta}_0$ serve as shorthand notation for $\boldsymbol{\theta}_{\boldsymbol{f}_k}$ and $\boldsymbol{\theta}_{\boldsymbol{g}}$, respectively.

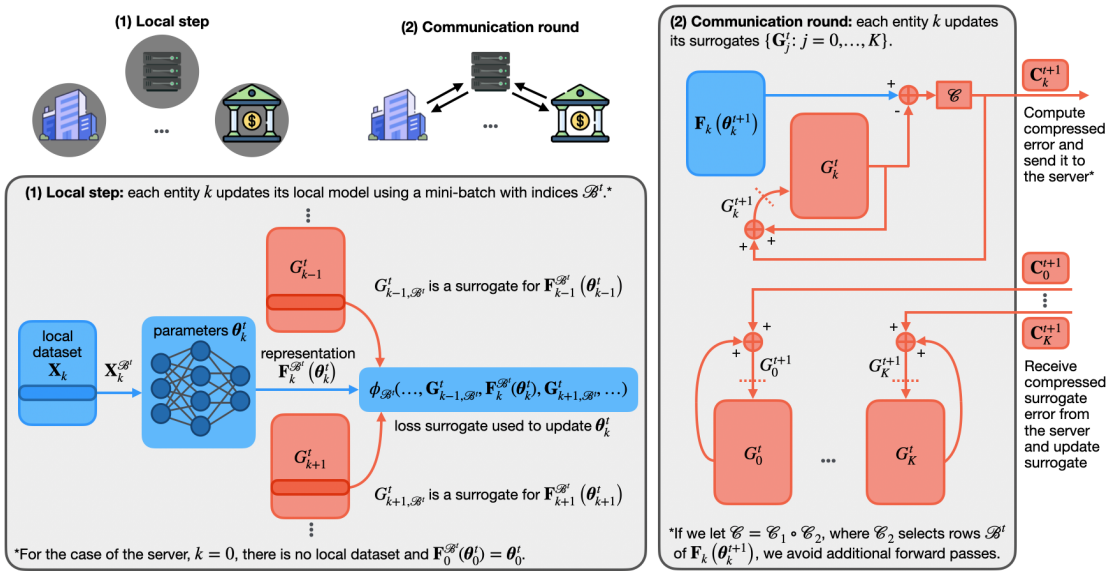


Figure 2.1: An illustration of an iteration of the `EF-VFL` algorithm. Step (1) concerns the model update and step (2) concerns the surrogate update.

2.3 Proposed method

To solve Problem (1.1) with a gradient-based method, at each step t, we need to perform a forward pass and a backward pass, to compute the gradient of our objective function. Recall that, in VFL, a standard uncompressed algorithm employing a single local update per communication round—that is, the `SVFL` method described in Chapter 1—is mathematically equivalent to gradient descent.[2] We now go over an iteration of this algorithm, which our method recovers if we set $\mathcal{C}$ to be the identity map, emphasizing the aspects that are particularly relevant for the context of communication-compressed VFL. Iteration t is as follows.

- **Forward pass:** Each client k computes representations $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$ and sends them to the server, which computes $\mathcal{L}(\boldsymbol{\theta}^t) = \phi\left(\boldsymbol{F}(\boldsymbol{\theta}^t)\right)$, thus completing the forward pass.

- **Backward pass:** First, the server backpropagates through ϕ, obtaining, for all $k = 0, 1, \ldots, K$,

$$\nabla_k \phi\left(\boldsymbol{F}(\boldsymbol{\theta}^t)\right),$$

  where ∇_k denotes the derivative with respect to argument k. Note that, since,

[2]When performing multiple local updates, we lose this mathematical equivalence. In that case, we instead get a parallel (block) coordinate descent method where the simultaneous updates use stale information about the other blocks of variables.

$\boldsymbol{F}_0(\boldsymbol{\theta}_0^t) = \boldsymbol{\theta}_0^t$, we have that:

$$\nabla_0 \phi\left(\boldsymbol{F}(\boldsymbol{\theta}^t)\right) = \nabla_0 \phi(\boldsymbol{\theta}_0^t, \boldsymbol{F}_1(\boldsymbol{\theta}_1^t), \ldots, \boldsymbol{F}_K(\boldsymbol{\theta}_K^t)) = \nabla_0 \mathcal{L}(\boldsymbol{\theta}^t).$$

That is, the derivative of ϕ with respect to its $k = 0$ argument corresponds to the derivative of the objective function $\mathcal{L}$ with respect to $\boldsymbol{\theta}_0^t$. This partial backward pass thus provides the server with the derivative that allows it to update the parameters of the fusion model. As for the remaining derivatives, each $\nabla_k \phi\left(\boldsymbol{F}(\boldsymbol{\theta}^t)\right)$, for $k \in \mathcal{K}$, is sent to client k, which uses this derivative to continue backpropagation over its local model, which allows it to compute $\nabla_k \mathcal{L}(\boldsymbol{\theta}^t)$.

We repeat these steps until convergence.

We will now go over the general case, where $\mathcal{C}$ may not be the identity map.

Forward pass. An exact forward pass would require each client k to send $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$ to the server, bringing a significant communication overhead. To address this, in our method, the server model does not have as input $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$, but rather a surrogate for it, $\boldsymbol{G}_k^t$, which is initialized as $\boldsymbol{G}_k^0 = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^0))$ and is updated as follows, based on (2.3):

$$\forall k \in \mathcal{K}: \quad \boldsymbol{G}_k^t \coloneqq \boldsymbol{G}_k^{t-1} + \boldsymbol{C}_k^t \quad \text{where} \quad \boldsymbol{C}_k^t \coloneqq \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)) - \boldsymbol{G}_k^{t-1}).$$

This requires keeping matrix $\boldsymbol{G}_k^t$, of the same $N \times E$ size as $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$, in memory at client k and at the server. (Note that $\boldsymbol{G}_k^t$ is often smaller than the local dataset $\boldsymbol{X}_k$.)

The server then computes

$$\phi\left(\boldsymbol{\theta}_0^t, \boldsymbol{G}_1^t, \ldots, \boldsymbol{G}_K^t\right),$$

which acts as a surrogate for the original objective

$$\mathcal{L}(\boldsymbol{\theta}^t) = \phi(\boldsymbol{\theta}_0^t, \boldsymbol{F}_1(\boldsymbol{\theta}_1^t), \ldots, \boldsymbol{F}_K(\boldsymbol{\theta}_K^t)).$$

Backward pass. The server performs a backward pass over the server model, obtaining

$$\nabla_0 \phi(\boldsymbol{\theta}_0^t, \{\boldsymbol{G}_k^t\}_{k=1}^K),$$

which it uses as a surrogate for

$$\nabla_0 \phi(\boldsymbol{\theta}_0^t, \boldsymbol{F}_1(\boldsymbol{\theta}_1^t), \ldots, \boldsymbol{F}_K(\boldsymbol{\theta}_K^t)) = \nabla_0 \mathcal{L}(\boldsymbol{\theta}^t)$$

Algorithm 1: `EF-VFL`

Input: initial point $\boldsymbol{\theta}^0$, stepsize η, and initial surrogates $\{\boldsymbol{G}_k^0 = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^0))\}$.

for $t = 0, \ldots, T-1$ **do**

- Update $\boldsymbol{\theta}_k^{t+1} = \boldsymbol{\theta}_k^t - \eta \widetilde{\boldsymbol{g}}_k^t$ in parallel, for $k \in \{0, 1, \ldots, K\}$, based on a shared sample $\mathcal{B}^t \subseteq \{1, \ldots, N\}$.
- Compute and send $\boldsymbol{C}_k^{t+1} = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1}) - \boldsymbol{G}_k^t)$ to the server in parallel, for $k \in \mathcal{K}$.
- Server broadcasts $\{\boldsymbol{C}_k^{t+1}\}_{k=0}^K$ to all clients.
- Update $\boldsymbol{G}_k^{t+1} = \boldsymbol{G}_k^t + \boldsymbol{C}_k^{t+1}$ in parallel, for $k \in \{0, 1, \ldots, K\}$.

to update $\boldsymbol{\theta}_0^t$. Similarly, to update each local model $\boldsymbol{\theta}_k^t$, for $k \in \mathcal{K}$, we want client k to have a surrogate for

$$\nabla_k \mathcal{L}(\boldsymbol{\theta}^t) = \sum_{i,j=1}^{N,E} \left[\nabla_k \phi(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\}_{k=0}^K)\right]_{ij} \left[\nabla \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\right]_{ij:}.$$

However, while $\nabla \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$ can be computed at each client k, the $\nabla_k \phi$ term cannot, as client k does not have access to $\boldsymbol{F}_\ell(\boldsymbol{\theta}_\ell^t)$, for $\ell \neq k$. So, we instead use the following surrogate for $\nabla_k \mathcal{L}(\boldsymbol{\theta}^t)$:

$$\boldsymbol{g}_k^t := \sum_{i,j=1}^{N,E} \left[\widetilde{\nabla}_k^t \phi\right]_{ij} \left[\nabla \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\right]_{ij:}, \tag{2.4}$$

where $\widetilde{\nabla}_k^t \phi := \nabla_k \phi(..., \boldsymbol{G}_{k-1}^t, \boldsymbol{F}_k(\boldsymbol{\theta}_k^t), \boldsymbol{G}_{k+1}^t, ...)$. Since the server does not hold $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$, it cannot compute $\widetilde{\nabla}_k^t \phi$. Thus, the server broadcasts $\{\boldsymbol{C}_k^t\}_{k=0}^K$, so that each client k, which does hold $\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)$, can compute $\{\boldsymbol{G}_\ell^t\}_{\ell \neq k}$ and use it to perform a forward and a backward pass over the server model locally, obtaining $\widetilde{\nabla}_k^t \Phi$. We illustrate this in Figure 2.1.

Therefore, while the forward pass only requires the error feedback module at each client k to hold the estimate $\boldsymbol{G}_k^t$, when we account for the backward pass too, each machine $k \in \{0, 1, \ldots, K\}$ must hold $\{\boldsymbol{G}_\ell^t\}_{\ell=0}^K$. We write our update as:

$$\forall t \geq 0: \quad \boldsymbol{x}^{t+1} = \boldsymbol{x}^t - \eta \boldsymbol{g}^t \quad \text{where} \quad \boldsymbol{g}^t := (\boldsymbol{g}_0^t, \ldots, \boldsymbol{g}_K^t)$$

and η is the stepsize.

Mini-batch. For the sake of computation efficiency, we further allow for the use of mini-batch approximations of the objective. Without compression, or with direct compression, the use of mini-batches allows client k to send only the entries of $\boldsymbol{F}_k(\boldsymbol{\theta}_k)$ corresponding to

mini-batch $\mathcal{B} \subseteq \{1, \ldots, N\}$, of size B, denoted by $\boldsymbol{F}_k^{\mathcal{B}}(\boldsymbol{\theta}_k) \in \mathbb{R}^{B \times E}$, instead of $\boldsymbol{F}_k(\boldsymbol{\theta}_k) \in \mathbb{R}^{N \times E}$. Yet, our method provides all machines with an estimate for all the entries of $\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}$ at all times, the error feedback states $\{\boldsymbol{G}_k\}$. Therefore, our communications, which serve the purpose of updating $\boldsymbol{G}_k$, may not depend on N and B at all. This is determined by our choice of compressor $\mathcal{C}$.

Our mini-batch surrogates depend on the entries of $\boldsymbol{F}_k(\boldsymbol{\theta}_k)$ and $\boldsymbol{G}_k$ corresponding to $\mathcal{B}$, $\boldsymbol{F}_k^{\mathcal{B}}(\boldsymbol{\theta}_k)$ and $\boldsymbol{G}_{k\mathcal{B}}$. (Note that $\boldsymbol{F}_0^{\mathcal{B}}(\boldsymbol{\theta}_0) = \boldsymbol{\theta}_0$ and $\boldsymbol{G}_{0\mathcal{B}}^t = \boldsymbol{G}_0^t$.) Thus, we approximate the partial derivative of the mini-batch function $\mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta}) = \phi_{\mathcal{B}}(\{\boldsymbol{F}_k^{\mathcal{B}}(\boldsymbol{\theta}_k)\})$ with respect to $\boldsymbol{\theta}_k$ by:

$$\widetilde{\boldsymbol{g}}_k^t \coloneqq \sum_{i \in \mathcal{B}^t} \sum_{j=1}^{E} \left[\widetilde{\nabla}_k^t \phi_{\mathcal{B}^t}\right]_{ij} \left[\nabla \boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)\right]_{ij:},$$

where $\widetilde{\nabla}_k^t \phi_{\mathcal{B}^t} \coloneqq \nabla_k \phi_{\mathcal{B}^t}(..., \boldsymbol{G}_{k-1,\mathcal{B}^t}^t, \boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t), \boldsymbol{G}_{k+1,\mathcal{B}^t}^t, ...)$. We write the mini-batch version of the update as:

$$\forall t \geq 0\colon \quad \boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \eta \widetilde{\boldsymbol{g}}^t \quad \text{where} \quad \widetilde{\boldsymbol{g}}^t \coloneqq \left(\widetilde{\boldsymbol{g}}_0^t, \ldots, \widetilde{\boldsymbol{g}}_K^t\right).$$

We now describe our method, which we summarize in Algorithm 1.

- **Initialization:** We initialize our model parameters as $\boldsymbol{\theta}^0$. Each machine $k \in \{0, 1, \ldots, K\}$ must hold $\boldsymbol{\theta}_k^0$ and our compression estimates $\{\boldsymbol{G}_k^0 = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^0))\}_{k=0}^K$.
- **Model parameters update:** In parallel, all machines $k \in \{0, 1, \ldots, K\}$ take a (stochastic) coordinate descent step with respect to their local surrogate objective. They update $\boldsymbol{\theta}_k^{t+1} = \boldsymbol{\theta}_k^t - \eta \widetilde{\boldsymbol{g}}_k^t$ based on a shared batch $\mathcal{B}^t$, sampled locally at each client following a shared seed.
- **Compressed communications:** All clients $k \in \mathcal{K}$ compute $\boldsymbol{C}_k^{t+1}$ and send it to the server, who broadcasts $\{\boldsymbol{C}_k^{t+1}\}$.
- **Compression estimates update:** Lastly, all machines $k \in \{0, 1, \ldots, K\}$ use the compressed error feedback $\{\boldsymbol{C}_k^{t+1}\}$ to update their (matching) compression estimates $\{\boldsymbol{G}_k^t\}$.

Note that, in the absence of compression (that is, if $\mathcal{C}$ is the identity operator), each client k must send $\boldsymbol{C}_k^{t+1}$, of size NE, at each iteration. Further, the server must broadcast $\{\boldsymbol{C}_k^{t+1}\}_{k=0}^K$. Thus, in the absence of compression, the total communication complexity of `EF-VFL` is $\mathcal{O}(NET)$. In contrast, when compression is used, NE is replaced by some

smaller amount which depends on the compression mechanism. For example, for top-k sparsification, the communication complexity is reduced to $\mathcal{O}(kT)$.

In Algorithm 1, we formulate our method in a general setting which allows for the compression of both $\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}_{k=1}^K$ and $\boldsymbol{\theta}_0$. This setting is covered by our analysis in Section 2.4. However, given that the bottleneck typically lies in the uplink (client-to-server) communications, rather than the server broadcasting (Haddadpour et al., 2021), our experiments in Section 2.5 focus on the compression of $\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}_{k=1}^K$, rather than $\boldsymbol{\theta}_0$.

2.3.1 Adapting our method for handling private labels

In this section, we propose an adaptation of `EF-VFL` to allow for private labels. That is, we remove the assumption that all clients hold ϕ_n, which contains the label of sample $\boldsymbol{x}^n$. Instead, only the server holds the labels. Further, in this adaptation, the parameters of the server model, $\boldsymbol{\theta}_0$, are not shared with the clients either.

Note that, without holding ϕ_n, the clients cannot perform the entire forward pass locally. Instead, in this setting, the forward and backward pass over ϕ_n take place at the server, while the forward and backward pass over $\boldsymbol{F}_k(\boldsymbol{\theta}_k)$ takes place at client k. More precisely, in the forward pass, each client k sends $\boldsymbol{C}_k$ to the server, who holds $\boldsymbol{\theta}_0$ and the labels, and can thus compute the loss. Then, for the backward pass, the server backpropagates over its model and sends only the derivative of the loss function with respect to $\boldsymbol{G}_k$ to each client k. This requires replacing our surrogate of $\nabla_k \mathcal{L}(\boldsymbol{\theta}^t)$ in (2.4), which uses the exact local representation $\boldsymbol{F}_k(\boldsymbol{\theta}_k)$ at each client, by one based on our error feedback surrogates:

$$\boldsymbol{\nabla}_k^t := \sum_{i,j=1}^{N,E} \left[\nabla_k \phi\left(\boldsymbol{F}_0(\boldsymbol{\theta}_0^t), \{\boldsymbol{G}_j^t\}_{j=1}^K\right)\right]_{ij} \left[\nabla \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\right]_{ij:}. \tag{2.5}$$

Note that we do not backpropagate through the error-feedback update.

More generally, we use the following (possibly) mini-batch update vector:

$$\widetilde{\boldsymbol{\nabla}}_k^t := \sum_{i\in\mathcal{B}^t} \sum_{j=1}^{E} \left[\nabla_k \phi_{\mathcal{B}^t}\left(\boldsymbol{F}_0^{\mathcal{B}^t}(\boldsymbol{\theta}_0^t), \{\boldsymbol{G}_{j\mathcal{B}^t}^t\}_{j=1}^K\right)\right]_{ij} \left[\nabla \boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)\right]_{ij:}.$$

We summarize the adaption of the `EF-VFL` method to the private labels setting in Algorithm 2.

Allowing for private labels broadens the range of applications for our method, since many VFL applications require not only private features, but also private labels, rendering any method requiring public labels inapplicable. However, as we will see in Section 2.5.2,

Algorithm 2: `EF-VFL` with private labels

Input: initial point $\boldsymbol{\theta}^0$, stepsize η, and initial surrogates $\{\boldsymbol{G}_k^0 = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^0))\}$.

for $t = 0, \ldots, T-1$ **do**

  Update $\boldsymbol{\theta}_k^{t+1} = \boldsymbol{\theta}_k^t - \eta \widetilde{\boldsymbol{\nabla}}_k^t$ in parallel, for $k \in \{0, 1, \ldots, K\}$, based on shared mini-batch indices $\mathcal{B}^t \subseteq \{1, \ldots, N\}$.

  Compute and send $\boldsymbol{C}_k^{t+1} = \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1}) - \boldsymbol{G}_k^t)$ to the server in parallel, for $k \in \mathcal{K}$.

  Server sends $\nabla_k \phi_{\mathcal{B}^t}\left(\boldsymbol{F}_0^{\mathcal{B}^t}(\boldsymbol{\theta}_0^t), \{\boldsymbol{G}_{j\mathcal{B}^t}^t\}_{j=1}^K\right)$ to client k, in parallel, for $k \in \{0, 1, \ldots, K\}$.

  Update $\boldsymbol{G}_k^{t+1} = \boldsymbol{G}_k^t + \boldsymbol{C}_k^{t+1}$ in parallel, for $k \in \{0, 1, \ldots, K\}$.

using surrogate (2.5) instead of (2.4) can slow down convergence. Further, unlike Algorithm 1, which can be easily extended to allow for multiple local updates at the clients between rounds of communication, Algorithm 2 works only for a single local update. This is because, for a VFL algorithm to perform multiple local updates, ϕ_n and $\boldsymbol{x}^0$ must be available at the clients, so that the forward and backward passes over the server model and the loss function can be performed locally after each update.

2.4 Convergence guarantees

In this section, we provide convergence guarantees for `EF-VFL`. We present our results for Algorithm 1 only, rather then stating them again for Algorithm 2, since they exhibit only a minor difference in Lemma 1 and in the main theorem, where the constant K is replaced with $K+1$. We defer the details to Appendix A.1.

First, let us define the following sigma-algebra

$$\mathcal{F}_t \coloneqq \sigma(\boldsymbol{G}^0, \boldsymbol{\theta}^1, \boldsymbol{G}^1, \ldots, \boldsymbol{\theta}^t, \boldsymbol{G}^t),$$

where $\boldsymbol{G}^t \coloneqq \{\boldsymbol{G}_0^t, \ldots, \boldsymbol{G}_K^t\}$. We make the following assumptions on our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ and the use of mini-batches:

- We assume that our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ is an unbiased estimate of $\boldsymbol{g}^t$ (D3) conditional on $\mathcal{F}_t$, for all $\boldsymbol{\theta}$ and t.

- We assume that our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ has its variance upper bounded (D4) by σ^2/B, conditional on $\mathcal{F}_t$, for all $\boldsymbol{\theta}$ and t.

We now present Lemma 1 and Lemma 2, which we will use to prove our main theorems. We let $D^{(t)} \coloneqq \sum_{k=0}^{K} \|\boldsymbol{G}_k^t - \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\|^2$ denote the total distortion (caused by compression) at time t.

Lemma 1 (Surrogate offset bound)**.** *If ϕ is L-smooth (D2) and $\{\boldsymbol{F}_k\}$ have bounded derivatives (D7), then, for all $t \geq 0$,*

$$\|\boldsymbol{g}^t - \nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq KH^2L^2D^{(t)}. \tag{2.6}$$

Proof. See Appendix A.2.1. □

Lemma 2 (Recursive distortion bound)**.** *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 1. If $\mathcal{C}$ is a contractive compressor (D6), $\{\boldsymbol{F}_k\}$ have bounded derivatives (D7), and our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ is an unbiased estimate of $\boldsymbol{g}^t$ (D3) with a bounded variance (D4), then, for all $t \geq 0$ and $\epsilon > 0$:*

$$\mathbb{E}D^{(t+1)} \leq (1-\alpha)(1+\epsilon)\mathbb{E}D^{(t)} + (1-\alpha)(1+\epsilon^{-1})\eta^2H^2\left(\mathbb{E}\|\boldsymbol{g}^t\|^2 + \frac{\sigma^2}{B}\right). \tag{2.7}$$

Proof. See Appendix A.2.2. □

2.4.1 Nonconvex setting

We now present our main convergence result for `EF-VFL`.

Theorem 1. *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 1, if $\mathcal{L}$ has a finite infimum (D1), $\mathcal{L}$ and ϕ are L-smooth (D2), $\{\boldsymbol{F}_k\}$ have bounded derivatives (D7), our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ is an unbiased estimate of $\boldsymbol{g}^t$ (D3) with a bounded variance (D4), and $\mathcal{C}$ is a contractive compressor (D6), then, for $0 < \eta \leq 1/(\sqrt{\rho_{\alpha 1}}L + L)$:*

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2\Delta}{\eta T} + (1+\eta L\rho_{\alpha 1})\frac{\eta L\sigma^2}{B} + \rho_{\alpha 2}\frac{\mathbb{E}D^{(0)}}{T}, \tag{2.8}$$

where the expectation is over the randomness in $\mathcal{C}$ and in $\{\mathcal{B}_t\}$, $\Delta \coloneqq \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$, and

$$\rho_{\alpha 1} \coloneqq KH^4\left(\frac{1+\sqrt{1-\alpha}}{\alpha} - 1\right)^2 \quad \text{and} \quad \rho_{\alpha 2} \coloneqq \frac{KH^2L^2}{1-\sqrt{1-\alpha}}.$$

Proof. See Appendix A.3.1. □

If the batch size is large enough, $B = \Omega(\sigma^2/\delta)$, the iteration complexity to reach $\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left\|\nabla\mathcal{L}\left(\boldsymbol{\theta}^t\right)\right\|^2 \leq \delta$ matches the $\mathcal{O}(1/T)$ rate of the centralized, uncompressed setting. Also, in the absence of compression ($\alpha = 1$ and $D^{(t)} = 0$), we recover that, for $\eta \in (0, 1/L]$, we can output an $\boldsymbol{\theta}^{\text{out}}$ such that $\mathbb{E}\|\nabla f(\boldsymbol{\theta}^{\text{out}})\|^2 \leq \frac{2\Delta}{\eta T} + \frac{\eta L\sigma^2}{B}$. If we are also in the full-batch case ($\sigma = 0$), we recover the gradient descent bound exactly: $\mathbb{E}\|\nabla f(\boldsymbol{\theta}^{\text{out}})\|^2 \leq \frac{2\Delta}{\eta T}$.

Note that, if we start our method by sending noncompressed representations (at $t = 0$ only), we can drop the last term in the upper bound in (2.8).

Our results improve over the prior state-of-the-art compressed vertical FL (`CVFL`) (Castiglia et al., 2022), whose convergence results are as follows:

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{4\Delta}{\eta T} + \mathcal{O}\left(\frac{\eta L\sigma^2}{B}\right) + \mathcal{O}\left(\frac{1}{T}\sum_{t=0}^{T-1} D_d^{(t)}\right).$$

Note how, even for full-batch updates, the upper bound above does not go to zero as $T \to \infty$ unless $D_d^{(t)} \to 0$, where $D_d^{(t)}$ is the distortion resulting from direct compression $D_d^{(t)} = \sum_{k=0}^{K}\left\|\mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)) - \boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\right\|^2$. That is, to achieve $\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \to 0$, `CVFL` requires a vanishing compression error, which necessitates that $\alpha \to 1$. This means that, despite reducing the total amount communications across the training, `CVFL` does not reduce the maximum amount of communications per round. In contrast, by allowing for nonvanishing compression, `EF-VFL` ensures small communication cost at every round.

2.4.2 Under the PL inequality

In this section, we establish the linear convergence of `EF-VFL` under the assumption that the PL inequality holds for $\mathcal{L}$ (D5).

To show linear convergence, we resort to the following Lyapunov function:

$$V_t \coloneqq \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star + c_\alpha \mathbb{E}D^{(t)}, \tag{2.9}$$

where c_α is a positive constant. We now present Theorem 2.

Theorem 2. *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 1, if $\mathcal{L}$ has a finite infimum (D1), $\mathcal{L}$ and ϕ are L-smooth (D2), $\{\boldsymbol{F}_k\}$ have bounded derivatives (D7), our stochastic update vector $\widetilde{\boldsymbol{g}}^t$ is an unbiased estimate of $\boldsymbol{g}^t$ (D3) with a bounded variance (D4), $\mathcal{C}$ is a contractive compressor (D6), and the PL inequality holds for $\mathcal{L}$ (D5), then, for η such*

Table 2.1: Total communication cost to reach error level $\delta > 0$ (top-k; full-batch; single local update).

$\mathcal{C}$	Uplink (total)	Downlink (total/broadcast)
`SVFL`	$NEK \cdot \mathcal{O}(1/\delta)$	$NEK \cdot \mathcal{O}(1/\delta)$
`CVFL` (Castiglia et al., 2022)	$\min\{k \cdot \mathcal{O}(1/\sqrt{\delta}), NE\} \cdot K \cdot \mathcal{O}(1/\delta^2)$	$\min\{k \cdot \mathcal{O}(1/\sqrt{\delta}), NE\} \cdot (K+1) \cdot \mathcal{O}(1/\delta^2)$
`EF-VFL` (ours)	$kK \cdot \mathcal{O}(1/\delta)$	$k(K+1) \cdot \mathcal{O}(1/\delta)$
`EF-VFL` with private labels (ours)	$kK \cdot \mathcal{O}(1/\delta)$	$kK \cdot \mathcal{O}(1/\delta)$

that $\eta^2 L^2 (1 - \mu/L) + \eta\mu \leq \alpha^2$, *we have that:*

$$V_T \leq (1 - \eta\mu)^T V_0 + \frac{\sigma^2}{2B\mu}.$$

Proof. See Appendix A.3.2. □

Since $\mu \leq L$, we have that $\eta \in (0, 1/L)$ implies that $1 - \eta\mu \in (0, 1)$. Hence, `EF-VFL` converges linearly to a $\mathcal{O}(\sigma^2)$ neighborhood around the global optimum.

In Table 2.1, where E is the embedding size for each sample at each client (assumed to match for simplicity), we present the total communication cost to reach $\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\left\|\nabla f\left(\boldsymbol{\theta}^t\right)\right\|^2 \leq \delta$, where $\delta > 0$ (top-k; full-batch; single local update). We now discuss different downlink communication schemes for `EF-VFL`.

Comparison of different downlink communication schemes. As in most communication-compressed optimization literature (Haddadpour et al., 2021), our primary concern is uplink communications, which is typically the main bottleneck in training. Nevertheless, we now discuss three alternative downlink communication schemes in `EF-VFL`: **1)** the one in Algorithm 1, **2)** the one in Algorithm 2, and **3)** a modified version of the one in Algorithm 1 for common VFL fusion models, enabling broadcasts of a size that is independent of the number of clients, K. Approaches **1)** and **3)** are mathematically equivalent, yet Approach **2)** is not, as discussed earlier. Each approach has its pros and cons, making it suitable for different applications. For simplicity, this discussion focuses on top-k sparsification and the full-batch case.

- **Approach 1)** In Algorithm 1, each round of downlink communications consists of a broadcast of size $k(K+1)$—a compressed object of size k for each client (the intermediate representations) and one for the server (the fusion model).
- **Approach 2)** In Algorithm 2, each client receives only the derivative of the loss function with respect to its representation, resulting in a total downlink communi-

cation cost of kK. Recall that this is only an option when performing a single local update.

Approach **2)** avoids the dependency of the downlink communications to each client on K, seen in Approach **1)**, but requires K different communications (one to each client), rather than a single broadcast, thus the total communication cost still depends on K. The one-to-many nature of broadcasting makes it is more appropriate to compare broadcasted information with the total downlink communications, rather than the communication to a single client, as the latter ignores the cost of contacting the other $K-1$ clients.

- **Approach 3)** To ensure that the downlink communication cost does not depend on K, we can often exploit the structure of the fusion model g. In particular, a common choice is $g(\boldsymbol{\theta}) = g_1(\boldsymbol{\theta}_0, g_0(\boldsymbol{F}_1(\boldsymbol{\theta}_1), \ldots, \boldsymbol{F}_K(\boldsymbol{\theta}_K)))$, where g_0 is a nonparameterized representation aggregator, such as a sum or an average, and g_1 is a map parameterized by $\boldsymbol{\theta}_0$. In this case, instead of broadcasting $K+1$ objects, as in Approach **1)**, the server can broadcast the aggregation of the representations, $g_0(\{\boldsymbol{F}_j(\boldsymbol{\theta}_j^t)\})$. This allows us to collapse the dimension of length K, as long as each client i can replace $\boldsymbol{F}_i(\boldsymbol{\theta}_i^t)$ with $\boldsymbol{F}_i(\boldsymbol{\theta}_i^{t+1})$ in $g_0(\{\boldsymbol{F}_j(\boldsymbol{\theta}_j^t)\})$ using its local knowledge of its own representation. For example, if g_0 is a sum, client i can subtract its previous intermediate representation and add the updated one to obtain an updated aggregation. This allows client i to perform forward and backward passes over both its local model and the fusion model, and thus perform multiple local updates without requiring further communications. Yet, this downlink communication of the aggregated representations will no longer be in the range of the compressor. For example, if $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are within the range of top-k, their sum, $\boldsymbol{v}_1 + \boldsymbol{v}_2$, will generally not be. Therefore, we have a broadcast of up to size $NE + d_0$, where d_0 is the size of the parameters of the fusion model. That is, we avoid the dependency on K, but this comes at the cost of losing the compressed nature of the downlink communications. (This sum may still lie in a lower-dimensional manifold, but this typically recovers the dependency on K, e.g., for top-k sparsification, we can upper bound the number of nonzero entries of the sum of K k-sparse vectors by $\min\{kK, NE\} + d_0$.) Like Approach **1)**, Approach **3)** does not allow for private labels.

We present Approach **1)** in Algorithm 1, rather than Approach **3)**, because most VFL applications are in the cross-silo setting (Liu et al., 2024) and thus the number of clients K is small, therefore $k(K+1) \ll N\bar{E} + d_0$. Yet, for applications where K is large, Approach

3) may be preferable.

2.5 Experiments

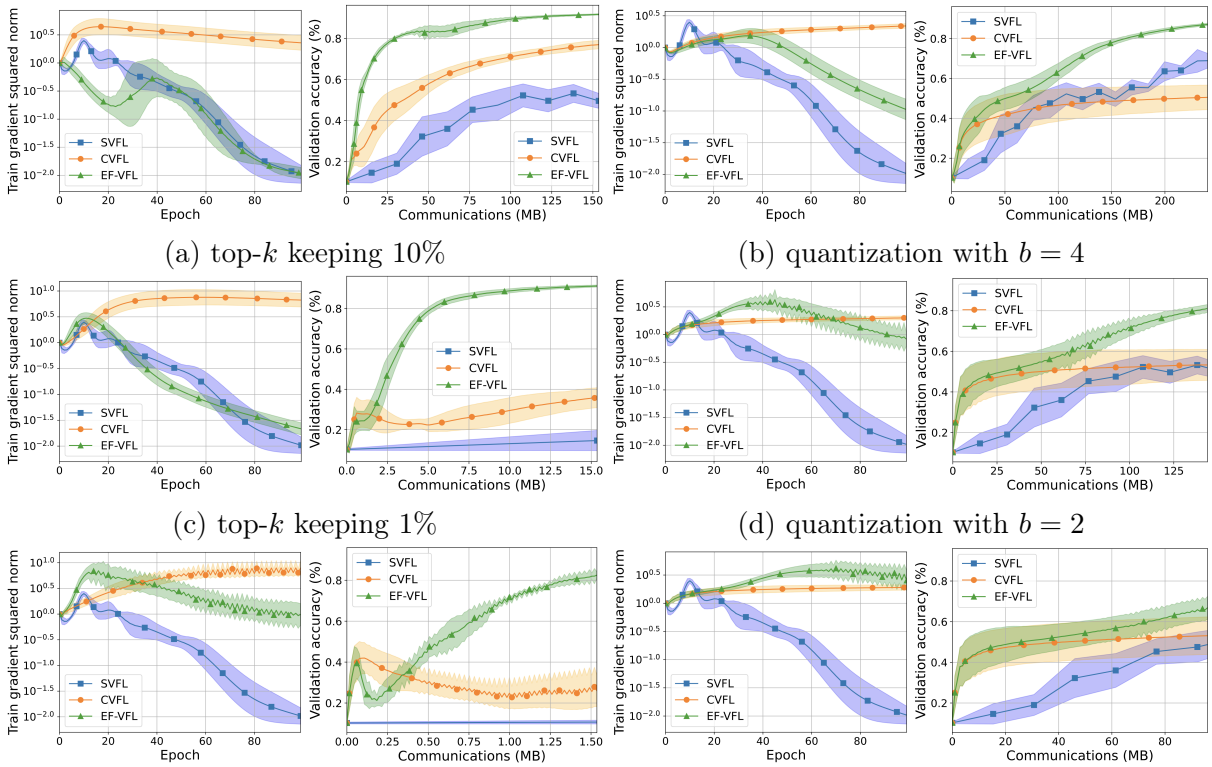
We compare `EF-VFL` with two baselines: **(1)** standard VFL (`SVFL`), which corresponds to the method in Liu et al. (2022b) with a single local update and is mathematically equivalent to stochastic gradient descent, and **(2)** `CVFL` (Castiglia et al., 2022), which recovers `SVFL` when an identity compressor is used (that is, without employing compression). All of our results correspond to the mean and standard deviation for five different seeds. We employ two popular compressors in our experiments: top-k sparsification (2.1) and stochastic quantization (2.2).

For the sake of the comparison with `CVFL`, we employ compressors $\mathcal{C} = \mathcal{C}_2 \circ \mathcal{C}_1$, where $\mathcal{C}_2$ is either $\text{top}_k(\boldsymbol{v})$ or qsgd_s and $\mathcal{C}_1$ selects the rows in $\mathcal{B}^t$, similarly to `CVFL`. Further, as explained in Section 2.3, our experiments focus on the compression of $\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}_{k=1}^K$, and not $\boldsymbol{\theta}_0$.

2.5.1 Comparison with `SVFL` and `CVFL`

The detailed hyperparameters for the following experiments can be found in the provided code.

MNIST. We train a shallow neural network (one hidden layer) on the MNIST digit recognition dataset (LeCun et al., 1998). The 28×28 images in the original dataset $\boldsymbol{X}$ are split into four local datasets $\boldsymbol{X}_k$ of 14×14 images, its quadrants ($K = 4$). The local representation models $\boldsymbol{f}_k$ are maps $\boldsymbol{v} \mapsto \text{sigmoid}(\boldsymbol{W}_{k1}\boldsymbol{v})$, with $\boldsymbol{W}_{k1} \in \mathbb{R}^{128\times 196}$, and the fusion model is $(\boldsymbol{v}_1, \dots, \boldsymbol{v}_4) \mapsto \boldsymbol{W}_2(\sum_{k=1}^4 \boldsymbol{v}_k)$, with $\boldsymbol{W}_2 \in \mathbb{R}^{10\times 16}$. We use cross-entropy loss. In Figure 2.2, we present the results for when `EF-VFL` and `CVFL` employ top_k, keeping 10%, 1%, and 0.1% of the entries, and when they employ qsgd_s, sending $b \in \{4, 2, 1\}$ bits per entry, instead of the uncompressed $b = 32$. In both figures, we see that `EF-VFL` outperforms `SVFL` and `CVFL` in communication efficiency. In terms of results per epoch, `EF-VFL` significantly outperforms `CVFL` and, for a sufficiently large k (for top_k) or b (for qsgd_s) `EF-VFL` achieves a similar performance to `SVFL`. As predicted in Section 2.4, the train gradient squared norm during training goes to zero for `EF-VFL`, as it does for `SVFL`, but not for `CVFL`.



(a) top-k keeping 10%

(b) quantization with $b = 4$

(c) top-k keeping 1%

(d) quantization with $b = 2$

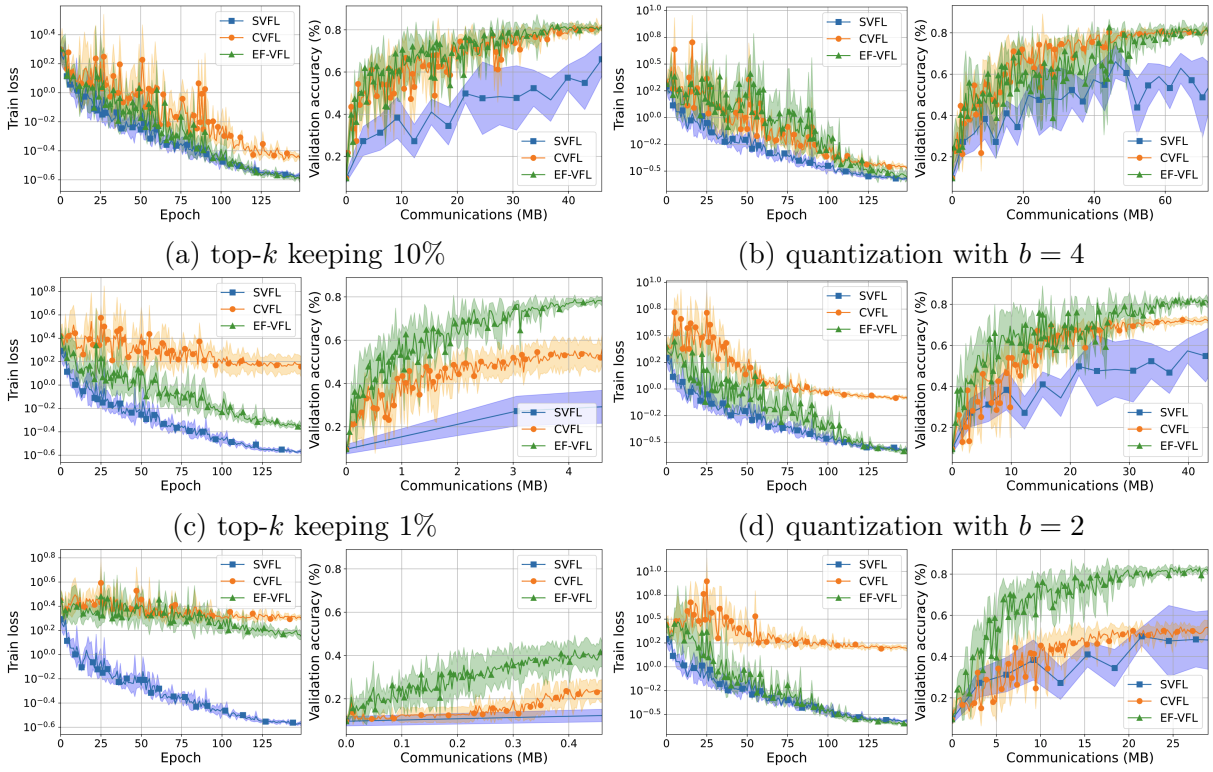
(e) top-k keeping 0.1%

(f) quantization with $b = 1$

Figure 2.2: The (relative) training gradient squared norm with respect to epochs and validation accuracy with respect to communication cost for the training of a shallow neural network on MNIST. On the left, `CVFL` and `EF-VFL` employ top-k sparsification with a decreasing k across rows. On the right, they employ stochastic quantization with a decreasing number of bits across rows. `SVFL` is the same throughout.

ModelNet10. We train a multi-view convolutional neural network (MVCNN) (Su et al., 2015) on ModelNet10 (Wu et al., 2015), a dataset of three-dimensional CAD models. We use a preprocessed version of ModelNet10, where each sample is represented by 12 two-dimensional views. We assign a view per client ($K = 12$). In Figure 2.3, we present the results for when `EF-VFL` and `CVFL` employ top_k, keeping 20%, 2%, and 0.2% of the entries, and when they employ qsgd_s, with $b \in \{8, 4, 2\}$. We plot the train loss with respect to the number of epochs and the validation accuracy with respect to the communication cost. We observe that, for `EF-VFL`, the training loss decreases more rapidly than for `CVFL`. Further, if the compression is not excessively aggressive, `EF-VFL` performs similarly to `SVFL`. In terms of communication efficiency, `EF-VFL` outperforms both `SVFL` and `CVFL`.

CIFAR-100. We train a model based on a residual neural network, ResNet18 (He et al., 2016), on CIFAR-100 (Krizhevsky and Hinton, 2009). More precisely, we divide each



(a) top-k keeping 10%

(b) quantization with $b = 4$

(c) top-k keeping 1%

(d) quantization with $b = 2$

(e) top-k keeping 0.1%

(f) quantization with $b = 1$

Figure 2.3: Train loss with respect to the number of epochs and validation accuracy with respect to the communication cost for the training of an MVCNN on ModelNet10. On the left, `CVFL` and `EF-VFL` employ top-k sparsification with a decreasing k across rows. On the right, they employ stochastic quantization with a decreasing number of bits across rows. `SVFL` is the same throughout.

image into four quadrants and allocate one quadrant to each client ($K = 4$), with each client using a ResNet18 model as its local model. The server model is linear (a single layer). In Figure 2.4, we present the results for when `EF-VFL` and `CVFL` employ top_k, keeping 10%, 1%, and 0.1% of the entries, and when they employ qsgd_s, with $b \in \{4, 2, 1\}$. We plot the train loss with respect to the number of epochs and the validation accuracy with respect to the communication cost. Regarding the results with respect to the number of epochs, `EF-VFL` achieves a similar performance to that of `SVFL`, significantly outperforming `CVFL`. In terms of communication efficiency, `EF-VFL` outperforms both `SVFL` and `CVFL`.

We summarize the test metrics for all three tasks in Table 2.2. In brief, while `CVFL` performs well for less aggressive compression is employed, the improved performance of `EF-VFL` is significant when more aggressive compression is employed.

Table 2.2: Test accuracy for SVFL, CVFL, and EF-VFL across different tasks, for a fixed number of epochs. We run reach experiment for 5 seeds and present the mean accuracy ± standard deviation, highlighting the highest accuracy in bold.

		MNIST (accuracy, %)					
		top-k compressor			qsgd compressor		
	Uncompressed	keep 10%	keep 1%	keep 0.1%	$b = 4$	$b = 2$	$b = 1$
SVFL	91.6 ± 0.1	—	—	—	—	—	—
CVFL	—	77.2 ± 1.9	35.7 ± 5.0	25.7 ± 7.6	50.3 ± 6.1	53.0 ± 7.4	52.7 ± 8.7
EF-VFL	—	$\mathbf{91.8} \pm 0.1$	$\mathbf{91.1} \pm 0.3$	$\mathbf{82.4} \pm 2.1$	$\mathbf{87.2} \pm 0.4$	$\mathbf{81.1} \pm 1.6$	$\mathbf{66.8} \pm 5.9$
		ModelNet10 (accuracy, %)					
		top-k compressor			qsgd compressor		
	Uncompressed	keep 10%	keep 1%	keep 0.1%	$b = 4$	$b = 2$	$b = 1$
SVFL	81.2 ± 0.8	—	—	—	—	—	—
CVFL	—	$\mathbf{80.7} \pm 3.1$	53.2 ± 6.5	24.5 ± 4.2	$\mathbf{80.3} \pm 2.0$	70.7 ± 1.6	52.0 ± 3.2
EF-VFL	—	80.4 ± 1.9	$\mathbf{77.4} \pm 2.5$	$\mathbf{40.3} \pm 4.8$	79.4 ± 3.7	$\mathbf{80.4} \pm 2.7$	$\mathbf{81.1} \pm 2.8$
		CIFAR-100 (accuracy, %)					
		top-k compressor			qsgd compressor		
	Uncompressed	keep 10%	keep 1%	keep 0.1%	$b = 4$	$b = 2$	$b = 1$
SVFL	57.7 ± 0.6	—	—	—	—	—	—
CVFL	—	56.8 ± 0.6	45.1 ± 2.3	11.6 ± 1.4	19.2 ± 0.6	5.9 ± 0.7	2.0 ± 0.1
EF-VFL	—	$\mathbf{57.2} \pm 0.8$	$\mathbf{54.8} \pm 0.9$	$\mathbf{36.4} \pm 2.8$	$\mathbf{57.8} \pm 0.5$	$\mathbf{50.2} \pm 1.3$	$\mathbf{34.7} \pm 2.8$

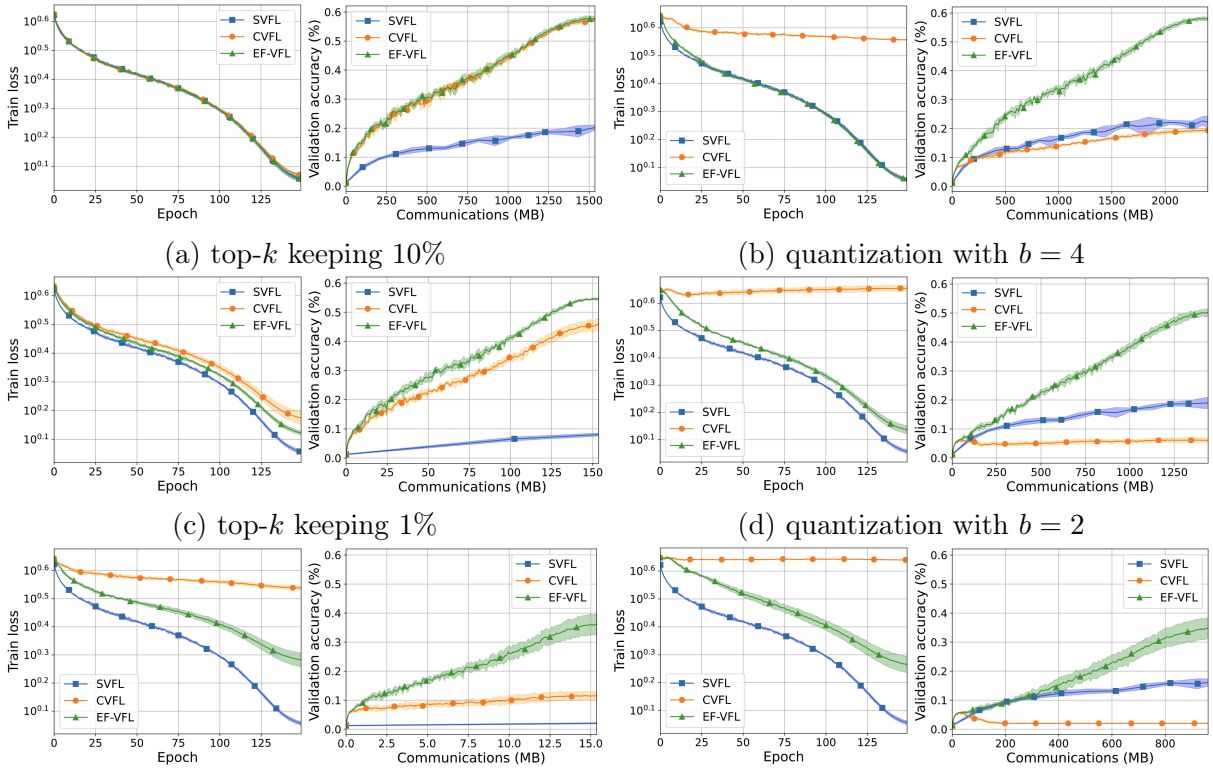
2.5.2 Performance under private labels

In this section, we run experiments on the adaption of EF-VFL to handle private labels, proposed in Section 2.3.1.

In Castiglia et al. (2022), the authors assume that the labels are available at all clients and do not propose an adaptation of CVFL to deal with private labels. Yet, to get a baseline for a compressed VFL method allowing for label privacy, we adapt CVFL in a similar manner to how we adapt EF-VFL (that is, sending back the derivative from the server to the clients, instead of ϕ_n and $\boldsymbol{x}^0$, and without backpropagating through the compression operator).

We run an experiment on MNIST, training the same shallow neural network as in model as in Section 2.5.1, with all the same settings, except for the use of batch size $B = 1024$. Further, we train a ResNet18-based model with a linear server model (a single layer) on CIFAR-100 (Krizhevsky and Hinton, 2009). For all the optimizers, we use an initial stepsize of $\eta = 0.01$ and a cosine annealing scheduler with a minimum stepsize of $1/100$ the initial value and use a batch size $B = 128$ and a weight decay of 0.01. The compressed-communication methods employ top_k, keeping 5% of the entries.

In Figure 2.5, we observe that, although the modified EF-VFL for handling private labels converges noticeably slower than the original method, it still performs effectively. For both



(a) top-k keeping 10%

(b) quantization with $b = 4$

(c) top-k keeping 1%

(d) quantization with $b = 2$

(e) top-k keeping 0.1%

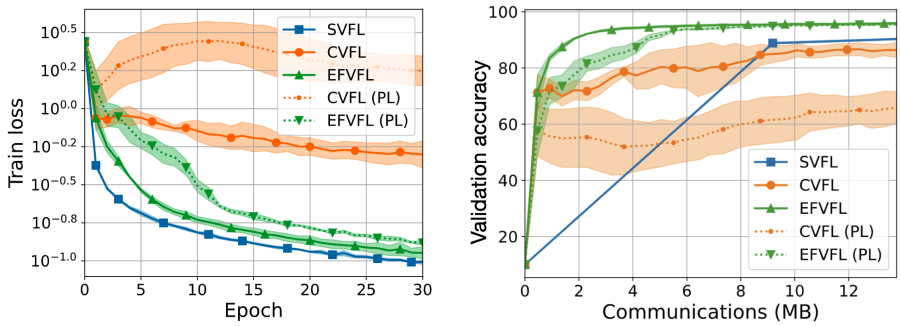
(f) quantization with $b = 1$

Figure 2.4: Train loss with respect to the number of epochs and the validation accuracy with respect to the communication cost for the training of a ResNet18-based model on CIFAR-100. On the left, `CVFL` and `EF-VFL` employ top-k sparsification with a decreasing k across rows. On the right, they employ stochastic quantization with a decreasing number of bits across rows. `SVFL` is the same throughout.

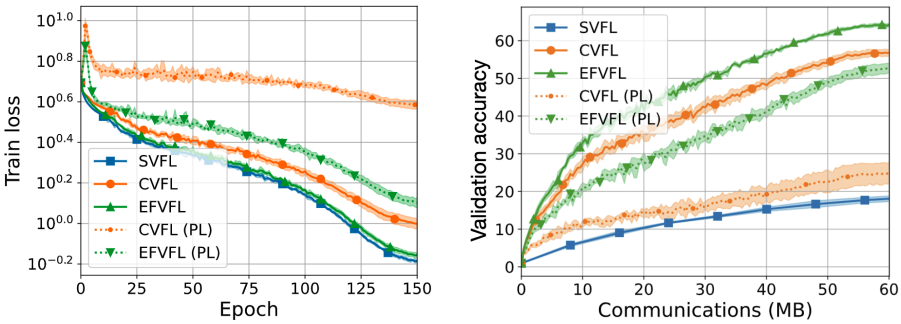
the MNIST experiment and the CIFAR-100 experiment, we see that, while adapting `CVFL` to handle private labels leads to a severe drop in performance, `EF-VFL` slows down much less noticeably. In fact, for the MNIST experiment, `EF-VFL` with private labels still outperforms `CVFL`, even with public labels.

2.5.3 Performance under multiple local updates

As mentioned earlier, some VFL works employ $Q > 1$ local updates per round (Liu et al., 2022b), using stale information from the other machines. We now show that, although our analysis focuses on the case where each client performs a single local update at each round of communications (that is, $Q = 1$), `EF-VFL` performs well in the $Q > 1$ case too. In particular, to study the performance of `EF-VFL` when carrying out multiple local updates, we train an MVCNN on ModelNet10 and a ResNet18 on CIFAR-10.



(a) MNIST, top-k keeping 5%



(b) CIFAR-100, top-k keeping 5%

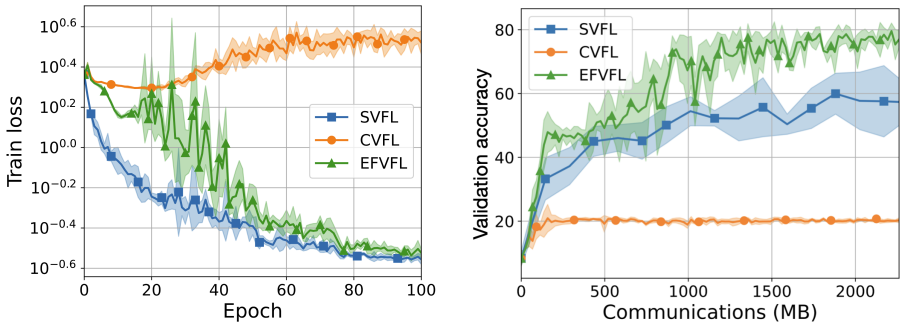
Figure 2.5: Train loss with respect to the number of epochs and validation accuracy with respect to the communication cost for the training of a shallow neural network on MNIST and a ResNet18-based model on CIFAR-100. In the legend, PL stands for private labels. The communication compressed methods—`CVFL`, `EF-VFL`, `CVFL` (PL), and `EF-VFL` (PL)—employ top-k sparsification.

For ModelNet10, all three VFL optimizers use a batch size $B = 128$, a stepsize $\eta = 0.004$, and a weight decay of 0.01. Further, we use a learning rate scheduler, halving the learning rate at epochs 50 and 75. The results are presented in Figure 2.6a and Figure 2.6b. For CIFAR-10, all three VFL optimizers use a batch size $B = 128$, a stepsize $\eta = 0.0025$, and a weight decay of 0.01. Further, we use a learning rate scheduler, halving the learning rate at epochs 40, 60, and 80. The results are presented in Figure 2.6c and Figure 2.6d.

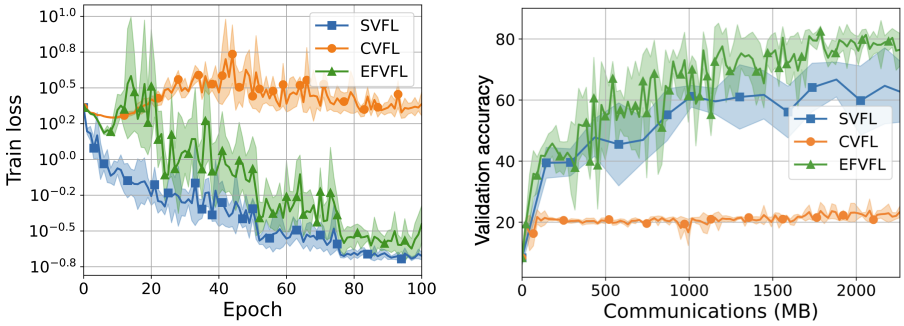
For both ModelNet10 and CIFAR-10, we see that, similarly to the $Q = 1$ case, our method outperforms `SVFL` and `CVFL` in communication efficiency. In terms of results per epoch, `EF-VFL` performs similarly to `SVFL` and significantly better than `CVFL`. Interestingly, for the CIFAR-10 task, `EF-VFL` even outperforms `SVFL` with respect to the number of epochs. We suspect this may be due to the fact that compression helps to mitigate the overly greedy nature of the parallel updates based on stale information.

2.6 Conclusions

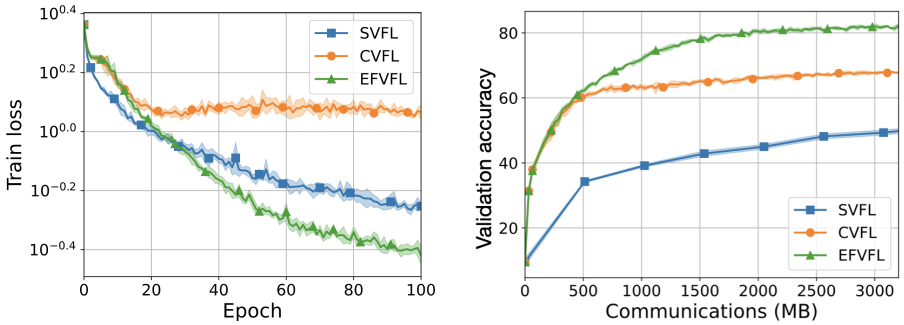
In this chapter, we proposed `EF-VFL`, a method for compressed vertical federated learning. Our method leverages an error feedback mechanism to achieve a $\mathcal{O}(1/T)$ convergence rate for a sufficiently large batch size, improving upon the state-of-the-art rate of $\mathcal{O}(1/\sqrt{T})$. Numerical experiments further demonstrate the faster convergence of our method. We further show that, under the PL inequality, our method converges linearly and introduce a modification of `EF-VFL` supporting the use of private labels. In the future, it would be interesting to study the use of error feedback based compression methods for VFL in the fully-decentralized and semi-decentralized settings, in setups with asynchronous updates, and in combination with privacy mechanisms, such as differential privacy as done in the horizontal setting (Li et al., 2022b; Li and Chi, 2025).



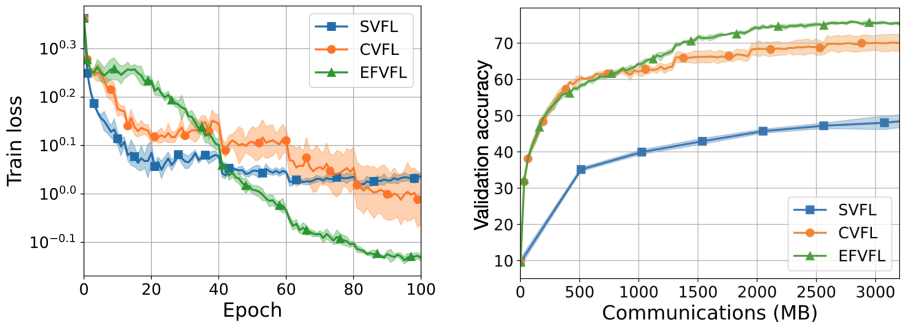
(a) ModelNet10 (qsgd_s with $b = 4$; $Q = 2$)



(b) ModelNet10 (qsgd_s with $b = 4$; $Q = 4$)



(c) CIFAR-10 (qsgd_s with $b = 1$; $Q = 2$)



(d) CIFAR-10 (qsgd_s with $b = 1$; $Q = 4$)

Figure 2.6: Train loss with respect to the number of epochs and validation accuracy with respect to the communication cost for the training of a multi-view convolutional neural network on ModelNet10 and a residual neural network on CIFAR-10. `CVFL` and `EF-VFL` use stochastic quantization. On the left, all three vertical FL optimizers use $Q = 2$ local updates and, on the right, they all use $Q = 4$ local updates.

Chapter 3

VFL with missing features

In this chapter, we address the data efficiency challenge of dealing with missing features in VFL during both training and inference without wasting data. Most VFL methods require all clients to observe their feature block for a given sample to compute its loss; as a result, they cannot leverage incomplete samples, where only a subset of clients observe their partition, either during training or inference. Yet, the assumption that all feature blocks are observed rarely holds in practice.

To ensure that this assumption holds, we can discard incomplete samples, and then apply standard VFL methods. Yet, this wastes data. Wasting incomplete training samples harms generalization, and no supporting incomplete test samples reduces model availability. Further, if a client leaves the federation after training, its partition becomes permanently unavailable, rendering the trained model unusable.

Alternatively, the clients may avoid collaboration altogether and instead train local predictors on their own data, ignoring the feature blocks from other clients, yet, this leads to reduced predictive power. We thus need a VFL method capable of handling missing features during both training and inference without wasting data.

To address this, we propose `LASER-VFL`, a simple, hyperparameter-free method that leverages parameter sharing and task sampling to train and perform inference with split neural networks on arbitrary sets of feature blocks. To the best of our knowledge, this is the first method that is able to handle any subset of feature blocks during both training and inference without wasting data, and it requires only minor modifications to `SVFL`. We provide convergence guarantees and, through numerical experiments on multiple datasets, show that LASER-VFL outperforms baselines when features are missing and even when all features are present.

This chapter is based on our previous publication (Valdeira et al., 2025a).

3.1 Related work

VFL shares challenges with horizontal FL, such as communication efficiency (Liu et al., 2022b; Valdeira et al., 2025b) and privacy preservation (Yu et al., 2024), but also faces unique obstacles, such as missing feature blocks. During training, these unavailable partitions render the observed blocks of other clients unusable, and at test time, they can prevent inference altogether. Therefore, most VFL literature assumes that all features are available for both training and inference—an often unrealistic assumption that has hindered broader adoption of VFL (Liu et al., 2024).

Joint predictors for VFL with missing features during training. To address the problem of missing features during VFL training, some works use nonoverlapping, or nonaligned, samples (that is, samples with missing feature blocks) to improve generalization. In particular, Feng (2022) and He et al. (2024) apply self-supervised learning to leverage nonaligned samples locally for better representation learning, while using overlapping samples for collaborative training of a joint predictor. Alternatively, Kang et al. (2022), Yang et al. (2022), and Sun et al. (2023) employ semi-supervised learning to take advantage of nonaligned samples. Although these methods enable VFL to utilize data that conventional approaches would discard, they still train a joint predictor and thus require all features to be available for inference, which remains a limitation.

Local predictors for VFL with missing features during inference. A recent line of research leverages information from the entire federation to train local predictors. This can be achieved via transfer learning, as in the works by Feng and Yu (2020); Kang et al. (2024) for overlapping training samples and in the works by Liu et al. (2020); Feng et al. (2022) for handling missing features. Knowledge distillation is another approach, as used by Ren et al. (2022); Li et al. (2022a) for overlapping samples and by Li et al. (2023); Huang et al. (2023) which leverage nonaligned samples when training local predictors (only the latter considers scenarios with more than two clients). Xiao et al. (2025) recently proposed using a distributed generative adversarial network for collaborative training on nonoverlapping data and synthetic data generation. These methods yield local predictors that outperform naive local approaches, but fail to utilize valuable information when other feature blocks are available during inference.

Collaborative predictors for VFL with missing features during inference. A few recent works enable a varying number of clients to collaborate during inference. Sun

et al. (2024) employ party-wise dropout during training to mitigate performance drops from missing feature blocks at inference; Gao et al. (2024) introduce a multi-stage approach with complementary knowledge distillation, enabling inference with different subsets of clients for binary classification tasks; and Ganguli et al. (2023) extend Sun et al. (2024) to deal with communication failures during inference in cross-device VFL. However, all of these methods consider the case of fully-observed training data and they all lack convergence guarantees.

3.2 Preliminaries

Our global dataset $\boldsymbol{X}$ is drawn from an input space $\mathcal{X}$ which is partitioned into K feature spaces $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_K$ such that $\boldsymbol{X}_k \subseteq \mathcal{X}_k$, where, recall, $\boldsymbol{X}_k$ is the local dataset of client k. Ideally, we would train a general, unconstrained predictor $\widetilde{h}\colon \mathcal{X} \mapsto \mathcal{Y}$, where $\mathcal{Y}$ is the label space, by solving $\min_{\widetilde{\boldsymbol{\theta}}} \frac{1}{N} \sum_{n=1}^{N} \ell_n(\widetilde{h}(\boldsymbol{x}^n; \widetilde{\boldsymbol{\theta}}))$. However, VFL brings the additional constraint that this must be achieved without sharing the local datasets. That is, for all k, the local dataset $\boldsymbol{X}_k$ must remain at client k. As mentioned earlier, standard VFL methods approximate $\widetilde{h}$ by $h\colon \mathcal{X} \mapsto \mathcal{Y}$, as defined in (1.1), allowing for collaborative training without sharing the local datasets, but they cannot handle missing features during training nor inference.

Another way to train predictors without sharing local data is to learn local predictors. In particular, each client $k \in \mathcal{K}$ can learn the parameters $\boldsymbol{\theta}_k$ of a predictor $h_k : \mathcal{X}_k \mapsto \mathcal{Y}$ to approximate $\widetilde{h}$:

$$h_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_k) \coloneqq g_k(\boldsymbol{f}_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_{\boldsymbol{f}_k}); \boldsymbol{\theta}_{g_k}) \quad \text{where} \quad \boldsymbol{\theta}_k \coloneqq (\boldsymbol{\theta}_{\boldsymbol{f}_k}, \boldsymbol{\theta}_{g_k}). \tag{3.1}$$

The representation models $\boldsymbol{f}_k : \mathcal{X}_k \mapsto \mathcal{E}_k$, where $\mathcal{E}_k$ is the representation space of client k, are as in (1.1), yet the fusion models $g_k : \mathcal{E}_k \mapsto \mathbb{R}$ differ from $g : \mathcal{E}_1 \times \cdots \times \mathcal{E}_K \mapsto \mathbb{R}$ in (1.1) and h_k differs from h. This approach is useful in that h_k allows client k to perform inference (and be trained) independently of other clients, but it does not make use of the features observed by clients $j \neq k$.

More generally, to achieve robustness to missing blocks in the test data while avoiding wasting other features, we wish to be able to perform inference based on any possible subset of blocks, $\mathscr{P}(\mathcal{K}) \setminus \{\varnothing\}$, where $\mathscr{P}(\mathcal{K})$ denotes the power set of $\mathcal{K}$. Standard VFL allows us to obtain a predictor for the blocks $\mathcal{K}$ and the local approach provides us with predictors for the singletons $\{\{i\}\colon i \in \mathcal{K}\}$ in the power set. However, none of the other

subsets of $\mathcal{K}$ is covered by either approach.

A naive way to achieve this would be to train a predictor for each set $\mathcal{I} \in \mathscr{P}(\mathcal{K}) \setminus \{\varnothing\}$ in a decoupled manner. That is, we could train each of the following predictors $\{h_{\mathcal{I}} \colon \prod_{k\in\mathcal{I}} \mathcal{X}_k \mapsto \mathcal{Y}\}$ independently using the collaborative training approach of standard VFL:

$$\left\{h_{\mathcal{I}}(\boldsymbol{x}_{\mathcal{I}}^n; \boldsymbol{\theta}_{\mathcal{I}}) \coloneqq g_{\mathcal{I}}((\boldsymbol{f}_k(\boldsymbol{x}_k^n; \boldsymbol{\theta}_{\boldsymbol{f}_k(\mathcal{I})}) \colon k \in \mathcal{I}); \boldsymbol{\theta}_{g_{\mathcal{I}}}) \colon \mathcal{I} \in \mathscr{P}(\mathcal{K}) \setminus \{\varnothing\}\right\}, \tag{3.2}$$

where $\boldsymbol{\theta}_{\mathcal{I}} = ((\boldsymbol{\theta}_{\boldsymbol{f}_k(\mathcal{I})} \colon k \in \mathcal{I}), \boldsymbol{\theta}_{g_{\mathcal{I}}})$ and $\boldsymbol{x}_{\mathcal{I}}^n = (\boldsymbol{x}_k^n \colon k \in \mathcal{I})$. The fusion models $\{g_{\mathcal{I}}\}$ can either be at one of the clients or at the server. The set of predictors in (3.2) includes the local predictors $\{h_k\}$ in (3.1) and the standard VFL predictor h in (1.1), but also all the other nonempty sets in the power set $\mathscr{P}(\mathcal{K})$.[1] This approach addresses the issue of limited flexibility and robustness in prior methods, which struggle with varying numbers of available or participating clients during inference. However, by requiring the independent training of $2^K - 1$ distinct predictors, it introduces a new challenge: the number of models would grow exponentially with the number of clients, K. Consequently, the associated memory, computation, and communication costs would also increase exponentially.

Further, if the fusion models in (3.2) are all held by a single entity, this setup introduces a dependency of all clients on that entity. To enhance robustness against clients dropping from the federation, we would like each client to be able to ensure inference whenever it has access to its corresponding block of the sample. That is, we want each client $k \in \mathcal{K}$ to train a predictor for every nonempty subset of $\mathcal{K}$ that includes k. We define this family of subsets as $\mathscr{P}_k(\mathcal{K}) \coloneqq \{\mathcal{I} \in \mathscr{P}(\mathcal{K}) \colon k \in \mathcal{I}\}$. This allows client k to make predictions even if the information from blocks observed by other clients is not available and thus cannot be leveraged to improve performance.

In the next section, we present our method, `LASER-VFL`, which enables the efficient training of a family of predictors that can handle scenarios where features from an arbitrary set of clients are missing.

3.3 Proposed method

Key idea. The main challenge, when striving for each client to be able to perform inference from any subset of blocks that includes its own, is to circumvent the exponential complexity arising from the combinatorial explosion of possible combinations of blocks. To address this, in `LASER-VFL`, we *share model parameters* across the predictors leveraging

[1] The notation in (3.2) differs slightly from (1.1) and (3.1), which use a simpler, more specific formulation.

different subsets of blocks and train them so that the representation models allow for good performance across the different combinations of blocks. Further, we train the fusion model at each client to handle any combination of representations that includes its own. To train the predictors on an exponential number of combinations of missing blocks while avoiding an exponential computational complexity, we employ a *sampling* mechanism during training, which allows us to essentially estimate an exponential combination of objectives with a subexponential complexity.

3.3.1 A tractable family of predictors sharing model parameters

In our approach, each client $k \in \mathcal{K}$ resorts to a set of predictors $\{h_{k\mathcal{I}} \colon \mathcal{I} \in \mathscr{P}_k(\mathcal{K})\}$. Each predictor $h_{k\mathcal{I}} \colon \prod_{i\in\mathcal{I}} \mathcal{X}_i \mapsto \mathcal{Y}$ has the same target, but a different domain. Therefore, we say that each predictor performs a distinct *task*. We define the predictors of client k as follows:

$$\left\{ h_{k\mathcal{I}}\left(\boldsymbol{x}_{\mathcal{I}}^n; \boldsymbol{\theta}_{(k,\mathcal{I})}\right) \coloneqq g_k\left(\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \boldsymbol{f}_i(\boldsymbol{x}_i^n; \boldsymbol{\theta}_{\boldsymbol{f}_i}); \boldsymbol{\theta}_{g_k}\right) : \mathcal{I} \in \mathscr{P}_k(\mathcal{K}) \right\}, \tag{3.3}$$

where $\boldsymbol{\theta}_{(k,\mathcal{I})} = ((\boldsymbol{\theta}_{\boldsymbol{f}_i} \colon i \in \mathcal{I}), \boldsymbol{\theta}_{g_k})$ and $|\mathcal{I}|$ denotes the cardinality of set $\mathcal{I}$.

Sharing model parameters. Note that, although we can see (3.3) as $|\mathscr{P}_k(\mathcal{K})| = 2^{K-1}$ different predictors, the predictors are made up of different combinations of only K representation models and K fusion models. That is, we only require parameters $\boldsymbol{\theta} = (\boldsymbol{\theta}_{\boldsymbol{f}_1}, \ldots, \boldsymbol{\theta}_{\boldsymbol{f}_K}, \boldsymbol{\theta}_{g_1}, \ldots, \boldsymbol{\theta}_{g_K})$.[2] In particular, in contrast to (3.2), where the predictors used different representation models for each task, in (3.3), we share the representation models across predictors. Similarly to the representation models, we have a single fusion model per client.

It is also important to note that each fusion model $g_k\left((\boldsymbol{f}_i(\boldsymbol{x}_i^n; \boldsymbol{\theta}_{\boldsymbol{f}_i}) \colon i \in \mathcal{I}); \boldsymbol{\theta}_{g_k}\right)$ takes the specific form of $g_k\left(\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \boldsymbol{f}_i(\boldsymbol{x}_i^n; \boldsymbol{\theta}_{\boldsymbol{f}_i}); \boldsymbol{\theta}_{g_k}\right)$. By employing a nonparameterized aggregation mechanism on the extracted representations whose output does not depend on the number of aggregated representations, the same fusion model can handle different sets of representations of different sizes. In particular, we opt for an average (rather than, say, a sum) because neural networks perform better when their inputs have similar distributions (Ioffe and Szegedy, 2015). Note that the representation model can be adjusted so that using averaging instead of concatenation as the aggregation mechanism does not

[2] We use $\boldsymbol{\theta}$ to refer to all the model parameters, as in other chapters. Yet, with some abuse of notation, in this chapter this includes the parameters of multiple fusion models.

reduce the flexibility of the overall model, but simply shifts it from the fusion model to the representation models, as explained in Appendix B.1.2.

With this approach, we have avoided an exponential memory complexity by sharing the weights across an exponential number of predictors such that we have K representation models and K fusion models. In the next subsection, we will go over the efficient training of this family of predictors.

3.3.2 Efficient training via task-sampling

The family of predictors introduced in Section 3.3.1 can efficiently perform inference for an exponential number of tasks. We will now discuss how to train this family of predictors while avoiding exponential computation and communication complexity during the training process. The key to this approach lies in carefully sampling tasks at each gradient step of model training.

Our optimization problem. As noted earlier, we want to address missing features not only during inference, but also during training. Let $\mathcal{K}_o^n$ be the set of observed feature blocks of sample n, we assume that $\mathcal{K}_o := \{\mathcal{K}_o^n : \forall n\}$ follows a random distribution. Recalling (3.3), we denote the loss ℓ of sample n, with label y^n, for the predictor of client k using blocks $\mathcal{I} \in \mathscr{P}_k(\mathcal{K}_o^n)$ of $\boldsymbol{x}^n$ as follows:

$$\mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}) := \ell_n \left(h_{k\mathcal{I}} \left(\boldsymbol{x}_{\mathcal{I}}^n; \boldsymbol{\theta}_{(k,\mathcal{I})} \right) \right).$$

For sample n and client k, the (weighted) prediction loss across all observed subsets of blocks that include k is denoted by:

$$\mathcal{L}_{nk}^o(\boldsymbol{\theta}) := \sum_{\mathcal{I} \in \mathscr{P}_k(\mathcal{K}_o^n)} \frac{1}{|\mathcal{I}|} \cdot \mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}),$$

where superscript o indicates that this loss concerns the observed dataset. The normalization by $|\mathcal{I}|$ avoids assigning a larger weight to tasks concerning a larger number of blocks, since the set $\mathcal{I}$ is used for prediction at $|\mathcal{I}|$ different clients, whose losses are combined in the loss of sample n:

$$\mathcal{L}_n^o(\boldsymbol{\theta}) := \sum_{k \in \mathcal{K}_o^n} \mathcal{L}_{nk}^o(\boldsymbol{\theta}). \tag{3.4}$$

Algorithm 3: `LASER-VFL` Training

Input: initial point $\boldsymbol{\theta}^0$, training data $\mathcal{D}$.

for $t = 0, \dots, T-1$ **do**

  Clients $\mathcal{K}$ sample the same mini-batch $\mathcal{B}^t$ with $\mathcal{K}_o^{(t)} = \mathcal{K}_o^n$, $\forall n \in \mathcal{B}^t$, via a shared seed.

  for $k \in \mathcal{K}_o^{(t)}$ **in parallel do**

    Broadcast $\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_{\boldsymbol{f}_k}^t)$ and receive $\{\boldsymbol{F}_i^{\mathcal{B}^t}(\boldsymbol{\theta}_{\boldsymbol{f}_i}^t) : i \in \mathcal{K}_o^{(t)}, i \neq k\}$.

    Sample a task-defining set of blocks $\mathcal{S}_{kj}^t \sim \mathcal{U}(\mathscr{P}_k^j(\mathcal{K}_o^{(t)}))$ for $j = 1, \dots, K_o^t$.

    Compute $\widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}_k^t}(\boldsymbol{\theta}^t)$, as in (3.7), and backpropagate over fusion model g_k.

    Send the derivative of $\widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}_k^t}(\boldsymbol{\theta}^t)$ with respect to each $k \in \mathcal{K}_o^{(t)}$ and receive theirs.

    Sum the received gradients and backpropagate over $\boldsymbol{f}_k$.

  Update the weights $\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \eta \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t)$, where $\widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t)$ is as in (3.6).

Now, to train the predictors in (3.3), we consider the following optimization problem:

$$\min_{\boldsymbol{\theta}} \{\mathcal{L}(\boldsymbol{\theta}) \coloneqq \mathbb{E}[\mathcal{L}^o(\boldsymbol{\theta})]\} \quad \text{where} \quad \mathcal{L}^o(\boldsymbol{\theta}) \coloneqq \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_n^o(\boldsymbol{\theta}), \tag{3.5}$$

where the expectation is over the set of observed feature blocks, $\mathcal{K}_o$.[3]

Note that, if different blocks of features have different probabilities of being observed, we can also normalize the loss with respect to the different probabilities, which can be computed during the entity alignment stage of VFL (see below), before taking the expectation in (3.4). We omit this here for simplicity. We assume that the data is missing completely at random (Zhou et al., 2024).

Looking at (3.4), we see that, while we are interested in tackling $K \times 2^{K-1}$ different tasks, which falls in the realm of multi-objective optimization and multi-task learning (Caruana, 1997), we opt for combining them into a single objective, rather than employing a specialized multi-task optimizer. This corresponds to a weighted form of unitary scalarization (Kurin et al., 2022).

Our optimization method. To solve (3.5), we employ a gradient-based method, summarized in Algorithm 3, which we now describe. At iteration t, with the current iterate $\boldsymbol{\theta}^t$,

[3]With some abuse of notation, we use $\mathcal{L}$ to refer to the objective function of interest in this chapter, which differs from the original objective in standard VFL (1.1).

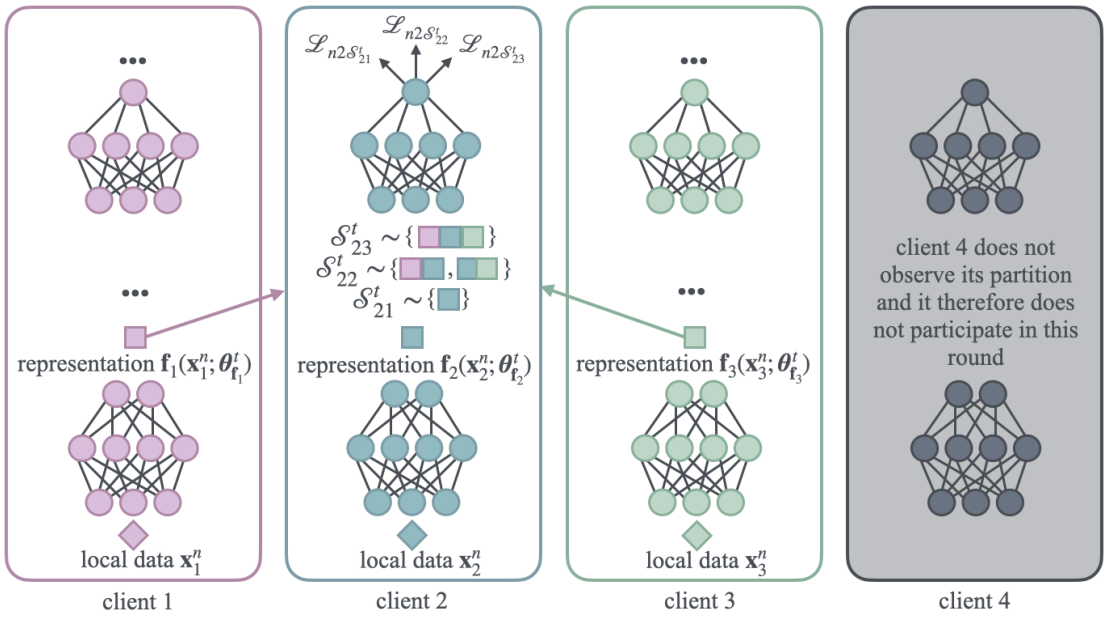


Figure 3.1: An illustration of a forward pass in `LASER-VFL`, including the task-sampling mechanism.

we select mini-batch indices $\mathcal{B}^t \subseteq \{1, \dots, N\}$, such that the set of observed feature blocks for each sample $n \in \mathcal{B}^t$, denoted by $\mathcal{K}_o^n$, is identical. We denote this common set by $\mathcal{K}_o^{(t)}$, that is, $\mathcal{K}_o^{(t)} = \mathcal{K}_o^n$ for all $n \in \mathcal{B}^t$, and define $\mathcal{L}_{\mathcal{B}}^o(\boldsymbol{\theta}) \coloneqq \frac{1}{|\mathcal{B}|} \sum_{n \in \mathcal{B}} \mathcal{L}_n^o(\boldsymbol{\theta})$. Then, each client $k \in \mathcal{K}_o^{(t)}$ broadcasts its representation

$$\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_{\boldsymbol{f}_k}^t)$$

to the other clients in $\mathcal{K}_o^{(t)}$. At this stage, each client $k \in \mathcal{K}_o^{(t)}$ holds the representations corresponding to the set of observed feature blocks of the current mini-batch, $\{\boldsymbol{F}_i^{\mathcal{B}^t}(\boldsymbol{\theta}_{\boldsymbol{f}_i}^t) \colon i \in \mathcal{K}_o^{(t)}\}$.

To train the model on all possible combinations of feature blocks without incurring exponential computational complexity at each iteration, each client $k \in \mathcal{K}_o^{(t)}$ samples $K_o^t \coloneqq |\mathcal{K}_o^{(t)}|$ tasks from the $2^{\mathcal{K}_o^{(t)}-1}$ tasks in $\mathscr{P}_k(\mathcal{K}_o^{(t)})$—the subset of the power set of $\mathcal{K}_o^{(t)}$ whose elements contain k. These sampled tasks estimate $\mathcal{L}_{\mathcal{B}^t}^o(\boldsymbol{\theta}^t)$ without computing predictions for an exponential number of tasks (note that $K_o^t \leq K$). We use this estimate to update the model parameters.

More precisely, the task-defining feature blocks sampled by client k at step t are given by $\mathcal{S}_k^t \coloneqq \{\mathcal{S}_{k1}^t, \dots, \mathcal{S}_{kK_o^t}^n\}$, where set $\mathcal{S}_{ki}^t$ contains i feature blocks. Each set $\mathcal{S}_{ki}^t$ is sampled from a uniform distribution over a subset of $\mathscr{P}_k(\mathcal{K}_o^{(t)})$ that contains its sets of size i. That

Algorithm 4: `LASER-VFL` Inference

Input: test sample $\boldsymbol{x}'$, trained parameters $\boldsymbol{\theta}^T$.

The blocks of features $\mathcal{K}'_o$ of sample $\boldsymbol{x}'$ are observed.

for $k \in \mathcal{K}'_o$ **in parallel do**

  Broadcast $\boldsymbol{f}_k(\boldsymbol{x}'_k; \boldsymbol{\theta}^T_{\boldsymbol{f}_k})$ and receive $\boldsymbol{f}_i(\boldsymbol{x}'_i; \boldsymbol{\theta}^T_{\boldsymbol{f}_i})$, for all $i \in \mathcal{K}'_o \setminus \{k\}$.

  Client k predicts the label using $h_{k\mathcal{K}'_o}\left(\boldsymbol{x}^n_{\mathcal{K}'_o}; \boldsymbol{\theta}_{(k,\mathcal{K}'_o)}\right)$.

is:

$$\mathcal{S}^t_{ki} \sim \mathcal{U}(\mathscr{P}^i_k(\mathcal{K}^{(t)}_o)) \quad \text{where} \quad \mathscr{P}^i_k(\mathcal{K}^{(t)}_o) \coloneqq \{\mathcal{I} \in \mathscr{P}_k(\mathcal{K}^{(t)}_o) \colon |\mathcal{I}| = i\}.$$

This task-sampling mechanism allows us to estimate the loss with linear complexity, instead of the exponential cost of exact computation. More precisely, we use the following loss estimate:

$$\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t) \coloneqq \sum_{k \in \mathcal{K}^{(t)}_o} \widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t_k}(\boldsymbol{\theta}^t), \tag{3.6}$$

where $\mathcal{S}^t \coloneqq \{\mathcal{S}^t_1, \ldots, \mathcal{S}^t_K\}$ and each $\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t_k}(\boldsymbol{\theta}^t)$, computed at client k, is given by:

$$\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t_k}(\boldsymbol{\theta}^t) \coloneqq \frac{1}{|\mathcal{B}^t|} \sum_{n \in \mathcal{B}^t} \sum_{i=1}^{K^t_o} a^t_i \cdot \mathcal{L}_{nk\mathcal{S}^t_{ki}}(\boldsymbol{\theta}^t) \quad \text{where} \quad a^t_i \coloneqq \frac{1}{i} \cdot \binom{K^t_o - 1}{i-1}. \tag{3.7}$$

The weighting a^t_i counteracts the probability of each task being sampled, allowing us to obtain an unbiased estimate of the observed loss. Now, we use this to minimize (3.4) by doing the following update:

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \eta \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t).$$

We illustrate a forward pass of `LASER-VFL` in Figure 3.1.

After completing the training described in Algorithm 3, we can perform inference according to Algorithm 4. In this inference algorithm, each client $k \in \mathcal{K}'_o$ that observes a partition of the test sample $\boldsymbol{x}'$ shares its corresponding representation of $\boldsymbol{x}'$. Then, each such client k uses $h_{(k,\mathcal{K}'_o)}$ to predict the label.

Block availability-aware entity alignment. Entity alignment is a process the precedes training in VFL where the samples of the different local datasets are matched so that their features are correctly aligned, thus allowing for collaborative model training (Liu et al., 2024). The identification of the available blocks for each sample can be performed during this prelude to VFL training, allowing for a batch selection procedure that ensures

that each batch contains samples with the same observed blocks, which we use in our method.

Extensions to our method. It is important to highlight that our method can also be easily employed in situations where different clients hold different labels, as it naturally fits multi-task learning problems. Further, it can easily be applied to setups where any subset of $\mathcal{K}$ holds the labels, sufficing to drop fusion models from the clients which do not hold the labels or have them tackle unsupervised or self-supervised learning tasks instead.

3.4 Convergence guarantees

In this section, we present convergence guarantees for our method. We make the following assumptions in our results.

- We assume that $\mathcal{L}^o$ is L-smooth (D2) and has a finite infimum $\mathcal{L}^\star$ (D1). This holds if $\mathcal{L}$ is L'-smooth.
- We assume that the mini-batch gradient estimate $\nabla\mathcal{L}^o_{\mathcal{B}}(\boldsymbol{\theta})$ is an unbiased estimate of $\nabla\mathcal{L}^o(\boldsymbol{\theta})$ (D3) with variance bounded by conditional on $\mathcal{K}_o$ (D4).
- $\nabla\widehat{\mathcal{L}}_{\mathcal{B}\mathcal{S}_k}(\boldsymbol{\theta})$, which we show in Lemma 3 to be an unbiased estimate of $\nabla\mathcal{L}^o_{\mathcal{B}k}(\boldsymbol{\theta})$, where $\mathcal{L}^o_{\mathcal{B}k}(\boldsymbol{\theta}) \coloneqq \frac{1}{|\mathcal{B}|}\sum_{n\in\mathcal{B}}\mathcal{L}^o_{nk}(\boldsymbol{\theta})$, has a variance bounded by σ_s^2 conditional on $\mathcal{K}_o$ and $\mathcal{B}$ (D4), where the (conditional) expectations is with respect to the tasks sampled at each client k, $\mathcal{S}_k$.

We use the following set in our results:

$$\mathcal{F}^t \coloneqq \{\mathcal{B}^0, \mathcal{S}^0, \mathcal{B}^1, \mathcal{S}^1, \ldots, \mathcal{B}^{t-1}, \mathcal{S}^{t-1}\}.$$

Let us start by presenting a lemma which we resort to in our proof of Theorem 3.

Lemma 3 (Unbiased update vector)**.** *If the mini-batch gradient estimate is unbiased* (D3)*, then, for all* $t \geq 0$*:*

$$\mathbb{E}\left[\nabla\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t) \mid \mathcal{K}_o, \mathcal{F}^t\right] = \nabla\mathcal{L}^o(\boldsymbol{\theta}^t), \tag{3.8}$$

where the expectation is with respect to mini-batch $\mathcal{B}^t$ *and the sampled set of tasks* $\mathcal{S}^t$*.*

Theorem 3 (Main result)**.** *Let* $\{\boldsymbol{\theta}^t\}$ *be a sequence generated by Algorithm 3, if* $\mathcal{L}$ *is* L*-smooth and has a finite infimum* (D2)*, the mini-batch gradient estimate* $\nabla\mathcal{L}^o_{\mathcal{B}}(\boldsymbol{\theta})$ *is an*

unbiased estimate of $\nabla\mathcal{L}^o(\boldsymbol{\theta})$ (D3) *with variance bounded by conditional on* $\mathcal{K}_o$ (D4)*, and our update vector* $\nabla\widehat{\mathcal{L}}_{\mathcal{BS}_k}(\boldsymbol{\theta})$ *has a variance bounded by* σ_s^2 *conditional on* $\mathcal{K}_o$ *and* $\mathcal{B}$ (D4)*, we then have that, for* $\eta \in (0, 1/L]$*:*

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\right\|^2 \leq \frac{2\Delta}{\eta T} + \eta L\left(\sigma_b^2 + K\cdot\sigma_s^2\right), \tag{3.9}$$

where $\Delta \coloneqq \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$ *and the expectation is with respect to* $\{\mathcal{B}^t\}$*,* $\{\mathcal{S}^t\}$*, and* $\mathcal{K}_o$*.*

If we take the stepsize to be $\eta = \sqrt{\frac{2\Delta}{LT}\left(\sigma_b^2 + K\cdot\sigma_s^2\right)^{-1}}$, we get from (3.9) that

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\right\|^2 \leq \sqrt{\frac{8\Delta L}{T}\left(\sigma_b^2 + K\cdot\sigma_s^2\right)},$$

thus achieving a $\mathcal{O}(1/\sqrt{T})$ convergence rate. Note the effect of the variance induced by the task-sampling mechanism, which grows with K.

Linear convergence. We can further establish linear convergence of `LASER-VFL` to a neighborhood around the optimum under the assumption that the PL inequality holds for $\mathcal{L}$ (D5).

We use the expected suboptimality $\delta^t \coloneqq \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star$, where the expectation is with respect to $\{\mathcal{B}^t\}$, $\{\mathcal{S}^t\}$, and $\mathcal{K}_o$, as our Lyapunov function in the following result.

Theorem 4 (Linear convergence)**.** *Let* $\{\boldsymbol{\theta}^t\}$ *be a sequence generated by Algorithm 3, if* $\mathcal{L}$ *is* L*-smooth and has a finite infimum* (D2)*, the mini-batch gradient estimate* $\nabla\mathcal{L}^o_{\mathcal{B}}(\boldsymbol{\theta})$ *is an unbiased estimate of* $\nabla\mathcal{L}^o(\boldsymbol{\theta})$ (D3) *with variance bounded by conditional on* $\mathcal{K}_o$ (D4)*, our update vector* $\nabla\widehat{\mathcal{L}}_{\mathcal{BS}_k}(\boldsymbol{\theta})$ *has a variance bounded by* σ_s^2 *conditional on* $\mathcal{K}_o$ *and* $\mathcal{B}$ (D4)*, and the PL inequality* (D5) *holds for* $\mathcal{L}$*, we then have that, for* $\eta \in (0, 1/L]$*:*

$$\delta^T \leq (1-\mu\eta)^T\delta^0 + \frac{\eta L}{2\mu}\left(\sigma_b^2 + K\cdot\sigma_s^2\right).$$

It follows from $\mu \leq L$ that $1 - \mu\eta \in (0, 1)$. Therefore, we have linear convergence to a $\mathcal{O}\left(\sigma_b^2 + K\cdot\sigma_s^2\right)$ neighborhood of the global optimum.

We defer the proofs of Lemma 3, Theorem 3, and Theorem 4 to Appendix B.

3.5 Experiments

In this section, we describe our numerical experiments and analyze their results, comparing the empirical performance of `LASER-VFL` against baseline approaches. We begin with a brief overview of the baseline methods and tasks considered, followed by a the presentation and discussion of the experimental results.

We compare our method to the following baselines:

- `Local`: as in (3.1); the local approach leverages its block for training and inference whenever it is observed, but ignores the remaining blocks.
- `Standard VFL` (Liu et al., 2022b): as in (1.1); the standard VFL model can only be trained on and used for inference for fully-observed samples. When the model is unavailable for prediction, a random prediction is made.
- `VFedTrans` (Huang et al., 2023): `VFedTrans` introduces a multi-stage VFL training pipeline to collaboratively train local predictors. (We give more details about VFedTrans below.)
- `Ensemble`: we train local predictors as in `Local`. During inference, the clients share their predictions, and a joint prediction is selected by majority vote, with ties broken at random.
- `Combinatorial`: as in (3.2); during training, each batch is used by all the models corresponding to subsets of the observed blocks. During inference, we use the predictor corresponding to the set of observed blocks. We go over the scalability problems of this approach below.
- `PlugVFL` (Sun et al., 2024): we extend `PlugVFL` to handle missing features during training by replacing missing representations with zeros. This method matches the work proposed by (Ganguli et al., 2023) for scenarios where all clients have reliable links to one another.

Our experiments focus on the following tasks:

- **HAPT** (Reyes-Ortiz et al., 2016)**:** a human activity recognition dataset.
- **Credit** (Yeh and Lien, 2009)**:** a dataset for predicting credit card payment defaults.

Table 3.1: Test metrics for $K = 4$ clients across different datasets and varying probabilities of missing blocks during training and inference, averaged over five seeds ($\pm$ standard deviation).

Training p_{miss}		0.0			0.1			0.5	
Inference p_{miss}	0.0	0.1	0.5	0.0	0.1	0.5	0.0	0.1	0.5
					HAPT (accuracy, %)				
`Local`	82.3 ± 0.4	82.5 ± 0.5	82.1 ± 1.2	81.7 ± 0.6	81.8 ± 0.4	81.5 ± 1.6	80.7 ± 0.9	80.8 ± 0.8	80.5 ± 2.1
`Standard VFL`	91.6 ± 0.9	67.3 ± 3.5	16.0 ± 4.1	90.0 ± 1.4	66.0 ± 2.8	15.8 ± 3.9	84.6 ± 2.5	62.2 ± 2.2	15.3 ± 3.5
`VFedTrans`	82.5 ± 0.4	82.5 ± 0.4	82.3 ± 0.7	82.9 ± 0.3	82.9 ± 0.2	82.8 ± 0.7	82.2 ± 0.5	82.2 ± 0.5	82.2 ± 0.8
`Ensemble`	90.9 ± 0.9	90.3 ± 0.9	84.8 ± 1.1	89.8 ± 0.4	89.3 ± 0.5	84.1 ± 1.6	88.7 ± 1.5	88.1 ± 1.8	83.0 ± 2.5
`Combinatorial`	92.5 ± 0.2	92.4 ± 0.3	88.5 ± 1.7	90.0 ± 1.6	90.0 ± 1.4	$\mathbf{87.3} \pm 1.6$	84.2 ± 2.5	84.7 ± 1.9	83.0 ± 1.3
`PlugVFL`	91.1 ± 1.4	88.5 ± 1.6	75.1 ± 6.3	90.9 ± 0.9	88.6 ± 2.6	78.3 ± 4.8	87.7 ± 1.6	87.2 ± 1.5	82.8 ± 1.6
`LASER-VFL` (ours)	$\mathbf{94.2} \pm 0.1$	$\mathbf{93.9} \pm 0.2$	$\mathbf{89.0} \pm 1.1$	$\mathbf{92.4} \pm 1.3$	$\mathbf{92.0} \pm 1.3$	86.6 ± 1.2	$\mathbf{90.1} \pm 3.2$	$\mathbf{89.5} \pm 3.1$	$\mathbf{84.8} \pm 2.6$
					Credit (F_1-score$\times$100)				
`Local`	37.7 ± 3.1	37.6 ± 3.1	37.5 ± 3.2	37.6 ± 3.0	37.6 ± 2.9	37.5 ± 3.2	36.2 ± 3.5	36.2 ± 3.6	36.0 ± 3.6
`Standard VFL`	45.7 ± 2.6	38.8 ± 2.5	31.0 ± 0.7	42.4 ± 1.2	38.3 ± 0.7	31.0 ± 0.7	32.4 ± 2.4	32.5 ± 1.3	30.3 ± 0.5
`VFedTrans`	37.7 ± 1.5	37.6 ± 1.4	37.2 ± 1.6	39.5 ± 0.8	39.4 ± 0.8	39.2 ± 0.9	35.7 ± 0.3	35.6 ± 0.3	35.6 ± 0.4
`Ensemble`	42.1 ± 1.0	42.1 ± 1.0	40.1 ± 0.8	41.4 ± 1.3	40.8 ± 0.9	39.2 ± 1.1	$\mathbf{42.4} \pm 2.1$	$\mathbf{41.8} \pm 1.7$	40.1 ± 1.1
`Combinatorial`	42.8 ± 2.9	42.7 ± 2.5	41.7 ± 1.5	$\mathbf{44.4} \pm 1.0$	41.8 ± 2.2	$\mathbf{41.7} \pm 1.5$	32.8 ± 3.6	36.3 ± 0.6	37.6 ± 2.1
`PlugVFL`	44.4 ± 3.7	41.3 ± 4.0	32.7 ± 4.6	40.8 ± 2.0	39.8 ± 1.6	37.9 ± 1.7	38.6 ± 2.1	37.8 ± 0.9	35.4 ± 3.4
`LASER-VFL` (ours)	$\mathbf{46.5} \pm 2.8$	$\mathbf{45.0} \pm 2.5$	$\mathbf{43.7} \pm 1.2$	43.1 ± 4.2	$\mathbf{41.9} \pm 4.0$	41.3 ± 1.9	41.5 ± 4.2	40.9 ± 4.0	$\mathbf{41.4} \pm 1.7$
					MIMIC-IV (F_1-score$\times$100)				
`Local`	74.9 ± 0.5	74.9 ± 0.5	74.8 ± 0.5	75.0 ± 0.3	75.0 ± 0.2	75.0 ± 0.3	73.0 ± 0.5	73.0 ± 0.5	73.0 ± 0.5
`Standard VFL`	91.6 ± 0.2	77.0 ± 1.3	52.5 ± 2.2	90.9 ± 0.3	76.8 ± 1.3	52.5 ± 2.2	81.1 ± 0.4	70.2 ± 0.8	51.8 ± 1.8
`Ensemble`	84.2 ± 0.3	83.9 ± 0.3	77.9 ± 1.6	84.1 ± 0.4	83.7 ± 0.5	78.1 ± 1.1	82.1 ± 0.3	81.8 ± 0.5	76.4 ± 1.1
`Combinatorial`	91.3 ± 0.3	87.2 ± 1.7	83.6 ± 0.4	91.0 ± 0.3	86.7 ± 1.4	83.4 ± 0.4	80.5 ± 1.2	80.6 ± 1.7	78.9 ± 0.5
`PlugVFL`	91.2 ± 0.5	89.0 ± 0.8	79.6 ± 2.8	90.6 ± 0.2	88.8 ± 0.5	79.8 ± 2.6	86.5 ± 0.7	85.3 ± 0.9	77.9 ± 2.8
`LASER-VFL` (ours)	$\mathbf{92.0} \pm 0.2$	$\mathbf{91.2} \pm 0.3$	$\mathbf{85.5} \pm 1.0$	$\mathbf{91.1} \pm 0.5$	$\mathbf{90.2} \pm 0.3$	$\mathbf{84.7} \pm 0.9$	$\mathbf{87.7} \pm 0.2$	$\mathbf{86.9} \pm 0.2$	$\mathbf{82.0} \pm 1.0$
					CIFAR-10 (accuracy, %)				
`Local`	76.1 ± 0.1	76.0 ± 0.1	76.2 ± 0.2	75.6 ± 0.1	75.6 ± 0.1	75.7 ± 0.3	71.2 ± 0.4	71.1 ± 0.4	71.3 ± 0.7
`Standard VFL`	89.7 ± 0.2	60.5 ± 12.1	11.6 ± 3.5	87.0 ± 0.6	58.8 ± 11.4	11.5 ± 3.4	54.9 ± 10.0	38.6 ± 9.1	10.9 ± 2.2
`Ensemble`	86.4 ± 0.2	85.2 ± 0.7	78.3 ± 0.9	86.1 ± 0.3	85.0 ± 0.5	77.8 ± 0.7	82.4 ± 0.2	81.0 ± 0.8	73.8 ± 0.6
`Combinatorial`	89.4 ± 0.1	88.7 ± 0.4	83.4 ± 1.2	87.1 ± 0.8	86.7 ± 0.4	82.4 ± 0.8	54.9 ± 9.7	58.6 ± 7.7	68.4 ± 3.1
`PlugVFL`	89.4 ± 0.2	88.1 ± 0.8	81.8 ± 1.7	88.1 ± 0.5	86.9 ± 0.6	81.2 ± 1.2	76.5 ± 2.1	75.6 ± 1.5	72.4 ± 1.0
`LASER-VFL` (ours)	$\mathbf{91.5} \pm 0.1$	$\mathbf{90.5} \pm 0.4$	$\mathbf{83.8} \pm 1.5$	$\mathbf{91.2} \pm 0.2$	$\mathbf{90.2} \pm 0.4$	$\mathbf{83.3} \pm 1.4$	$\mathbf{87.4} \pm 0.3$	$\mathbf{86.4} \pm 0.6$	$\mathbf{79.4} \pm 1.6$
					CIFAR-100 (accuracy, %)				
`Local`	50.3 ± 0.1	50.3 ± 0.1	50.4 ± 0.1	49.1 ± 0.2	49.2 ± 0.2	49.1 ± 0.3	41.3 ± 0.7	41.3 ± 0.6	41.3 ± 0.7
`Standard VFL`	64.7 ± 0.4	41.5 ± 9.7	2.4 ± 2.9	57.2 ± 2.2	36.6 ± 7.6	2.3 ± 2.8	15.8 ± 7.0	10.5 ± 4.6	1.4 ± 1.0
`Ensemble`	62.1 ± 0.3	60.2 ± 1.2	52.3 ± 0.9	60.8 ± 0.3	58.8 ± 1.0	51.0 ± 0.5	52.6 ± 0.6	50.4 ± 0.7	43.5 ± 0.8
`Combinatorial`	64.4 ± 0.5	64.0 ± 0.4	58.5 ± 1.3	57.2 ± 2.2	57.8 ± 1.8	55.5 ± 0.7	16.4 ± 6.9	19.5 ± 5.9	31.6 ± 4.3
`PlugVFL`	66.0 ± 0.3	64.1 ± 1.2	55.3 ± 2.1	63.2 ± 1.1	61.5 ± 0.7	53.1 ± 1.9	44.6 ± 2.3	43.9 ± 1.7	41.2 ± 1.6
`LASER-VFL` (ours)	$\mathbf{72.3} \pm 0.1$	$\mathbf{71.0} \pm 0.7$	$\mathbf{61.3} \pm 1.8$	$\mathbf{70.4} \pm 0.3$	$\mathbf{68.9} \pm 0.6$	$\mathbf{59.8} \pm 1.6$	$\mathbf{61.4} \pm 0.9$	$\mathbf{59.9} \pm 0.5$	$\mathbf{51.9} \pm 1.2$

- **MIMIC-IV** (Johnson et al., 2020)**:** a time series dataset concerning healthcare data. We focus on the task of predicting in-hospital mortality from ICU data corresponding to patients admitted with chronic kidney disease. We resort to the data processing pipeline of Gupta et al. (2022).

- **CIFAR-10 & CIFAR-100** (Krizhevsky and Hinton, 2009)**:** two popular image datasets.

For all datasets, we split the samples into $K = 4$ feature blocks. For example, for the CIFAR datasets, each image is partitioned into four quadrants, each assigned to a different client. Given our interest in ensuring that all clients perform well at inference time, we average the metrics across clients. We now describe how we compute accuracy

and F_1-score.

Computing metrics. In order for the metrics to capture that fact that we want all clients to perform well during inference, we consider the average metrics across the clients. More precisely, we compute the metrics for Table 3.1 as follows:

- For the test accuracy, let $\widehat{y}^n(k)$ denote the prediction of client k for sample n, we compute:

$$\text{accuracy} = 100 \times \frac{1}{N} \sum_{n=1}^{N} \left(\frac{1}{|\mathcal{K}_o^n|} \sum_{k \in \mathcal{K}_o^n} \mathbf{1}(\widehat{y}^n(k) = y^n) \right).$$

- For the F_1-score, we compute:

$$F_1\text{-score} = \frac{1}{K} \sum_{k=1}^{K} \frac{P_k \cdot R_k}{P_k + R_k},$$

  where P_k and R_k are the precision and recall of client k, with respect to its predictor and (only) the samples it observed.

Experiments on HAPT and Credit show that `VFedTrans` does not significantly outperform `Local` despite being considerably more complex and slower. The training pipeline of `VFedTrans` includes three stages: federated representation learning, local representation distillation, and training a local model. The first two stages are repeated $K-1$ times for each client pair. This makes `VFedTrans` time-intensive to run and hyperparameter tune, as seen in Appendix B.3. For this reason, we limit our experiments with `VFedTrans` to these simpler tasks.

Performance for different missing data probabilities. In Table 3.1, we see that the methods with local predictors (`Local` and `VFedTrans`) are robust to missing blocks during both training and inference, yet they fall behind when all the blocks are observed. In contrast, `Standard VFL` performs well when all the blocks are observed, yet its performance degrades very quickly in the presence of missing blocks. The `Ensemble` method improves upon `Standard VFL` significantly in the presence of missing blocks, as it leverages the robustness its local predictors. Yet, when all the blocks are observed, although `Ensemble` improves over the local predictors significantly, it still cannot match the performance of `Standard VFL`. We see that the `Combinatorial` approach outperforms most baselines when there is little to no missing training data ($p_{\text{miss}} \in \{0.0, 0.1\}$). Yet, for a larger amount of

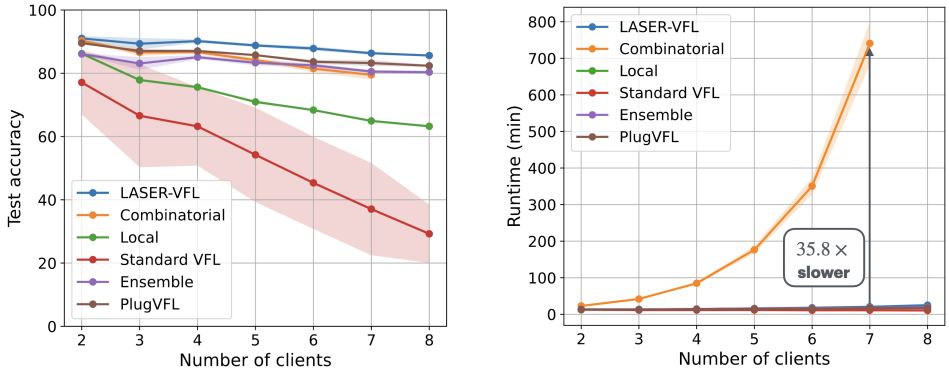


Figure 3.2: Performance and runtime across different numbers of clients (CIFAR-10, $p_{\text{miss}} = 0.1$ for training and inference). We did not run `Combinatorial` for 8 clients due to resource constraints.

missing training data ($p_{\text{miss}} = 0.5$), its performance degrades significantly. This degradation is due to the fact that each mini-batch is only used to train predictors that use observed feature blocks, harming the generalization of predictors that use a larger subset of the features. This also leads to an interesting phenomenon: when $p_{\text{miss}} = 0.5$ for training data, `Combinatorial` performs better for higher probabilities of missing testing data. This is because missing test data dictates the use of predictors trained on smaller, more frequently observed feature subsets. `PlugVFL` often outperforms most of the other baselines, though not consistently. Specifically, it is at times outperformed by `Combinatorial` when the missing probability of the training data is low, and by `Ensemble` when it is high. `LASER-VFL` consistently outperforms the baselines across the different probabilities of missing blocks during training and inference. (In fact, in Table 3.1, `LASER-VFL` is never outperformed by more than a standard deviation.) In particular, even when the samples are fully-observed, `LASER-VFL` outperforms `Standard VFL` and `Combinatorial`. We believe this is due to the regularization effect of the task-sampling mechanism in our method, which effectively behaves as a form of dropout.

Scalability. Figure 3.2 examines the scalability of the different methods with respect to the number of clients, K. In the top plot, we see that `LASER-VFL` performs the best for all K. It is followed by `Combinatorial`, `Ensemble`, and `PlugVFL`. However, the performance of `Combinatorial` drops faster as K increases, since each exact combination of feature blocks is observed less frequently. In contrast, `Ensemble` and `PlugVFL`, like `LASER-VFL`, train each representation model on all batches for which its feature block is observed. In the bottom plot, we observe the expected scalability issues of `Combinatorial`, caused by its exponential complexity.

Table 3.2: Test accuracy for different probabilities of missing blocks during training and inference (including nonuniform) for CIFAR-10 with $K = 4$ clients, averaged over five seeds ($\pm$ standard deviation).

Training $p_{\text{miss}}(k)$	$\sim$ Beta(2.0,2.0)				0.5		
Inference $p_{\text{miss}}(k)$	0.0	0.1	0.5	$\sim$ Beta(2.0,2.0)	0.0	0.1	0.5
	CIFAR-10 (accuracy, %)						
`Local`	72.7 ± 1.9	72.8 ± 1.8	72.6 ± 2.2	73.4 ± 1.0	71.2 ± 0.4	71.1 ± 0.4	71.3 ± 0.7
`Standard VFL`	68.2 ± 11.6	54.0 ± 15.7	13.9 ± 5.4	13.5 ± 5.3	54.9 ± 10.0	38.6 ± 9.1	10.9 ± 2.2
`Ensemble`	83.9 ± 1.7	82.7 ± 2.0	75.3 ± 2.6	75.4 ± 1.7	82.4 ± 0.2	81.0 ± 0.8	73.8 ± 0.6
`Combinatorial`	67.8 ± 11.5	70.2 ± 10.1	73.4 ± 6.6	75.9 ± 2.9	54.9 ± 9.7	58.6 ± 7.7	68.4 ± 3.1
`PlugVFL`	80.9 ± 2.1	80.1 ± 1.7	77.8 ± 0.4	77.1 ± 0.1	76.5 ± 2.1	75.6 ± 1.5	72.4 ± 1.0
`LASER-VFL` (ours)	$\mathbf{88.8} \pm 1.4$	$\mathbf{88.2} \pm 1.6$	$\mathbf{81.1} \pm 2.3$	$\mathbf{81.7} \pm 1.0$	$\mathbf{87.4} \pm 0.3$	$\mathbf{86.4} \pm 0.6$	$\mathbf{79.4} \pm 1.6$

Nonuniform missing features. In Table 3.2, we present an experiment where the probability of each feature block k not being observed, $p_{\text{miss}}(k)$, is sampled independent and identically distributed following $p_{\text{miss}}(k) \sim \text{Beta}(2.0, 2.0)$. Note that, for $X \sim \text{Beta}(2.0,2.0)$, we have that $\mathbb{E}[X] = 0.5$. This motivates our comparison with $p_{\text{miss}}(k) = 0.5$ for all k. For the column corresponding to the case where both the probability of missing training data and inference data are nonuniform, these probabilities are sampled independently for training and inference.

As in the previous experiments, we see that `LASER-VFL` outperforms the baselines. Further, not only does the nonuniform nature of $p_{\text{miss}}(k)$ not harm performance, but in fact our `LASER-VFL` method seems to do slightly better when the feature blocks have missing training data with a probability $p_{\text{miss}}(k) \sim \text{Beta}(2.0, 2.0)$ rather than when we simply have $p_{\text{miss}}(k) = 0.5$, which corresponds to the expected value of the random variable $p_{\text{miss}}(k)$.

3.6 Discussion

In this chapter, we introduced `LASER-VFL`, a novel method for efficient training and inference in vertical federated learning that is robust to missing blocks of features without wasting data. This is achieved by carefully sharing model parameters and by employing a task-sampling mechanism that allows us to efficiently estimate a loss whose computation would otherwise lead to an exponential complexity. We provide convergence guarantees for `LASER-VFL` and present numerical experiments demonstrating its improved performance, not only in the presence of missing features but, remarkably, even when all samples are fully observed. For future work, it would be interesting to apply `LASER-VFL` to multi-task learning and investigate how it behaves when different clients tackle different tasks—a scenario that naturally fits our setup.

Chapter 4

Semi-decentralized VFL

The conventional client-server scheme considered in the previous chapters works well for cross-silo VFL with a small number of clients. However, as the number of clients increases, server bandwidth is strained and latency grows. To address this, in this chapter we explore an orthogonal strategy to the lossy compression approach in Chapter 2 for alleviating the bandwidth bottleneck in the client-server scheme: the use of direct client–client links.

Fully decentralized schemes rely solely on client-client links, eliminating the need for a server. The more common consensus methods distribute communications but can incur high overall costs, whereas token methods avoid the staleness inherent in consensus approaches at the expense of sacrificing their parallelism. Multi-token methods strike a balance by running several tokens in parallel. Regardless, decentralized approaches tend to converge slowly on poorly connected graphs.

To address this, semi-decentralized schemes combine client-client links, which alleviate the server-bandwidth bottleneck, with client-server links, which accelerate convergence in poorly connected networks and allow for convergence even in disconnected client networks. Yet, while VFL methods for both client-server and decentralized schemes have been proposed, existing work lacks a semi-decentralized VFL method.

We introduce multi-token coordinate descent (`MTCD`), a semi-decentralized VFL method that recovers client-server and decentralized schemes as special cases and spans the spectrum of semi-decentralized configurations in between, by adjusting hyperparameters. We prove the convergence of `MTCD` and, based on the relative cost of client-server to client-client communications, we show, analytically and empirically, that its semi-decentralized settings outperforms both ends of the spectrum in applications where neither extreme is ideal. To the best of our knowledge, `MTCD` is both the first semi-decentralized VFL method and the first semi-decentralized multi-token method overall.

This chapter is based on Valdeira et al. (2023).

4.1 Related work

Vertical FL. Some VFL works employ primal-dual optimization techniques. Namely, Smith et al. (2018) focuses on the client-server setting, He et al. (2018) generalizes it to the decentralized setting, and `DCPA` (Alghunaim et al., 2021) improves upon the convergence rate of He et al. (2018). In contrast, some coordinate descent-based methods work in the primal domain, allowing them to train a broader class of models. The work in Chen et al. (2020) employs asynchronous updates, Liu et al. (2022b) resorts to multiple local updates between rounds of communication, and Castiglia et al. (2022) introduces the use of lossy compression on the communicated representations. An interesting line of work related to VFL is hybrid FL (Zhang et al., 2020), which deals with datasets distributed by both samples and features. For a detailed survey of VFL methods, see Yang et al. (2023); Liu et al. (2024).

Coordinate descent. Coordinate descent methods (Wright, 2015), where (blocks of) coordinates are updated sequentially, rather than simultaneously, are natural candidates for optimization in feature-distributed learning. The block selection is most often cyclic (Beck and Tetruashvili, 2013) or independent and identically distributed at random (Nesterov, 2012; Richtárik and Takáč, 2012), yet Sun et al. (2019) considers block selection following a Markov chain. Several extensions to coordinate descent have been proposed, such as acceleration and parallelization (Fercoq and Richtárik, 2015) and adaptations to the distributed setting (Liu et al., 2022b; Chen et al., 2022).

Semi-decentralized FL. Recently, semi-decentralized approaches have been proposed to lower communication costs and deal with data heterogeneity (Lin et al., 2021; Guo et al., 2021; Wang and Chi, 2025), and to handle intermittent connections, latency, and stragglers (Bian and Xu, 2022; Yemini et al., 2023). Additionally, other semi-decentralized FL works deal with (multi-layered) hierarchical networks (Zhang et al., 2021; Hosseinalipour et al., 2022). Semi-decentralized FL is sometimes also referred to as hybrid FL; however, we opt for the term semi-decentralized FL to avoid confusion with the data partitioning setting mentioned above.

4.2 Problem setup

The fusion model g introduced in Problem (1.1), can be seen as the composition of two functions: a nonparameterized map g_0 that aggregates the representations sent by the clients, which we refer to as an *aggregation mechanism*, and a parameterized part, g_1, which takes these aggregated representations and performs the prediction for the task of interest. That is, we have that:

$$g = g_1 \circ g_0.$$

While other aggregation mechanisms can be used (Ceballos et al., 2020), we focus on aggregation by sum and by concatenation, as they are the most common and most general aggregators, respectively.

Throughout this chapter, we that assume $\mathcal{L}$ is L-smooth (D2) and has a finite infimum (D1).

As in Chapter 2, in this chapter, we let $\boldsymbol{\theta}_k$, for $k \in \mathcal{K}$, and $\boldsymbol{\theta}_0$ serve as shorthand notation for $\boldsymbol{\theta}_{\boldsymbol{f}_k}$ and $\boldsymbol{\theta}_{\boldsymbol{g}}$, respectively. Further, while $\boldsymbol{\theta}_k$ is associated with and updated at client k, for $k \in \mathcal{K}$, the fusion model parameters $\boldsymbol{\theta}_0$ may be updated at different entities (clients or server), depending on the communication scheme.

4.3 Proposed method: decentralized setting

We start by introducing a simple, special case of our algorithm, which we refer to as single-token coordinate descent (`STCD`). This method is also a subroutine of our general algorithm. Mathematically, `STCD` is closely related to Sun et al. (2019) and the application mentioned therein taken from Mao et al. (2020). However, `STCD` works in the primal domain and on a feature-distributed setting.

In this section, we do not require the existence of a server. We solve Problem (1.1) in a fully decentralized manner, communicating through channels described by a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} \coloneqq \mathcal{K}$ is the vertex set and $\mathcal{E}$ is the edge set. We denote the set of neighbors of client k by $\mathcal{N}_k \coloneqq \{i \colon \{i, k\} \in \mathcal{E}\}$ and define $\bar{\mathcal{N}}_k \coloneqq \mathcal{N}_k \cup \{k\}$. In this section only, $\boldsymbol{\theta}_0$ is associated with some client k, which is responsible for updating it, as well as the parameters of its local model $\boldsymbol{\theta}_k$.[1]

Since all clients know g_1 and ℓ_n, if a client knows $\mathcal{T} \coloneqq \{g_0\big(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}\big), \boldsymbol{\theta}_0\}$, it can

[1]We do this for alignment with the analysis in Sun et al. (2019), where all blocks are selected following a Markov Chain. However, in practice, we may want to update $\boldsymbol{\theta}_0$ at each client instead, in which case the analysis would need to be adjusted.

compute $\mathcal{L}$. We call $\mathcal{T}$ our *token*. The size of the token depends on the model being considered. For example, if we have an E-dimensional representation per sample, and thus $\boldsymbol{F}_k$ is of size NE, for all k, then aggregation by concatenation leads to a token $\mathcal{T}$ of size $KNE + \dim(\boldsymbol{\theta}_0)$, where $\dim(\boldsymbol{\theta}_0)$ denotes the dimension of $\boldsymbol{\theta}_0$. Yet, for aggregation mechanisms of the form $g_0 \colon \prod_k \mathbb{R}^a \mapsto \mathbb{R}^a$, such as the sum, the token size is independent of the number of clients. In this case, the token size only exceeds that of a single representation in that it includes the parameters of the fusion model, $\boldsymbol{\theta}_0$, which are typically small. (The fusion model is often a linear model, or even nonparameterized (Liu et al., 2024).) In particular, for generalized linear models, $\mathcal{T} = \{\boldsymbol{X}\boldsymbol{\theta}\}$ is of size N. (In Section 4.4, we allow for the use of mini-batches, further dropping the dependency on the number of samples.)

A client holding $\mathcal{T}$ can compute $\mathcal{L}$. Yet, more importantly, if client k holds its local data $\boldsymbol{X}_k$ and local parameters $\boldsymbol{\theta}_k$, then holding $\mathcal{T}$ enables it to compute the partial gradient with respect to $\boldsymbol{\theta}_k$, given by:

$$\nabla_k \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial g_1(g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\}), \boldsymbol{\theta}_0)}{\partial g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\})} \cdot \frac{\partial g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k)\})}{\partial \boldsymbol{F}_k(\boldsymbol{\theta}_k)} \cdot \frac{d\boldsymbol{F}_k(\boldsymbol{\theta}_k)}{d\boldsymbol{\theta}_k},$$

where $\mathcal{T}$ is used to compute the first two terms. Computing $\nabla_k \mathcal{L}(\boldsymbol{\theta})$ allows client k to update its local model $\boldsymbol{\theta}_k$.

We now describe `STCD`, summarized in Algorithm 5. We index $\boldsymbol{\theta}$ and $\mathcal{T}$ with two counters: **1)** step s of the token roaming, at which client k^s is visited, and **2)** local update q at client k^s. To simplify the algorithm description, we omit $\boldsymbol{\theta}_0$ for the rest of Section 4.3, as if it were part of the local model of a given client k where it is updated. Yet, unlike $\boldsymbol{\theta}_0$ (which is part of the token), $\boldsymbol{\theta}_k$ does not leave k.

Initialization Token $\mathcal{T}^{s,q}$ must always be in accordance with iterate $\boldsymbol{\theta}^{s,q}$. Namely, as the roaming starts at client k^0, this client must know $\mathcal{T}^{0,0} = \{g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^{0,0})\}), \boldsymbol{\theta}_0^{0,0}\}$. For some models, this can be achieved by initializing $\boldsymbol{\theta}_k^{0,0}$ such that $\boldsymbol{F}_k(\boldsymbol{\theta}_k^{0,0})$ is independent of the local data $\boldsymbol{X}_k$. When this is not possible, the clients can send their initial representations to k^0 as a prelude.

Updating the token and the local models As explained above, client k^s can compute the partial gradient with respect to its local model locally, allowing it to perform a coordinate descent step. Further, to lower communication costs, we do Q local steps at each client. That is:

$$\forall q \in \{0, \dots, Q-1\}: \quad \boldsymbol{\theta}_{k^s}^{s,q+1} = \boldsymbol{\theta}_{k^s}^{s,q} - \eta \nabla_{k^s} f(\boldsymbol{\theta}^{s,q}) \tag{4.1}$$

Algorithm 5: `STCD`

Input: initial $\boldsymbol{\theta}^{0,0}$ and k^0, stepsize η, number of hops S, number of local updates Q.
$\mathcal{T}^{0,0} \leftarrow \{g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^{0,0}),\}), \boldsymbol{\theta}_0^{0,0}\}$.
$\boldsymbol{\theta}^{S,Q}, \mathcal{T}^{S,Q}, k^S \leftarrow$ `TokenRoaming`$(\boldsymbol{\theta}^{0,0}, \mathcal{T}^{0,0}, k^0, \boldsymbol{X}, S, Q)$.
Function `TokenRoaming`$(\boldsymbol{\theta}, \mathcal{T}, k, \boldsymbol{D}, S, Q)$**:**
  $\boldsymbol{\theta}^{0,0}, \mathcal{T}^{0,0}, k^0 \leftarrow \boldsymbol{\theta}, \mathcal{T}, k$.
  for $s = 0, \ldots, S-1$ **do**
    for $q = 0, \ldots, Q-1$ **in client** k^s **do**
      Compute $\boldsymbol{\theta}_{k^s}^{s,q+1}$ via (4.1).
      Compute $\mathcal{T}^{s,q+1}$.
    Client k^s sends $\mathcal{T}^{s,Q}$ to $k^{s+1} \sim \mathcal{U}(\bar{\mathcal{N}}_{k^s})$.
    $\boldsymbol{\theta}^{s+1,0}, \mathcal{T}^{s+1,0} \leftarrow \boldsymbol{\theta}^{s,Q}, \mathcal{T}^{s,Q}$.
  return $\boldsymbol{\theta}^{S,Q}, \mathcal{T}^{S,Q}, k^S$.

and $\boldsymbol{\theta}_k^{s,q+1} = \boldsymbol{\theta}_k^{s,q}$ for $k \neq k^s$. We must now update the token accordingly. To compute $\mathcal{T}^{s,q+1}$, we need $\mathcal{T}^{s,q}$, $\boldsymbol{F}_{k^s}(\boldsymbol{\theta}_{k^s}^{s,q+1})$, and $\boldsymbol{h}_{k^s}(\boldsymbol{\theta}_{k^s}^{s,q})$, all of which are held by k^s. For example, for aggregation by sum, we have:

$$g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^{s,q+1})\}) = g_0(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^{s,q})\}) + \boldsymbol{F}_{k^s}(\boldsymbol{\theta}_{k^s}^{s,q+1}) - \boldsymbol{F}_{k^s}(\boldsymbol{\theta}_{k^s}^{s,q}).$$

This allows us to perform multiple local steps. After these steps, the updated token is communicated to client k^{s+1}. Thus, by induction, we keep the token up-to-date throughout our algorithm.

Communicating the token Client k^s sends token $\mathcal{T}^{s,Q}$ to client $k^{s+1} \sim \mathcal{U}(\bar{\mathcal{N}}_{k^s})$, where $\mathcal{U}$ denotes the uniform distribution. Thus, the roaming token performs a random walk over the communication graph, resulting in a Markov chain.

To summarize, `STCD` is a decentralized VFL method that allows us to employ an extension of Markov chain block coordinate descent (Sun et al., 2019) with multiple local updates in feature-distributed settings.

We now discuss the convergence of `STCD`. We define an auxiliary scalar iteration counter to summarize the tuple (S, Q) used in the description of `STCD`, $r := sQ + q$. This allows us to map `STCD` to the optimization framework of Sun et al. (2019), thereby enabling us to resort to their convergence results to establish the convergence of `STCD`.

Convergence guarantees. If $\mathcal{L}$ is an L-smooth (D2) function with a finite infimum (D1) and $\{\boldsymbol{\theta}^i\}_{i=1}^r$ is a sequence generated by Algorithm 5, Sun et al. (2019) gives convergence guarantees for $Q = 1$. In particular, under mild assumptions on the Markov chain—for example, being time-homogeneous, irreducible, and aperiodic—we have that $\lim_{r\to\infty} \mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^r)\| = 0$ and

$$\mathbb{E}\left[\min_{i\in[r]}\|\nabla\mathcal{L}(\boldsymbol{\theta}^i)\|^2\right] \leq \frac{(\Omega_1(\tau-1)^2+\Omega_2)\Delta}{r}, \tag{4.2}$$

where $\Delta \coloneqq \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$, Ω_1 and Ω_2 are constants that depend on the minimum value of the stationary distribution of the Markov chain, $\pi_{\min}$, and τ denotes the $\frac{\pi_{\min}}{2}$-mixing time of the the Markov chain. Although Sun et al. (2019) only considers $Q = 1$, we can leverage its analysis to prove convergence for $Q > 1$.

Recalling that $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is the original graph, we consider an auxiliary dynamic graph $\mathcal{G}^r = (\mathcal{V}, \mathcal{E}^r)$, where

$$\mathcal{E}^r = \begin{cases} \mathcal{E} & \text{if } r \bmod Q = 0, \\ \{\{i,i\} : i \in \mathcal{V}\} & \text{otherwise.} \end{cases}$$

Then, running `STCD` on $\mathcal{G}^r$ with a single local update is equivalent to running `STCD` on the original graph $\mathcal{G}$ with Q local updates. Further, this dynamic graph preserves the properties required for the analysis to hold.

To see this, let $\boldsymbol{P}$ denote the transition matrix of a random walk over the original graph $\mathcal{G}$ and let $\boldsymbol{P}(r)$ denote the transition matrix of a random walk over the auxiliary graph $\mathcal{G}^r$. Note that, for $r \mod Q \neq 0$, $\boldsymbol{P}(r) = \boldsymbol{I}$, where $\boldsymbol{I}$ denotes the identity matrix, and, for $r \mod Q = 0$, $\boldsymbol{P}(r) = \boldsymbol{P}$. At a given iteration R, assumed, for simplicity, to be $R \geq Q$, we have that $\boldsymbol{P}(r)\boldsymbol{P}(r+1)\ldots\boldsymbol{P}(r+R) = \boldsymbol{P}^{\lfloor R/Q \rfloor}$. We thus recover the results in Sun et al. (2019) up to a factor of Q in the mixing time. (That is, in (4.2), we replace τ by $Q\tau$.)

Limitations. The decentralized, single-token method described in this section is appealingly simple and outperforms the baselines across various setups (see Section 4.5). However, its convergence slows down as network connectivity decreases. To address this, we introduce `MTCD`, a multi-token extension to `STCD` that mitigates this problem.

4.4 Proposed method: semi-decentralized setting

In Section 4.3, we introduced a (fully) decentralized special case of `MTCD` where a single token roams over a set of clients. We now present our method in the semi-decentralized setting, which subsumes the setting in Section 4.3 as a special case. `MTCD` alternates between a *roaming* step and a *syncing* step.

- *Roaming.* We start with multiple, matching tokens at a subset of the clients. As each token performs a different random walk realization, they undergo different sequences of updates, becoming distinct.
- *Syncing.* To leverage these parallel computations while keeping our model estimates coupled, we periodically sync the roaming tokens at the server, combining the progress of multiple sequences.

By adjusting the number of tokens and syncing frequency, we achieve a flexible degree of parallelization and server dependency. This enables a smooth navigation along the spectrum between client-server and decentralized setups, allowing for communication schemes suitable for different applications.

We further extend `STCD` to allow for mini-batch estimates. More precisely, recall that we denote the mini-batch gradient estimate as $\nabla\mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta})$. We make the following standard assumptions on our mini-batch gradient estimate.

- We assume that the mini-batch gradient estimate $\nabla\mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta})$ is an unbiased estimate of gradient $\nabla\mathcal{L}(\boldsymbol{\theta})$ (D3).
- We assume that the mini-batch gradient estimate $\nabla\mathcal{L}_{\mathcal{B}}(\boldsymbol{\theta})$ has its variance upper bounded (D4) by σ^2/B.

In addition to the communication graph $\mathcal{G}$, we now also consider the existence of a central server with links to all clients, as illustrated in Figure 1.5c. The existence of a server brings a change to the model partitioning: $\boldsymbol{\theta}_0$ is now updated at the server. Further, we now have Γ tokens roaming simultaneously. Each token $\mathcal{T}_\gamma$, for $\gamma \in [\Gamma]$, has a model estimate $\boldsymbol{\theta}(\gamma)$ associated with it. Thus, during the roaming step, each client k must now keep up to Γ local model estimates $\boldsymbol{\theta}_k(\gamma)$ in memory.[2] (This requirement holds in the general case, but not in the important case discussed below, where the tokens roam over

[2]In practice, it suffices to to add a copy of $\boldsymbol{\theta}^t$ as a new token visits during a given roaming step (with a maximum of Γ model estimates at a client), resetting the number of copies when syncing.

Algorithm 6: `MTCD`

Input: initial $\boldsymbol{\theta}^0$, stepsize η, number of hops S, number of local updates Q, model estimates combination weights $\{w_{k\gamma}\}$, distributions $\{P_\gamma\}$.
for $t = 0, \ldots, T-1$ **do**
  for $k \in \mathcal{K}$ **in parallel do**
    Sample shared mini-batch indices $\mathcal{B}^t$ via a seed shared.
    Send $\boldsymbol{F}_{k,\mathcal{B}^t}(\boldsymbol{\theta}_k^t)$ to the server.
  Server computes $\mathcal{T}^t \leftarrow \{g_0(\{\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)\}), \boldsymbol{\theta}_0^t\}$.
  for $\gamma \in \{0, 1, \ldots, \Gamma\}$ **in parallel do**
    Server sends $\mathcal{T}^t$ to $k_\gamma^{t,0} \sim P_\gamma$.
    $\boldsymbol{\theta}^{t,S,Q}(\gamma), \mathcal{T}_\gamma^{t,S,Q}, k_\gamma^{t,S} \leftarrow$ `TokenRoaming`$(\boldsymbol{\theta}^t, \mathcal{T}^t, k_\gamma^{t,0}, \boldsymbol{B}^t, S, Q)$.
  for $k \in \{0, 1, \ldots, K\}$ **in parallel do**
    $\boldsymbol{\theta}_k^{t+1} = \sum_{\gamma=1}^{\Gamma} w_{k\gamma} \boldsymbol{\theta}_k^{t,S,Q}(\gamma)$.

disjoint subsets of clients.) To simplify the description of the algorithm, we consider a token $\gamma = 0$ which stays at the server ($k = 0$) throughout the roaming step.

We now describe `MTCD`, summarized in Algorithm 6, where $k_\gamma^{t,s}$ denotes the client holding token γ after t synchronizations and s hops and P_γ denotes the distribution over clients indicating where the roaming begins. (Note that, unlike in `STCD` where S was the iteration counter, in `MTCD` it is a fixed hyperparameter.) We index $\boldsymbol{\theta}$ and $\mathcal{T}$ with three counters: synchronization step t, and the two counters used in `STCD`. For simplicity, we denote

$$\mathcal{T}^t := \mathcal{T}_1^{t,0,0} = \cdots = \mathcal{T}_\Gamma^{t,0,0}$$

and

$$\boldsymbol{\theta}^t := \boldsymbol{\theta}^{t,0,0}(1) = \cdots = \boldsymbol{\theta}^{t,0,0}(\Gamma).$$

Lastly, we let $\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)$ denote the mini-batch representations of client k at time t.

Initialization. All model-token pairs $(\boldsymbol{\theta}(\gamma), \mathcal{T}_\gamma)$ are initialized to the same values. As in `STCD`, each token $\mathcal{T}_\gamma$ must be in accordance with $\boldsymbol{\theta}(\gamma)$.

Roaming. When $\mathcal{T}_\gamma$ is at client $k_\gamma^{t,s}$, it is used to update the block $k_\gamma^{t,s}$ of parameters $\boldsymbol{\theta}(\gamma)$. Each $\mathcal{T}_\gamma$ roams for S hops, as in `STCD`. In parallel, $\boldsymbol{\theta}_0(0)$ is updated at the server.

Syncing. After S hops, each client combines its model estimates, obtaining $\boldsymbol{\theta}_k^{t+1} = \sum_{\gamma=0}^{\Gamma} w_{k\gamma} \boldsymbol{\theta}_k^{t,S,Q}(\gamma)$. (We cover the choice of $\boldsymbol{w}_k := (w_{k0}, \ldots, w_{k\Gamma})$, which lies in the Γ-

dimensional probability simplex, later.) At the start of the next iteration, each client k samples a set of indices $\mathcal{B}^t$ locally, which is the same for all clients (as they share a seed), and sends its local representation $\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)$ to the server. This allows the server to compute $\mathcal{T}^t$ and send it to each client $k_\gamma^{t,0} \sim P_\gamma$, for $\gamma \in \{1, \ldots, \Gamma\}$, to start the next roaming step, with P_γ such that $\mathbb{P}(k_\gamma^{t,0} = k)$. (The point mass distribution P_0 has support over the server ($k = 0$), a node without neighbors.)

`MTCD` can recover client-server and decentralized setups:

- If client-server links are not used ($S \to \infty$), `MTCD` recovers the decentralized setup. Namely, this corresponds to running Γ instances of `STCD` in parallel.
- If the edge set $\mathcal{E}$ is empty, we recover the client-server setup. If $\Gamma = K$ and each P_γ has support over a different client, all clients are updated at each step t, whereas if $\Gamma < K$, only a subset of clients is updated at each step t.

On client-server communications. `MTCD` employs client-server communications twice per iteration t: first during the uplink phase (line 5), when each client k sends its representation $\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)$ to the server, and later during the downlink phase (line 8), when the server transmits Γ tokens to a subset of clients to initiate the next roaming step.

This scheme entails only Γ downlink communications, yet it requires K uplink communications. This contrasts with horizontal FL, where often only a subset of clients participates in both directions. However, in VFL, it is standard for all clients to participate in every round, since computing the loss for even a single sample requires input from every client. An exception to this is the special full-batch case, where only Γ uplink communications are needed, as the server can reuse representations from previous iterations to update the token. In this scenario, only clients updating their local model in a given iteration must send new representations to the server.

However, in the general mini-batch case, even if only a subset of clients updates their local models, all clients must transmit their mini-batch representations to the server in the subsequent iteration. This is because each mini-batch representation is associated with both a specific mini-batch and local parameters, and these parameters have generally been updated since the last time that mini-batch was sampled.

On the impact of S and Γ. In general, as we increase the amount of parallel computations (by increasing Γ) and client-server communications (by lowering S), the communication efficiency and the number of iterations needed to converge both decrease. Given this trade-off, we see that our choice of S and Γ depends on the application.

Having introduced the general MTCD algorithm, we now go over two instances of it, both for semi-decentralized setups, and provide convergence guarantees for both. Since our convergence results must accommodate $\Gamma > 1$, where different tokens sync only every S hops, the STCD analysis from the previous section does not apply, as it requires $S \to \infty$.[3] Thus, our convergence results for MTCD differ fundamentally from those for $S \to \infty$ and $\Gamma = 1$, as they require a new approach to the proof, addressing the challenges of **1)** stale information from other tokens and **2)** the coexistence and periodic combination of different model estimates.

We defer the proofs to the appendices.

4.4.1 Setting with a token per cluster

Setup. Consider C disjoint clusters of clients $\{\mathcal{C}_c, \ldots, \mathcal{C}_C\}$ and $\Gamma = C$ tokens. Each token γ roams $\mathcal{C}_\gamma$, allowing us to use γ and c interchangeably. Note that distribution P_γ now has support over $\mathcal{C}_\gamma$, and only over $\mathcal{C}_\gamma$. In this setting, the subsets of the blocks $\{\boldsymbol{\theta}_k\}$ updated by different tokens are disjoint. This allows us to combine the model estimates by simply concatenating the updated blocks. That is, we let $\boldsymbol{w}_k \in \mathbb{R}^\Gamma$ be the one-hot encoding for the token visiting client k. We also define a cluster containing only the server, $\mathcal{C}_0 \coloneqq \{0\}$.

Clusters $\{\mathcal{C}_c\}$ may reflect the natural topology of the communication graph, arising from physical limitations or privacy constraints. For example, a cluster may represent devices within a single household or company. However, we can also partition the graph artificially before training to enable the use of multiple tokens in this token-per-cluster setting, which prevents overlapping trajectories.

The selection of a graph clustering method is beyond the scope of this work, but it is worth noting that there exists an extensive literature addressing this topic. This field, sometimes referred to as community detection, focuses on grouping nodes so that connectivity within the clusters is greater than across clusters, and it includes distributed methods that do not require a centralized orchestrator (Hui et al., 2007).

Convergence results. We denote the number of stale updates between syncing steps by $P \coloneqq SQ$ and the client holding token c at update $p \in [P]$ of iteration t by $i_c^{t,p}$, where $i_c^{t,p} = k_c^{t,s}$ for all $p \in \{sQ, \ldots, s(Q+1)-1\}$. We define $\boldsymbol{p}^t \coloneqq (\boldsymbol{p}_1^t, \ldots, \boldsymbol{p}_C^t)$ where

[3] Note how, for $S \to \infty$, the sequence of iterates in MTCD, $\{\boldsymbol{\theta}^t\}$, would consist of a single element, preventing convergence.

$\boldsymbol{p}_c^t = (i_c^{t,0}, \ldots, i_c^{t,P-1})$, and resort to the set:

$$\mathcal{F}^t \coloneqq \{\mathcal{B}^0, \boldsymbol{p}^0, \ldots, \mathcal{B}^{t-1}, \boldsymbol{p}^{t-1}\}.$$

We denote the partition across clusters as $\boldsymbol{\theta} = (\boldsymbol{\theta}_{1,:}, \ldots, \boldsymbol{\theta}_{C,:})$, and each partition $\boldsymbol{\theta}_{c,:}$ is further partitioned into $K_c \coloneqq |\mathcal{C}_c|$ blocks, $\boldsymbol{\theta}_{c,:} = (\boldsymbol{\theta}_{c,1}, \ldots, \boldsymbol{\theta}_{c,K_c})$. Further, in this section, we define $\boldsymbol{\theta}^{t,p}(c)$ to refer to the iterate at cluster c, at stale update p of iteration t. We use the following shorthand notation:

$$\mathbb{E}_t[\cdot] \coloneqq \mathbb{E}\left[\cdot \mid \mathcal{F}^t\right], \quad \mathbb{E}_{t^+}[\cdot] \coloneqq \mathbb{E}\left[\cdot \mid \mathcal{F}^t, \boldsymbol{p}^t\right].$$

Let $\boldsymbol{I}$ denote the identity matrix and define $\boldsymbol{\Pi}_{c,i}$ as the diagonal matrix whose diagonal entries are $\boldsymbol{I}$ for indices corresponding to the partition $\boldsymbol{\theta}_{c,i}$ and 0 elsewhere. It will also be useful to define $\boldsymbol{\Pi}_c \coloneqq \sum_{i=1}^{K_c} \boldsymbol{\Pi}_{c,i}$ and $\|\boldsymbol{u}\|_{\boldsymbol{A}}^2 \coloneqq \boldsymbol{u}^\top \boldsymbol{A}\boldsymbol{u}$.

We now present Lemma 4 and Lemma 5, which we use to prove Theorem 5 and Theorem 6.

Lemma 4 (Inner product upper bound)**.** *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 6. If $\mathcal{L}$ is L-smooth (D2) and our mini-batch gradient estimate is unbiased (D3), we have that:*

$$\mathbb{E}_{t^+}[\nabla_{c,:}\mathcal{L}(\boldsymbol{\theta}^t)^\top(\boldsymbol{\theta}_{c,:}^{t+1} - \boldsymbol{\theta}_{c,:}^t)] \leq -\eta\left(1 - \frac{\eta^2 L^2 (P-1)^2}{2}\right)\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} - \frac{\eta}{2}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\boldsymbol{\Pi}_{c,i_c^{t,p}}} \quad - \frac{\eta}{2}(1 - \eta^2 L^2 (P-1)^2)\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}}. \tag{4.3}$$

Lemma 5 (Squared norm upper bound)**.** *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 6. If our mini-batch gradient estimate is unbiased (D3) and has a bounded variance (D4), we have that:*

$$\mathbb{E}_{t^+}\left[||\boldsymbol{\theta}_{c,:}^{t+1} - \boldsymbol{\theta}_{c,:}^t||^2\right] \leq \frac{\eta^2\sigma^2 P^2}{B} + \eta^2 P\Big(\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} + \sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}}\Big). \tag{4.4}$$

We now make a general assumption on the minimum probability of each client being visited by a token.

Assumption 1 (Minimum probability of visit)**.** *There exists a positive constant π such*

that, for all $c \in [C]$ *and* $j \in [K_c]$*:*

$$\forall t \geq 0: \quad \mathbb{P}\left(\bigcup_{p=0}^{P-1} \{i_c^{t,p} = j\}\right) \geq \pi > 0. \tag{A1}$$

This can be achieved through a variety of communication schemes. For example, if **1)** all the clients have a nonzero probability of receiving the token from the server, then this holds for any $P \geq 1$, and if **2)** at least one node in the cluster has a nonzero probability of receiving the token from the server, then this holds if S is greater than or equal to the diameter of the cluster (that is, the greatest distance between any pair of nodes in a graph). In Appendix C.4, we go over this topic in more detail and provide multiple ways to lower bound π for different communication schemes.

Theorem 5 (Main result—token-per-cluster setting)**.** *Let* $\{\boldsymbol{\theta}^t\}$ *be a sequence generated by Algorithm 6 in the setting with a token per cluster. If* $\mathcal{L}$ *is an* L*-smooth function with a finite infimum* (D2)*, our mini-batch gradient estimate is unbiased* (D3) *and has a bounded variance* (D4)*, and each node has a minimum probability* $\pi > 0$ *of being visited by a token at each iteration* (A1)*, we have that, for a stepsize* $\eta \in \left(0, \frac{1}{2LSQ}\right]$*:*

$$\frac{1}{T}\sum_{t=0}^{T-1} \mathbb{E}\|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2\Delta}{\eta\pi T} + \frac{\eta S^2 Q^2 \sigma^2 L}{\pi B}, \tag{4.5}$$

where $\Delta \coloneqq \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$ *and the expectation is with respect to token trajectories* $\{\boldsymbol{p}^t\}$ *and mini-batches* $\{\mathcal{B}^t\}$*. For the special, full-batch case, we also have that* $\nabla \mathcal{L}\left(\boldsymbol{\theta}^t\right) \to 0$ *almost surely.*

Proof sketch. To prove Theorem 5, we first employ the quadratic upper bound that follows from L-smoothness at iterates t and $t+1$. Next, we decouple this bound into cluster specific terms, leveraging the fact that the blocks updated at different tokens are disjoint. We then take the conditional expectation $\mathbb{E}_{t^+}$ and use Lemma 4 and Lemma 5. Next, setting $\eta \in \left(0, \frac{1}{2LP}\right]$ allows us to drop one of the terms in the upper bound. We then take the conditional expectation $\mathbb{E}_t$ and use our assumption on the existence of a minimum probability of token visit $\pi > 0$ to arrive at a descent lemma (in expectation). This allows us to use the classic result due to Robbins and Siegmund (1971) to get that $\nabla \mathcal{L}\left(\boldsymbol{\theta}^t\right) \to 0$ almost surely in the full-batch case. Finally, we take the (unconditional) expectation with respect to all the token trajectories and mini-batches and rearrange the terms. Averaging over $t \in \{0, \ldots, T-1\}$, and using our finite infimum assumption, we arrive at the result

in Theorem 5.

On the convergence results. When deriving convergence guarantees for VFL methods, it is quite challenging to deal with the presence of stale information in the updates. Even in client-server VFL, methods employing multiple local updates (Liu et al., 2022b; Castiglia et al., 2022) converge at a $\mathcal{O}(\Delta/\eta T)$ rate, for a stepsize $\eta \in \mathcal{O}(1/P)$. (Recall that P is the number of stale updates and Δ is the suboptimality of the initial iterate.) That is, although we can empirically observe a speedup when we increase the number of stale updates, this is not captured in the theoretical results. In fact, increasing P shrinks the upper bound on the stepsize and thus leads to a slower convergence rate.

Similarly, our convergence results achieve a $\mathcal{O}(\Delta/\eta\pi T)$ convergence rate for $\eta \in \mathcal{O}(1/P)$, where π is the minimum probability of any client being visited by a token during each roaming step. For the client-server setting, under full participation—the setting considered in Liu et al. (2022b) and Castiglia et al. (2022)—we have $\pi = 1$. Thus, our rate recovers the rate of client-server VFL methods.

Reduced time and communication complexity. Nevertheless, under some mild assumptions, we can show a reduction in the time and communication complexity of `MTCD` when compared to client-server VFL methods. To illustrate this, let us go over an example for which we can show that **1)** the rate of `MTCD` matches that of client-server VFL methods and that the **2)** time and **3)** communication cost per iteration of `MTCD` are lower than those of client-server VFL methods for a range of applications. Thus, for such applications, we provably achieve better time and communication complexity.

In particular, consider the case where K clients are divided into C clusters, each with K/C clients (assumed divisible) arranged in a path graph. We examine both `MTCD` and a client-server VFL method (such as Liu et al. (2022b) and Castiglia et al. (2022)), each using the same number of stale updates P, and thus with a matching upper bound on the stepsize, $\eta \in \mathcal{O}(1/P)$. `MTCD` employs $Q = 1$ and $S = K/C$ across each path graph, in the token-per-cluster setting, visiting each client in each cluster exactly once per iteration. The client-server method employs $Q = K/C$. It follows from the discussion above that both approaches converge at the same rate.

- **Time complexity:** We model the available bandwidth of each client-server link as $\propto 1/l$, where l is the number of links—a reasonable modeling assumption when a fixed bandwidth budget at the server is shared among the clients communicating with it. Consequently, we model the time needed to send a representation and a

token across a client-server link as $\tau_t^h \cdot l$ and $\tau_t^Z \cdot l$, respectively. We further denote the time to send a token across a client-client link by τ_s^Z and the time to perform a local update by τ_q.[4] Thus:

- For client-server VFL:

$$\text{time per iteration} = \tau_t^h \cdot K + \tau_t^Z \cdot K + Q \cdot \tau_q.$$

- For `MTCD`:

$$\text{time per iteration} = \tau_t^h \cdot K + \tau_t^Z \cdot \Gamma + S(\tau_s^Z + Q \cdot \tau_q).$$

Thus, for Q and S as mentioned above, we want to know whether

$$\tau_t^h K + \tau_t^Z K + \frac{K}{C} \cdot \tau_q \geq \tau_t^h K + \tau_t^Z \Gamma + \frac{K}{C}(\tau_s^Z + \tau_q).$$

Since $\Gamma = C$, this is equivalent to

$$\tau_t^Z / \tau_s^Z \geq \frac{K}{\Gamma(K - \Gamma)}.$$

Therefore, since typically $K \gg \Gamma$ and $\tau_t^Z > \tau_s^Z$—given that client-server links often have a greater latency than client-client links (Lin et al., 2021)—for a variety of applications, `MTCD` improves over the time complexity of client-server VFL.

- **Communication complexity:** As noted earlier, to succinctly capture the limitations of the client-server setup—namely, its vulnerability to server bandwidth bottlenecks and its single point of failure—we can quantify the relative impact of using client-server versus client-client links by comparing their communication costs, which vary by application.

  We denote the cost of transmitting a token over a client-client link and a client-server link by C_{CC}^Z and C_{CS}^Z, respectively. Similarly, we denote the cost of transmitting a representation over a client-client link and a client-server link by C_{CC}^h and C_{CS}^h, respectively. For the reasons above, we often have $C_{CS}^h > C_{CC}^h$ and $C_{CS}^Z > C_{CC}^Z$.

  In both `MTCD` and client-server VFL methods, all clients send $\boldsymbol{F}_k^{\mathcal{B}^t}(\boldsymbol{\theta}_k^t)$ to the server at the beginning of each iteration. However, while client-server VFL methods require

[4] Client bandwidth is typically less significant than server bandwidth, as the number of neighbors of a client usually does not grow with K.

sending the token to each client after synchronization, `MTCD` only requires the server to send Γ tokens to the clients, which then roam a cluster for S hops. Thus:

- For client-server VFL:

$$\text{comm. cost per iteration} = KC_{CS}^{h} + KC_{CS}^{Z}.$$

- For `MTCD`:

$$\text{comm. cost per iteration} = KC_{CS}^{h} + \Gamma C_{CS}^{Z} + S\Gamma C_{CC}^{Z}.$$

Thus, the question is whether the following inequality holds:

$$KC_{CS}^{h} + KC_{CS}^{Z} \geq KC_{CS}^{h} + \Gamma C_{CS}^{Z} + S\Gamma C_{CC}^{Z}.$$

Or, equivalently, whether

$$C_{CS}^{Z}/C_{CC}^{Z} \geq \frac{S\Gamma}{(K-\Gamma)}.$$

Since typically $K \gg \Gamma$ and $C_{CS}^{Z} > C_{CC}^{Z}$, we thus have that, for a range of applications, `MTCD` improves over the communication complexity of client-server VFL.

4.4.2 An extension with overlapping token trajectories

We now extend our convergence guarantees to a setting where we allow for overlapping token trajectories. We propose choosing the convex combination weights to be $\boldsymbol{w}_k = (\frac{1}{\Gamma}, \ldots, \frac{1}{\Gamma}) \in \mathbb{R}^{\Gamma}$ for all k, thus combining the model estimates by averaging them. To handle this periodic combination of the model estimates in this setting, we now further assume convexity. (We elaborate on this below.)

Note that the cluster-dependent notation—namely $i_{\gamma}^{t,p}$ and $\boldsymbol{p}_{\gamma}^{t}$ in the definition of $\boldsymbol{p}^{t}$—as well as the lemmas from the token-per-cluster setting, apply here as well. They differ only in that the "cluster" now represents the entire communication graph.

In this subsection, we assume that $\mathcal{L}$ is a convex function, as we now define.

Definition 8 (Convexity)**.** *A function f is convex if, for $a \in [0,1]$, we have that:*

$$\forall \boldsymbol{u}, \boldsymbol{v}\colon \quad f(a\boldsymbol{u} + (1-a)\boldsymbol{v}) \leq af(\boldsymbol{u}) + (1-a)f(\boldsymbol{v}). \tag{A8}$$

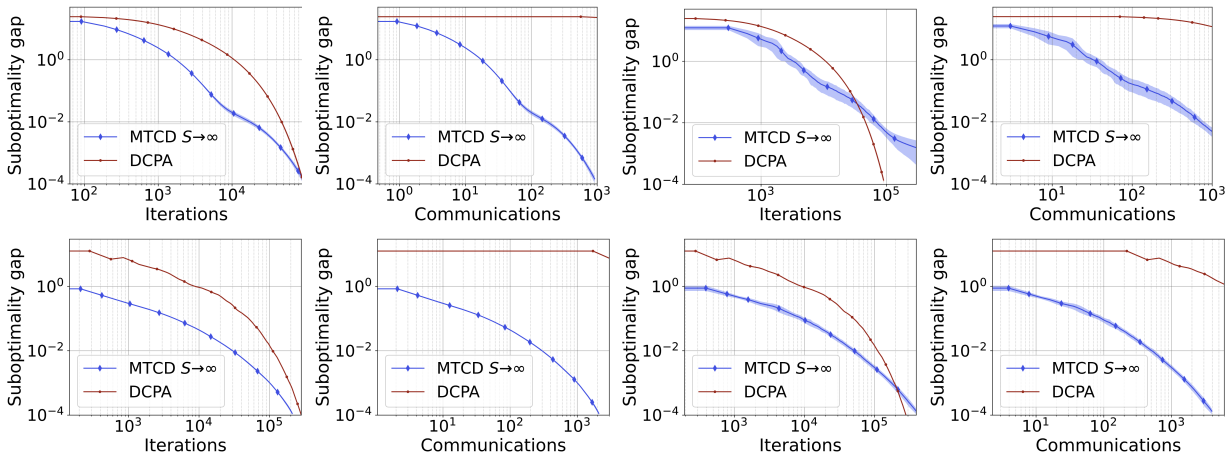


Figure 4.1: The four plots on the left correspond to an Erdős-Rényi graph with $p = 0.4$ and the four plots on the right to a path graph, both with $K = 40$. The top row concerns ridge regression and the bottom row concerns sparse logistic regression. The $S \to \infty$ `MTCD` run has $\Gamma = 1$ (i.e., it corresponds to `STCD`). Note that the flat `DCPA` trajectories with respect to communication cost have not plateaued, they simply take longer to see a drop in suboptimality.

Theorem 6 (Overlapping token trajectories setting)**.** *Let $\{\boldsymbol{\theta}^t\}$ be a sequence generated by Algorithm 6 in the setting allowing for overlapping token trajectories. If $\mathcal{L}$ is a convex (A8), L-smooth function with a finite infimum (D2), our mini-batch gradient estimate is unbiased (D3) and has a bounded variance (D4), and each node has a minimum probability $\pi > 0$ of being visited by a token at each iteration (A1), we have that, for a stepsize $\eta \in \left(0, \frac{1}{2LSQ}\right]$:*

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2\Delta}{\eta\pi T} + \frac{\eta S^2 Q^2 \sigma^2 L}{\pi B}, \tag{4.6}$$

where $\Delta \coloneqq \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$ and the expectation is with respect to token trajectories $\{\boldsymbol{p}^t\}$ and mini-batches $\{\mathcal{B}^t\}$. For the special, full-batch case, we also have that $\nabla\mathcal{L}\left(\boldsymbol{\theta}^t\right) \to 0$ almost surely.

The proof of Theorem 6 follows a similar approach to that of Theorem 5. However, allowing for overlapping trajectories prevents the quadratic upper bound from decoupling across tokens. We tackle this by resorting to the convexity assumption, which allows us to obtain a decoupled upper bound.[5]

[5]We discuss the derivation of results in suboptimality in Appendix C.3.

4.5 Experiments

We empirically evaluate our method, comparing it with `DCPA` (Alghunaim et al., 2021), a fully decentralized approach, and with `SVFL`, a standard VFL method in the client-server setting as described in Liu et al. (2022b). Although low trajectory variances conceal some confidence intervals, all experiments are run over 5 seeds.

4.5.1 Convex problems

In this section, to allow for a fair comparison between decentralized and semi-decentralized approaches, we define, for all methods, an iteration as the cumulative number of hops. We use CVXPY (Diamond and Boyd, 2016) to obtain $\mathcal{L}^\star$ and use the (relative) suboptimality gap $(\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star)/\mathcal{L}^\star$ as a metric.

We denote the ratio of the cost of client-server to client-client communications as $R_C := C^Z_{CS}/C^Z_{CC} = C^h_{CS}/C^h_{CC}$.[6] When plotting the suboptimality gap with respect to the communication cost, for $R_C = 100$: we take C^Z_{CS} as a unit cost, scale C^Z_{CC} by 0.01, and use these values to compute the total (weighted) communication cost. As explained in Section 4.2, for generalized linear models—such as the ones considered in this section—$C^h_{CC} = C^Z_{CC}$ and $C^h_{CS} = C^Z_{CS}$.

We perform ridge regression on a dataset generated as in Alghunaim et al. (2021), with $N = 1000$ samples and $d = 2000$ features split evenly across clients. Here, $\mathcal{L}(\boldsymbol{\theta}) = \|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2/2 + \alpha\|\boldsymbol{\theta}\|_2^2/2$, where $\boldsymbol{y}$ are the labels and $\alpha = 10$. For `MTCD`, $\eta = 10^{-5}$ and $Q = 20$. For `SVFL`, $\eta = 5 \times 10^{-7}$ and $Q = 20$. For `DCPA`, $\mu_w = 10^{-2}$, $\mu_y = 3 \times 10^{-4}$, and $\mu_x = 3 \times 10^{-2}$.

We perform sparse logistic regression on Gisette (Guyon et al., 2004), where $N = 6000$ and $d = 5000$. Again, the features are split evenly across clients. Let $s(z) := (1 + e^{-z})^{-1}$, $\mathcal{L}(\boldsymbol{\theta})$ is given by:

$$\sum_{n=1}^{N} \left[(1 - y_n)\log(1 - s((\boldsymbol{x}^n)^\top\boldsymbol{\theta})) - y_n \log s((\boldsymbol{x}^n)^\top\boldsymbol{\theta})\right] + \beta\|\boldsymbol{\theta}\|_1,$$

where $\boldsymbol{x}^n$ and $y_n \in \{0, 1\}$ denote sample n and its label, respectively, and $\beta = 1$. For `MTCD`, $\eta = 10^{-4}$ and $Q = 30$. For `DCPA`, $\mu_w = 10^{-3}, \mu_y = 3 \times 10^{-5}, \mu_x = 3 \times 10^{-3}$.[7]

[6] We assume, for simplicity, that client-server communications cost the same in both directions (uplink and downlink), although this does not always hold.

[7] In `DCPA`, the lack of a closed-form solution for the proximal operator of the convex conjugate of the logistic regression loss forces each client to solve a local optimization problem at every iteration.

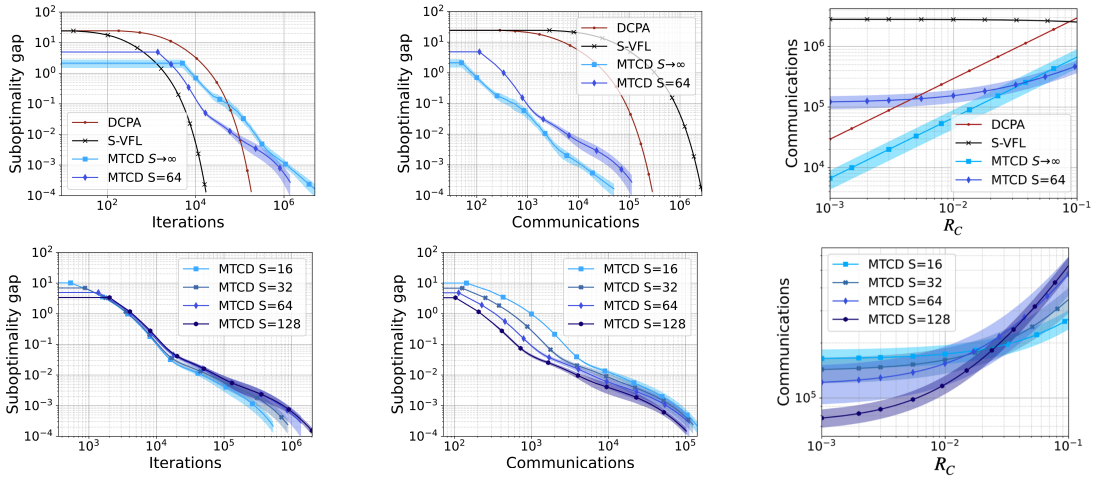


Figure 4.2: We perform ridge regression on a path graph with $K = 80$ nodes. We plot the suboptimality per iteration, the suboptimality per communication, and the number of communications needed to reach a given suboptimality for a range of ratios R_C. In the top row, `MTCD` with $S \to \infty$ has $\Gamma = 1$ (that is, it corresponds to `STCD`) and `MTCD` with $S = 64$ have $\Gamma = 2$ tokens. In the bottom row, all instances of `MTCD` have $\Gamma = 2$ tokens.

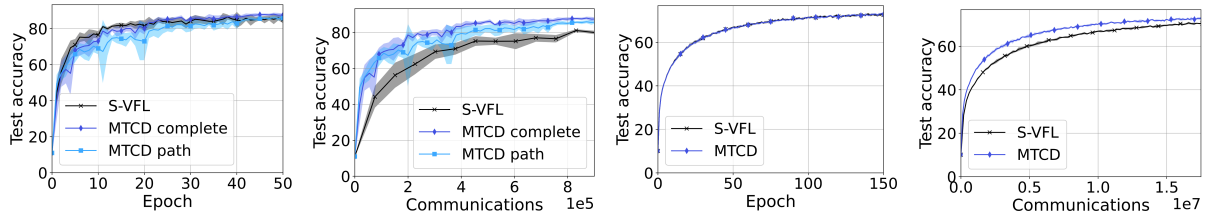


Figure 4.3: The plots on the left concern the training of an MVCNN on ModelNet10 and the ones on the right regard the training of a ResNet18 on CIFAR-10.

Fully decentralized setting. In Figure 4.1, we see that, while `MTCD` with $S \to \infty$ and $\Gamma = 1$ (that is, `STCD`) does not consistently outperform `DCPA` in convergence per iteration, it is significantly more communication efficient. However, we also observe that `STCD` is sensitive to poorly connected networks, as seen when comparing the results in the (well-connected) Erdős-Rényi graph to those in the (poorly-connected) path graph. Note that, for sparse logistic regression, while the proximal term used to handle the regularizer is not covered by our analysis, we observe that it performs well empirically.

Semi-decentralized setting. We now perform ridge regression on larger (and thus more poorly-connected) path graphs. The top row of Figure 4.2 compares `MTCD` with the baselines. For `MTCD`, we assume that P_γ is a uniform distribution over the clients and allow for overlapping token trajectories.

The two leftmost plots show that, despite its slower convergence per iteration, `MTCD` is has a lower communication cost than the baselines (for $R_C = 100$). Most interestingly, the

plot on the right illustrates the communication cost required to achieve a suboptimality gap of 10^{-4} for various values of R_C—keeping the cost of client-server communications constant and varying that of client-client communications—demonstrating that the method that performs the best varies with R_C. In the bottom row, we present similar results, now comparing various `MTCD` instances with different numbers of hops S. As expected, we see that increasing the syncing frequency of `MTCD` leads to faster convergence, yet this comes at the cost of higher communication costs.

In short, Figure 4.2 highlights the advantage of having a flexible method enabling us to select the degree of reliance on the server depending on the application, and confirms that `MTCD` achieves this.

4.5.2 Neural network training

We train an MVCNN (Su et al., 2015) on ModelNet10 (Wu et al., 2015), a dataset of 3D CAD models. We consider 12 clients split into two clusters with six clients each. Each client captures a different (2D) view of an object. We run `MTCD` for both complete graphs and path graphs, both with $S = 6$. For `SVFL`, $\eta = 0.001$. For `MTCD`, we use $\eta = 0.001$ for the complete graph and $\eta = 0.0005$ for the path graph, halving the learning rate every 20 epochs in both cases. For both `MTCD` and `SVFL`, $Q = 10$ and $B = 64$.

We also train a ResNet18 (He et al., 2016) on CIFAR-10 (Krizhevsky and Hinton, 2009). In this experiment, we consider $K = 4$ clients split into two clusters, each with 2 clients. For both `SVFL` and `MTCD`, we have $\eta = 0.0001$, $Q = 10$, and $B = 100$. For `MTCD`, we have $S = 2$

Figure 4.3 presents the results for the ModelNet10 and CIFAR-10 experiments, both in the token-per-cluster setting. In both experiments, we observe that `MTCD` obtains a similar performance to that of `SVFL` in terms of convergence per iteration, yet it achieves a lower communication cost.

4.6 Discussion

We formalize the semi-decentralized multi-token setup and propose `MTCD`, a multi-token method for semi-decentralized VFL. We show, both empirically and analytically, that, by leveraging both client-server and client-client communications, `MTCD` outperforms decentralized and client-server baselines in a range of relevant setups. A natural extension to this work is to combine `MTCD` with compression and privacy mechanisms, such as error-feedback compression and differential privacy.

Chapter 5

Conclusion

This thesis focuses on efficient optimization algorithms for VFL, both in the sense of communication efficiency and of data efficiency. To this end, it leverages techniques such as lossy compression, task sampling for dealing with missing feature blocks, and the coordinated use of different types of communication links, in a semi-decentralized scheme.

We first propose a method for communication-efficient training in the conventional client-server setting, in Chapter 2. `EF-VFL` leverages an error feedback mechanism to speed up and stabilize communication-compressed VFL, achieving a $\mathcal{O}(1/T)$ convergence rate, for sufficiently large batch sizes. This improves upon the state-of-the-art rate of $\mathcal{O}(1/\sqrt{T})$, which required a diminishing stepsize and compression error throughout training.

Next, in Chapter 3, we introduce a method for efficient training and inference in VFL from arbitrary sets of observed blocks of features, without wasting data. `LASER-VFL` achieves this by sharing model parameters across tasks (predictions from different sets of feature blocks) and by employing a task-sampling mechanism. `LASER-VFL` comes with convergence guarantees and improves the performance of the prior art both in the presence of missing features and, remarkably, even when all samples are fully observed.

Lastly, we propose a method for semi-decentralized VFL, in Chapter 4, relying not only on the conventional client-server links, but also on client-client links, to mitigate the bottleneck at the server. The proposed method, `MTCD` is a multi-token method capable of recovering the client-server and decentralized schemes as special cases, while also being able to navigate the semi-decentralized schemes in between. We show, both empirically and analytically, that `MTCD` outperforms decentralized and client-server baselines in a range of relevant setups where neither end of this spectrum of server-dependency is ideal.

The results in this thesis suggest some interesting areas for future research. Namely, while we have established the good performance of `EF-VFL` in standard VFL and in the

presence of multiple local updates, it would also be important to study its performance in other communication schemes—namely the fully and semi-decentralized schemes—and in combination with other techniques, such as the use of asynchronous updates, of privacy mechanisms (such as differential privacy), and of `LASER-VFL`. Recent work (Wu et al., 2025) has shown that the feature blocks across different clients often vary more in size and task relevance than the artificially partitioned datasets typically seen in VFL literature, and that this can impact method performance. It would therefore be valuable to evaluate the robustness of `EF-VFL` under these conditions. It would also be an interesting and natural extension to apply `LASER-VFL` to multi-task learning.

Appendix A

Appendix for Chapter 2

A.1 Preliminaries

If a function $\mathcal{L}$ is L-smooth (D2), then the following quadratic upper bound holds:

$$\forall \boldsymbol{x}, \boldsymbol{y}: \quad \mathcal{L}(\boldsymbol{y}) \leq \mathcal{L}(\boldsymbol{x}) + \nabla \mathcal{L}(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{L}{2} \|\boldsymbol{x} - \boldsymbol{y}\|^2. \tag{A.1}$$

Further, if a map M has a bounded derivative (D7), then the following inequality holds:

$$\forall\, \boldsymbol{x}, \boldsymbol{y}: \quad \|\boldsymbol{M}(\boldsymbol{x}) - \boldsymbol{M}(\boldsymbol{y})\| \leq H \|\boldsymbol{x} - \boldsymbol{y}\|. \tag{A.2}$$

Letting $\epsilon > 0$, we use the following standard inequality in our analysis:

$$\forall \boldsymbol{x}, \boldsymbol{y}: \quad \|\boldsymbol{x} + \boldsymbol{y}\|^2 \leq (1 + \epsilon) \|\boldsymbol{x}\|^2 + (1 + \epsilon^{-1}) \|\boldsymbol{y}\|^2. \tag{A.3}$$

We define the distortion associated with block k at time t as

$$D_k^{(t)} := \left\| \boldsymbol{G}_k^t - \boldsymbol{F}_k(\boldsymbol{\theta}_k^t) \right\|^2$$

and, recall, we denote the total distortion at time t as $D^{(t)} = \sum_{k=0}^{K} D_k^{(t)}$. We also define $\nu := (1 - \alpha) \in [0, 1)$.

In Section 2.4, we introduced the following sigma-algebra

$$\mathcal{F}_t = \sigma(\boldsymbol{G}^0, \boldsymbol{\theta}^1, \boldsymbol{G}^1, \ldots, \boldsymbol{\theta}^t, \boldsymbol{G}^t),$$

where $\boldsymbol{G}^t = \{\boldsymbol{G}_0^t, \ldots, \boldsymbol{G}_K^t\}$. We now further define

$$\mathcal{F}_t' \coloneqq \sigma(\boldsymbol{G}^0, \boldsymbol{\theta}^1, \boldsymbol{G}^1, \ldots, \boldsymbol{\theta}^t, \boldsymbol{G}^t, \boldsymbol{\theta}^{t+1}).$$

Recall that we let $\mathbb{E}_{\mathcal{F}}$ denote the conditional expectation $\mathbb{E}[\cdot \mid \mathcal{F}]$.

Note that, while we write our proofs for Algorithm 1, they can be easily adjusted to cover Algorithm 2. To do so, it suffices to adjust the notation, replacing $\boldsymbol{g}^t$ and $\widetilde{\boldsymbol{g}}^t$ by $\boldsymbol{\nabla}_k^t$ and $\widetilde{\boldsymbol{\nabla}}_k^t$, respectively, and to make minor changes to the proof of Lemma 1, which cause the constant K in Lemma 1 to be replaced with $K+1$. This, in turn, leads to a similar adjustment in the constants of our main theorems.

A.2 Supporting lemmas

A.2.1 Proof of Lemma 1

Decoupling the offset across blocks, we get that

$$\begin{aligned}
\|\boldsymbol{g}^t - \nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 &= \sum_{k=0}^{K} \left\|\boldsymbol{g}_k^t - \nabla_k \mathcal{L}\left(\boldsymbol{\theta}_k\right)\right\|^2, \\
&\leq \sum_{k=0}^{K} \|\nabla \boldsymbol{F}_k(\boldsymbol{\theta}_k)\|^2 \left\|\widetilde{\nabla}_k^t \phi - \nabla_k \phi(\{\boldsymbol{F}_k(\boldsymbol{\theta}_k^t)\}_{k=0}^K)\right\|^2,
\end{aligned}$$

where we use the chain rule and the fact that $\|\boldsymbol{A}\boldsymbol{x}\| \leq \|\boldsymbol{A}\|\|\boldsymbol{x}\|$. Now, it follows from the bounded gradient assumption (D7) on $\{\boldsymbol{F}_k\}$ and the L-smoothness (D2) of Φ that

$$\begin{aligned}
\|\boldsymbol{g}^t - \nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 &\leq H^2 L^2 \sum_{k=0}^{K} \sum_{j \neq k} \left\|\boldsymbol{G}_j^t - \boldsymbol{F}_j(\boldsymbol{\theta}_j^t)\right\|^2 \\
&= H^2 L^2 \sum_{k=0}^{K} \sum_{j \neq k} D_j^{(t)} \\
&= K H^2 L^2 D^{(t)},
\end{aligned}$$

as we set out to prove. For Algorithm 2, the sum $\sum_{j\neq k}$ would instead be $\sum_{j=1}^{K}$, leading to $\|\boldsymbol{g}^t - \nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq (K+1)H^2L^2D^{(t)}$. However, note that, for Algorithm 2, $D_0^{(t)} = 0$.

A.2.2 Proof of Lemma 2

It follows from the definition of distortion and from the update of our compression estimate that

$$\begin{aligned}\mathbb{E}_{\mathcal{F}_t'}\left[D_k^{(t+1)}\right] &= \mathbb{E}_{\mathcal{F}_t'}\left\|\boldsymbol{G}_k^{t+1} - \boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1})\right\|^2 \\ &= \mathbb{E}_{\mathcal{F}_t'}\left\|\boldsymbol{G}_k^{t} + \mathcal{C}(\boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1}) - \boldsymbol{G}_k^t) - \boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1})\right\|^2.\end{aligned}$$

Now, from the definition of contractive compressor (D6) and from (A.3), we have that

$$\begin{aligned}\mathbb{E}_{\mathcal{F}_t'}\left[D_k^{(t+1)}\right] &\leq \nu\left\|\boldsymbol{G}_k^{t} - \boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1})\right\|^2 \\ &\leq \nu(1+\epsilon)\left\|\boldsymbol{G}_k^{t} - \boldsymbol{F}_k(\boldsymbol{\theta}_k^{t})\right\|^2 + \nu(1+\epsilon^{-1})\left\|\boldsymbol{F}_k(\boldsymbol{\theta}_k^{t+1}) - \boldsymbol{F}_k(\boldsymbol{\theta}_k^{t})\right\|^2,\end{aligned}$$

where, recall, $\nu = (1-\alpha) \in [0,1)$. Further, from the bounded gradient assumption—in particular, from (A.2)—we arrive at

$$\begin{aligned}\mathbb{E}_{\mathcal{F}_t'}\left[D_k^{(t+1)}\right] &\leq \nu(1+\epsilon)D_k^{(t)} + \nu(1+\epsilon^{-1})H^2\|\boldsymbol{\theta}_k^{t+1} - \boldsymbol{\theta}_k^t\|^2 \\ &= \nu(1+\epsilon)D_k^{(t)} + \nu(1+\epsilon^{-1})\eta^2H^2\|\widetilde{\boldsymbol{g}}_k^t\|^2,\end{aligned}$$

where, recall, $\widetilde{\boldsymbol{g}}_k^t$ is our (possibly stochastic) update vector. Summing over $k = 0, 1, \ldots, K$ and taking the nonconditional expectation of both sides of the inequality, we get that

$$\mathbb{E}D^{(t+1)} \leq \nu(1+\epsilon)\mathbb{E}D^{(t)} + \nu(1+\epsilon^{-1})\eta^2H^2\mathbb{E}\|\widetilde{\boldsymbol{g}}^t\|^2.$$

Lastly, using the fact that our update vector is unbiased (D3), then bounded variance (D4) is equivalent to $\mathbb{E}\|\widetilde{\boldsymbol{g}}^t\|^2 \leq \mathbb{E}\|\boldsymbol{g}^t\|^2 + \frac{\sigma^2}{B}$, we arrive at (2.7).

A.3 Proof of main theorems

First, let us define some shorthand notation for terms we will be using throughout our proof, whose expectation is with respect to the (possible) randomness in the compression across all steps:

$$\begin{aligned}\text{(compression error)} \qquad & \Omega_1^t \coloneqq \mathbb{E}D^{(t)}, \\ \text{(surrogate norm)} \qquad & \Omega_2^t \coloneqq \mathbb{E}\|\boldsymbol{g}^t\|^2.\end{aligned}$$

A.3.1 Proof of Theorem 1

From the L-smoothness of $\mathcal{L}$—more specifically, from (A.1)—we have that

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}(\boldsymbol{\theta}^t) &\leq \langle \nabla \mathcal{L}(\boldsymbol{\theta}^t), \boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t \rangle + \frac{L}{2}\|\boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t\|^2 \\ &= -\eta \langle \nabla \mathcal{L}(\boldsymbol{\theta}^t), \widetilde{\boldsymbol{g}}^t \rangle + \frac{\eta^2 L}{2}\|\widetilde{\boldsymbol{g}}^t\|^2.\end{aligned}$$

Taking the conditional expectation over the batch selection, it follows from the unbiasedness of $\widetilde{\boldsymbol{g}}^t$ (D3) that

$$\mathbb{E}_{\mathcal{F}_t}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}(\boldsymbol{\theta}^t) \leq -\eta \langle \nabla \mathcal{L}(\boldsymbol{\theta}^t), \boldsymbol{g}^t \rangle + \frac{\eta^2 L}{2}\mathbb{E}_{\mathcal{F}_t}\|\widetilde{\boldsymbol{g}}^t\|^2.$$

From (D4), we have that $\mathbb{E}_{\mathcal{F}_t}\|\widetilde{\boldsymbol{g}}^t - \boldsymbol{g}^t\|^2 \leq \frac{\sigma^2}{B}$, which, under (D3), is equivalent to $\mathbb{E}_{\mathcal{F}_t}\|\widetilde{\boldsymbol{g}}^t\|^2 \leq \|\boldsymbol{g}^t\|^2 + \frac{\sigma^2}{B}$, so

$$\begin{aligned}\mathbb{E}_{\mathcal{F}_t}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}(\boldsymbol{\theta}^t) &\leq -\eta \langle \nabla \mathcal{L}(\boldsymbol{\theta}^t), \boldsymbol{g}^t \rangle + \frac{\eta^2 L}{2}\|\boldsymbol{g}^t\|^2 + \frac{\eta^2 L \sigma^2}{2B} \\ &= -\frac{\eta}{2}\|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 - \frac{\eta}{2}(1 - \eta L)\|\boldsymbol{g}^t\|^2 + \frac{\eta}{2}\|\boldsymbol{g}^t - \nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta^2 L \sigma^2}{2B},\end{aligned}$$

where the last equation follows from the polarization identity $\langle a, b \rangle = \frac{1}{2}(\|a\|^2 + \|b\|^2 - \|a - b\|^2)$. Now, using our surrogate offset bound (2.6) and taking the (non-conditional) expectation, we get that:

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2}\mathbb{E}\|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 - \frac{\eta}{2}(1 - \eta L)\mathbb{E}\|\boldsymbol{g}^t\|^2 + \frac{\eta K H^2 L^2}{2}\mathbb{E}D^{(t)} + \frac{\eta^2 L \sigma^2}{2B}. \quad \text{(A.4)}$$

Using the Ω_1^t and Ω_2^t notation defined earlier and recalling that $\nu = (1 - \alpha) \in [0, 1)$, we rewrite (2.7) and (A.4), respectively, as

$$\Omega_1^{t+1} \leq \nu(1 + \epsilon)\Omega_1^t + \nu(1 + \epsilon^{-1})\eta^2 H^2 \Omega_2^t + \frac{\nu(1 + \epsilon^{-1})\eta^2 H^2 \sigma^2}{B}$$

and

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2}\mathbb{E}\|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta K H^2 L^2}{2}\Omega_1^t - \frac{\eta}{2}(1 - \eta L)\Omega_2^t + \frac{\eta^2 L \sigma^2}{2B}.$$

Multiplying the first inequality by a positive constant w and adding it to the second

one, we get

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) + w\Omega_1^{t+1} - \psi_1(w)\Omega_1^t \leq -\frac{\eta}{2}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \psi_2(w)\Omega_2^t + \left(w\nu(1+\epsilon^{-1})H^2 + \frac{L}{2}\right)\frac{\eta^2\sigma^2}{B}, \tag{A.5}$$

where

$$\psi_1(w) \coloneqq w\nu(1+\epsilon) + \frac{\eta K H^2 L^2}{2}$$

and

$$\psi_2(w) \coloneqq w\nu(1+\epsilon^{-1})\eta^2 H^2 - \frac{\eta}{2}(1-\eta L).$$

Looking at (A.5), we see that, if $\psi_2(w) \leq 0$, we can drop the Ω_2^t term. Further, if $\psi_1(w) \leq w$, we can telescope the Ω_1^t term as we sum the inequalities for $t = 0, \ldots, T-1$, as we do for the $\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t)$ terms. We thus get that:

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2(\mathcal{L}(\boldsymbol{\theta}^0) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^T))}{\eta T} + \frac{2w(\Omega_1^0 - \Omega_1^T)}{\eta T} + \left(2w\nu(1+\epsilon^{-1})H^2 + L\right)\frac{\eta\sigma^2}{B}, \tag{A.6}$$

for

$$w \in \mathcal{W}_\epsilon \coloneqq \left\{w\colon \frac{\eta K H^2 L^2}{2(1-\nu(1+\epsilon))} \leq w \leq \frac{1-\eta L}{2\eta H^2 \nu(1+\epsilon^{-1})}\right\},$$

where the lower bound follows from $\psi_1(w) \leq w$ and the upper bound from $\psi_2(w) \leq 0$.

Bounding η and choosing ϵ. To ensure that $\mathcal{W}_\epsilon$ is not empty, we need

$$\eta^2\gamma(\epsilon)L^2 + \eta L \leq 1 \quad \text{where} \quad \gamma(\epsilon) \coloneqq K H^4 \frac{\nu(1+\epsilon^{-1})}{1-\nu(1+\epsilon)}.$$

From Lemma 5 of Richtárik et al. (2021), we know that, if $a, b > 0$, then $0 \leq \eta \leq \frac{1}{\sqrt{a}+b}$ implies $a\eta^2 + b\eta \leq 1$. Thus, we can ensure that $\mathcal{W}_\epsilon$ is not empty by requiring

$$\eta \leq \frac{1}{\sqrt{\gamma(\epsilon)L^2} + L} = \left(\sqrt{\gamma(\epsilon)}L + L\right)^{-1}.$$

Further, to ensure that all $w \in \mathcal{W}_\epsilon$ are positive, we need $\nu(1+\epsilon) < 1$, which holds for $\epsilon < \frac{1-\nu}{\nu}$. Thus, to have the largest upper bound possible on the stepsize η, we want ϵ to be the solution to the following optimization problem, solved in Lemma 3 of Richtárik et al.

(2021):

$$\epsilon^\star := \operatorname{argmin}_\epsilon \left\{ \widetilde{\gamma}(\epsilon) := \frac{\nu(1+\epsilon^{-1})}{1-\nu(1+\epsilon)} : 0 < \epsilon < \frac{1-\nu}{\nu} \right\} = \frac{1}{\sqrt{\nu}} - 1.$$

It follows that $\sqrt{\widetilde{\gamma}(\epsilon^\star)} = \frac{1+\sqrt{1-\alpha}}{\alpha} - 1$ and thus $\gamma(\epsilon^\star) = KH^4 \left(\frac{1+\sqrt{1-\alpha}}{\alpha} - 1 \right)^2 =: \rho_{\alpha 1}$. We therefore need

$$\eta \leq \left(\sqrt{\gamma(\epsilon^\star)} L + L \right)^{-1} = \left(\sqrt{\rho_{\alpha 1}} L + L \right)^{-1}.$$

Note that, for $\alpha = 1$, we recover $\eta \leq 1/L$.

Choosing w. Now, since $\mathcal{L}^\star \leq \mathcal{L}(\boldsymbol{\theta})$ for all $\boldsymbol{\theta}$ and $\Omega_1^T \geq 0$, we have from (A.6) that, for all $w \in \mathcal{W}_\epsilon$:

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2\Delta}{\eta T} + \frac{2w\Omega_1^0}{\eta T} + \left(2w\nu(1+\epsilon^{-1})H^2 + L \right) \frac{\eta \sigma^2}{B},$$

where $\Delta := \mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star$. From the inequality above, we see that we want $w \in \mathcal{W}_\epsilon$ to be as small as possible. Therefore, we take w to be the lower bound in $\mathcal{W}_\epsilon$. Since $1 - \nu(1+\epsilon^\star) = 1 - \sqrt{\nu}$, this corresponds to setting $w = \frac{\eta K H^2 L^2}{2(1-\sqrt{\nu})}$. Recalling that $\Omega_1^t = \mathbb{E} D^{(t)}$ and $\nu = 1 - \alpha$, we thus arrive at (2.8):

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{2\Delta}{\eta T} + \frac{KH^2L^2}{1-\sqrt{1-\alpha}} \cdot \frac{\mathbb{E} D^{(0)}}{T} + (\eta L \rho_{\alpha 1} + 1) \frac{\eta L \sigma^2}{B}.$$

A.3.2 Proof of Theorem 2

Recall that, using the Ω_1^t and Ω_2^t notation, we can rewrite (2.7) and (A.4), respectively, as

$$\Omega_1^{t+1} \leq \nu(1+\epsilon)\Omega_1^t + \nu(1+\epsilon^{-1})\eta^2 H^2 \Omega_2^t + \frac{\nu(1+\epsilon^{-1})\eta^2 H^2 \sigma^2}{B}$$

and

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2} \mathbb{E} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta K H^2 L^2}{2} \Omega_1^t - \frac{\eta}{2}(1 - \eta L)\Omega_2^t + \frac{\eta^2 L \sigma^2}{2B}.$$

Now, from our earlier introduced Lyapunov function (2.9), $V_t = \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star + c_\alpha \Omega_1^t$, we have that:

$$V_{t+1} = \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^\star + c_\alpha \Omega_1^{t+1}$$

$$
\begin{aligned}
&\overset{\text{(i)}}{\leq} \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star - \frac{\eta}{2}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \left(\frac{\eta K H^2 L^2}{2} + c_\alpha\nu(1+\epsilon)\right)\Omega_1^t + \psi_2(c_\alpha)\Omega_2^t \\
&\qquad + (L + 2c_\alpha\nu(1+\epsilon^{-1})H^2)\frac{\eta^2\sigma^2}{2B} \\
&\overset{\text{(ii)}}{\leq} (1-\eta\mu)(\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star) + \left(\frac{\eta K H^2 L^2}{2} + c_\alpha\nu(1+\epsilon)\right)\Omega_1^t \\
&\qquad + \psi_2(c_\alpha)\Omega_2^t + (L + 2c_\alpha\nu(1+\epsilon^{-1})H^2)\frac{\eta^2\sigma^2}{2B} \\
&= (1-\eta\mu)V_t + \underbrace{\left(\frac{\eta K H^2 L^2}{2} + c_\alpha\nu(1+\epsilon) - c_\alpha(1-\eta\mu)\right)}_{=:\psi_3(c_\alpha)}\Omega_1^t \\
&\qquad + \psi_2(c_\alpha)\Omega_2^t + (L + 2c_\alpha\nu(1+\epsilon^{-1})H^2)\frac{\eta^2\sigma^2}{2B},
\end{aligned}
$$

where (i) follows from (2.7), (A.4), $c_\alpha > 0$, and $\psi_2(w) = w\nu(1+\epsilon^{-1})\eta^2H^2 - \frac{\eta}{2}(1-\eta L)$ and (ii) follows from the PL inequality (D5). Looking the inequality above, we see that, if there is a c_α such that $\psi_2(c_\alpha), \psi_3(c_\alpha) \leq 0$, then

$$
V_{t+1} \leq (1-\eta\mu)V_t + (L + 2c_\alpha H^2\nu(1+\epsilon^{-1}))\frac{\eta^2\sigma^2}{2B}. \tag{A.7}
$$

Note that, similarly to what we had in the proof for Theorem 1, $\psi_2(c_\alpha) \leq 0$ corresponds to an upper bound on c_α, while $\psi_3(c_\alpha) \leq 0$ corresponds to a lower bound on c_α. We therefore want $c_\alpha \in \mathcal{W}'_\epsilon$, where

$$
\mathcal{W}'_\epsilon \coloneqq \left\{c_\alpha \colon \frac{\eta K H^2 L^2}{2(1-\nu(1+\epsilon)-\eta\mu)} \leq c_\alpha \leq \frac{1-\eta L}{2\eta\nu(1+\epsilon^{-1})H^2}\right\}.
$$

Recursing (A.7), we get

$$
\begin{aligned}
V_T &\leq (1-\eta\mu)^T V_0 + (L + 2c_\alpha H^2\nu(1+\epsilon^{-1}))\frac{\eta^2\sigma^2}{2B}\sum_{t=0}^{T-1}(1-\eta\mu)^t \\
&\leq (1-\eta\mu)^T V_0 + (L + 2c_\alpha H^2\nu(1+\epsilon^{-1}))\frac{\eta\sigma^2}{2B\mu},
\end{aligned}
$$

where the second inequality follows from the sum of a geometric series, arriving at the result we set out to prove.

Choosing ϵ and bounding η so that $\mathcal{W}'_\epsilon$ is nonempty. Note that the lower bound defining $\mathcal{W}'_\epsilon$ is positive if $\eta < \frac{1-\nu(1+\epsilon)}{\mu}$, where $1-\nu(1+\epsilon) > 0$ as long as $\epsilon < \frac{1-\nu}{\nu}$. Further,

$\mathcal{W}'_\epsilon$ is not empty, as long as

$$\frac{\eta K H^2 L^2}{2(1-\nu(1+\epsilon)-\eta\mu)} \leq \frac{1-\eta L}{2\eta\nu(1+\epsilon^{-1})H^2},$$

which is equivalent to

$$\eta^2 L^2 \left(\beta_\epsilon(\alpha) K H^4 - \mu/L\right) + \eta L(\theta_\epsilon(\alpha) + \mu/L) \leq \theta_\epsilon(\alpha),$$

where

$$\theta_\epsilon(\alpha) = 1-(1-\alpha)(1+\epsilon) \quad \text{and} \quad \beta_\epsilon(\alpha) = (1-\alpha)(1+\epsilon^{-1}).$$

If $\epsilon \leq \min\left\{\frac{1-\alpha}{\alpha}, \frac{\alpha}{1-\alpha}\right\}$, we have that $\beta_\epsilon(\alpha) \geq 1$ and $\theta_\epsilon(\alpha) \geq 0$ for all α. It follows from $\beta_\epsilon(\alpha) \geq 1$ that $\beta_\epsilon(\alpha) \geq 1KH^4 \geq KH^4 \geq H^4 \geq 1$, where the last inequality follows from (D7) holding for $\boldsymbol{F}_0$. We thus get that

$$\eta^2 L^2 (1-\mu/L) + \eta L(\mu/L) \leq \theta_\epsilon(\alpha). \tag{A.8}$$

Choosing ϵ to be

$$\epsilon^\star = \begin{cases} \alpha, & 0 < \alpha \leq 1/2, \\ 1-\alpha, & 1/2 < \alpha \leq 1, \end{cases}$$

we get that

$$\theta_{\epsilon^\star}(\alpha) = \begin{cases} \alpha^2, & 0 < \alpha \leq 1/2, \\ -1+3\alpha-\alpha^2, & 1/2 < \alpha \leq 1. \end{cases}$$

Since $\alpha^2 \leq -1+3\alpha-\alpha^2$ for $\alpha \in (1/2, 1]$, we have that $\alpha^2 \leq \theta_{\epsilon^\star}(\alpha)$ for all $\alpha \in (0,1]$. Thus, (A.8) holds if

$$\eta^2 L^2 (1-\mu/L) + \eta L(\mu/L) \leq \alpha^2. \tag{A.9}$$

Further, from (D2) and (D5), we get that $0 \leq \mu/L \leq 1$. Therefore, for a sufficiently small η, there exists a positive $c_\alpha \in \mathcal{W}'_\epsilon$ such that $\psi_2(c_\alpha), \psi_3(c_\alpha) \leq 0$. Lastly, note that we can also guarantee that $\eta < \frac{1-\nu(1+\epsilon^\star)}{\mu} = \theta_{\epsilon^\star}(\alpha)/\mu$ by having $\eta < \alpha^2/\mu$, which follows from (A.9).

Choosing c_α to minimize the upper bound. From (A.8), we see that we want c_α to be as small as possible. So, we choose c_α as the lower bound in the definition of $\mathcal{W}'_\epsilon$,

arriving at

$$V_T \leq (1-\eta\mu)^T V_0 + \left(L + 2\left(\frac{\eta K H^2 L^2}{2(1-\nu(1+\epsilon)-\eta\mu)} \right) H^2 \nu \left(1+\epsilon^{-1}\right) \right) \frac{\eta\sigma^2}{2B\mu}.$$

Now, we know that, for $\epsilon = \epsilon^\star$ and $\eta^2 L^2 (1-\mu/L) + \eta\mu \leq \alpha^2$, the lower bound in the definition of $\mathcal{W}'_\epsilon$ is less than or equal to the upper bound. We therefore have that

$$2\left(\frac{\eta K H^2 L^2}{2(1-\nu(1+\epsilon)-\eta\mu)} \right) H^2 \nu \left(1+\epsilon^{-1}\right) \leq \frac{1-\eta L}{\eta}.$$

Using this inequality in the bound above it follows that

$$V_T \leq (1-\eta\mu)^T V_0 + \frac{\sigma^2}{2B\mu},$$

thus arriving at the statement that we set out to prove.

Appendix B

Appendix for Chapter 3

B.1 Additional discussions

B.1.1 A more detailed comparison with closest related works

As mentioned in Chapter 3, a few recent works have addressed the problem of missing features during VFL training. In particular, a line of work uses all samples that are not fully observed locally to train the representation models via self-supervised learning (Feng, 2022; He et al., 2024). This is followed by collaborative training of a joint predictor for the whole federation using the samples without missing features. Another line of work utilizes a similar framework but resorting to semi-supervised learning instead of self-supervised learning in order to take advantage of nonaligned samples (Kang et al., 2022; Yang et al., 2022; Sun et al., 2023). In both cases, these works are lacking in some areas that are improved by our method: **1)** they require the existence of fully observed samples; **2)** they do not allow for missing features at inference time; and **3)** they do not leverage collaborative training when more than one, but less than the total number of clients observed their feature blocks.

Another approach leverages information across the entire federation to train local predictors, thereby enabling inference in the presence of missing features. Feng and Yu (2020); Kang et al. (2024) propose following standard, collaborative vertical FL with a transfer learning method allowing each client to have its own predictor. These works assume that the training samples are fully observed. Liu et al. (2020); Feng et al. (2022) follow a similar direction but they also consider the presence of missing features. Instead of utilizing transfer learning to train local predictors, Ren et al. (2022); Li et al. (2022a) use knowledge distillation under the assumption of fully observed training samples, while Li et al. (2023);

Huang et al. (2023) also employ knowledge distillation, but on top of this they further consider missing features during training. Finally, Xiao et al. (2025) recently proposed using a distributed generative adversarial network for collaborative training on nonoverlapping data, leveraging synthetic data generation. In all of these works, the output of training are local predictors which, despite being able to perform inference when other clients in the federation do not observe their blocks, are also not able to leverage other feature blocks when they *are* observed. This failure to utilize valuable information during inference leads to reduced predictive power.

Only a few, very recent works allow for inference from a varying number of feature blocks in vertical FL. `PlugVFL` (Sun et al., 2024) employs party-wise dropout during training to mitigate performance drops from missing feature blocks at inference; Gao et al. (2024) introduce a multi-stage approach with complementary knowledge distillation, enabling inference with different client subsets; and Ganguli et al. (2023) deal with the related task of handling communication failure during inference in cross-device VFL. These methods make progress towards inference in vertical FL in the presence of missing feature blocks, however, they do not consider missing features in the training data. Further, none of these methods provides convergence guarantees.

In contrast to the prior art, our `LASER-VFL` method can handle any missingness pattern in the feature blocks during both training and inference simultaneously *without wasting any data*. Further, `LASER-VFL` achieves this without requiring a complex pipeline with additional stages and hyperparameters, requiring only minor modifications to the standard VFL approach.

B.1.2 On the use of averaging as the aggregation mechanism

As mentioned in Section 3.3.1, using averaging instead of concatenation as the aggregation mechanism in the fusion model does not reduce the flexibility of the overall approach, provided the representation models are adjusted accordingly. We illustrate this with a simple example employing aggregation by sum (which differs only by a parameter scaling factor).

Consider the scenario with two clients, each extracting a representation, $\boldsymbol{v}_1 \in \mathbb{R}^{d_1}$ and $\boldsymbol{v}_2 \in \mathbb{R}^{d_2}$. When aggregation is by concatenation, the first layer of the fusion model can be written as $\boldsymbol{W} \in \mathbb{R}^{d_0 \times (d_1+d_2)}$, with $\boldsymbol{W} = \begin{bmatrix} \boldsymbol{W}_1 & \boldsymbol{W}_2 \end{bmatrix}$, and we define $\boldsymbol{v} \coloneqq \begin{bmatrix} \boldsymbol{v}_1^\top & \boldsymbol{v}_2^\top \end{bmatrix}^\top$. Hence, the output of the first layer is $\boldsymbol{W}\boldsymbol{v} = \boldsymbol{W}_1\boldsymbol{v}_1 + \boldsymbol{W}_2\boldsymbol{v}_2$.

In contrast, a naive aggregation by sum would restricts us to $\boldsymbol{W}_1(\boldsymbol{v}_1 + \boldsymbol{v}_2)$ and impose $d_1 = d_2$. However, if we include an extra linear layer in the representation models for each

client (namely $\boldsymbol{W}_1$ in the first and $\boldsymbol{W}_2$ in the second), then the outputs become $\boldsymbol{W}_1\boldsymbol{v}_1$ and $\boldsymbol{W}_2\boldsymbol{v}_2$. This way, even with aggregation by summation, the fusion model can recover $\boldsymbol{W}_1\boldsymbol{v}_1 + \boldsymbol{W}_2\boldsymbol{v}_2$.

Thus, by adjusting the representation models appropriately, we can indeed achieve the same flexibility as in the concatenation-based approach, while using aggregation by sum or average.

B.2 Proofs

B.2.1 Preliminaries

The following quadratic upper bound follows from the L-smoothness of $\mathcal{L}^o$ (D2):

$$\forall\, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2 \in \boldsymbol{\Theta}\colon \quad \mathcal{L}^o(\boldsymbol{\theta}_1) \leq \mathcal{L}^o(\boldsymbol{\theta}_2) + \nabla\mathcal{L}^o(\boldsymbol{\theta}_2)^\top(\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2) + \frac{L}{2}\|\boldsymbol{\theta}_1 - \boldsymbol{\theta}_2\|^2. \tag{B.1}$$

Let X denote a random variable and let F be a convex function, Jensen's inequality states that

$$F(\mathbb{E}(X)) \leq \mathbb{E}(F(X)). \tag{B.2}$$

We define the following shorthand notation for conditional expectations used in our proofs:

$$\mathbb{E}_o\left[\cdot\right] \coloneqq \mathbb{E}\left[\cdot \mid \mathcal{K}_o\right],$$

$$\mathbb{E}_t\left[\cdot\right] \coloneqq \mathbb{E}\left[\cdot \mid \mathcal{K}_o, \mathcal{F}^t\right],$$

$$\mathbb{E}_{t^+}\left[\cdot\right] \coloneqq \mathbb{E}\left[\cdot \mid \mathcal{K}_o, \mathcal{F}^t, \mathcal{B}^t\right].$$

B.2.2 Proof of Lemma 3

From the law of total expectation, we have that

$$\mathbb{E}_t\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] = \mathbb{E}_t\left[\mathbb{E}_{t^+}\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right]\right]. \tag{B.3}$$

Focusing on the inner expectation, we have from the definition of $\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)$ in (3.6) and (3.7) that

$$\mathbb{E}_{t^+}\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] = \frac{1}{|\mathcal{B}^t|}\sum_{n\in\mathcal{B}^t}\sum_{k\in\mathcal{K}_o^{(t)}}\sum_{i=1}^{K_o^t} a_i^t \cdot \mathbb{E}_{t^+}\left[\mathcal{L}_{nk\mathcal{S}_{ki}^t}(\boldsymbol{\theta}^t)\right]. \tag{B.4}$$

Now, from the fact that $\mathcal{S}_{ki}^t \sim \mathcal{U}(\mathscr{P}_k^i(\mathcal{K}_o^{(t)}))$, we have that

$$\begin{aligned}\mathbb{E}_{t^+}\left[\mathcal{L}_{nk\mathcal{S}_{ki}^t}(\boldsymbol{\theta}^t)\right] &= \sum_{\mathcal{I}\in\mathscr{P}_k^i(\mathcal{K}_o^{(t)})} \mathbb{P}(\mathcal{S}_{ki}^t = \mathcal{I}) \cdot \mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}^t) \\ &= \frac{1}{|\mathscr{P}_k^i(\mathcal{K}_o^{(t)})|} \cdot \sum_{\mathcal{I}\in\mathscr{P}_k^i(\mathcal{K}_o^{(t)})} \mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}^t) \\ &= \binom{K_o^t - 1}{j-1}^{-1} \cdot \sum_{\mathcal{I}\in\mathscr{P}_k^i(\mathcal{K}_o^{(t)})} \mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}^t).\end{aligned}$$

So, we have from (B.4) and from the fact that $a_i^t = \binom{K_o^t - 1}{i-1}/i$, as defined in (3.7), that

$$\begin{aligned}\mathbb{E}_{t^+}\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] &= \frac{1}{|\mathcal{B}^t|}\sum_{n\in\mathcal{B}^t}\sum_{k\in\mathcal{K}_o^{(t)}}\sum_{i=1}^{K_o^t}\frac{1}{i}\cdot\sum_{\mathcal{I}\in\mathscr{P}_k^i(\mathcal{K}_o^{(t)})}\mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}^t) \\ &= \frac{1}{|\mathcal{B}^t|}\sum_{n\in\mathcal{B}^t}\sum_{k\in\mathcal{K}_o^{(t)}}\sum_{\mathcal{I}\in\mathscr{P}_k(\mathcal{K}_o^{(t)})}\frac{1}{|\mathcal{I}|}\cdot\mathcal{L}_{nk\mathcal{I}}(\boldsymbol{\theta}^t) \\ &= \mathcal{L}_{\mathcal{B}^t}^o(\boldsymbol{\theta}^t).\end{aligned}$$

So, from (B.3) and (D3), it follows that

$$\mathbb{E}_t\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] = \mathbb{E}_t\left[\mathcal{L}_{\mathcal{B}^t}^o(\boldsymbol{\theta}^t)\right] = \mathcal{L}^o(\boldsymbol{\theta}^t).$$

Therefore, we have from the dominated convergence theorem, which allows us to interchange the expectation and the gradient operators, that

$$\mathbb{E}_t\left[\nabla\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] = \nabla\mathbb{E}_t\left[\widehat{\mathcal{L}}_{\mathcal{B}^t\mathcal{S}^t}(\boldsymbol{\theta}^t)\right] = \nabla\mathcal{L}^o(\boldsymbol{\theta}^t).$$

B.2.3 Proof of Theorem 3

Let us show the convergence of our method. It follows from the quadratic upper bound in (A.1) which ensues from the L-smoothness assumption (D2), that

$$\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^o(\boldsymbol{\theta}^t) \leq \nabla\mathcal{L}^o(\boldsymbol{\theta}^t)^\top(\boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t) + \frac{L}{2}\left\|\boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t\right\|^2.$$

Using the fact that $\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \eta \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t)$, we get that:

$$\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^o(\boldsymbol{\theta}^t) \leq -\eta \nabla \mathcal{L}^o(\boldsymbol{\theta}^t)^\top \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) + \frac{\eta^2 L}{2} \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right\|^2.$$

We now take the conditional expectation $\mathbb{E}_t[\cdot]$, defined in Appendix B.2.2, arriving that:

$$\mathbb{E}_t \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) \right] - \mathcal{L}^o(\boldsymbol{\theta}^t) \leq -\eta \nabla \mathcal{L}^o(\boldsymbol{\theta}^t)^\top \mathbb{E}_t \left[\nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right] + \frac{\eta^2 L}{2} \mathbb{E}_t \left[\left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right\|^2 \right].$$

Now, under the unbiasedness assumption (D3), we can use Lemma 3 to arrive at:

$$\mathbb{E}_t \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) \right] - \mathcal{L}^o(\boldsymbol{\theta}^t) \leq -\eta \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \mathbb{E}_t \left[\left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right\|^2 \right]. \tag{B.5}$$

From Lemma 3, we also have that the following equation holds

$$\mathbb{E}_t \left[\left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \right] = \mathbb{E}_t \left[\left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right\|^2 \right] - \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2. \tag{B.6}$$

Further, we have that

$$\begin{aligned}
\mathbb{E}_t \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 &= \mathbb{E}_t \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) + \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \\
&= \mathbb{E}_t \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) \right\|^2 + \mathbb{E}_t \left\| \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \\
&\quad + 2\mathbb{E}_t \left[\mathbb{E}_{t^+} \left[\left(\nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) \right)^\top \left(\nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right) \right] \right].
\end{aligned}$$

Recalling that $\mathbb{E}_{t^+} \left[\nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right] = \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t)$, as seen in Appendix B.2.2, we get

$$\mathbb{E}_t \left[\mathbb{E}_{t^+} \left[\left(\nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) \right)^\top \left(\nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right) \right] \right] = 0.$$

Further, it follows from the bounded variance assumption in (D4) that

$$\mathbb{E}_t \left\| \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \leq \sigma_b^2$$

and

$$\mathbb{E}_t \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}^t}(\boldsymbol{\theta}^t) \right\|^2 = \mathbb{E}_t \left\| \sum_{k \in \mathcal{K}_o^{\mathcal{B}^t}} \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}_k^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o_{\mathcal{B}k}(\boldsymbol{\theta}^t) \right\|^2 \leq K \cdot \sigma_s^2.$$

Therefore, we arrive at

$$\mathbb{E}_t \left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) - \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \leq \sigma_b^2 + K \cdot \sigma_s^2.$$

Using the inequality above in (B.6) we get that

$$\mathbb{E}_t \left[\left\| \nabla \widehat{\mathcal{L}}_{\mathcal{B}^t \mathcal{S}^t}(\boldsymbol{\theta}^t) \right\|^2 \right] \leq \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \sigma_b^2 + K \cdot \sigma_s^2,$$

which we can use in (B.5) to arrive at

$$\begin{aligned} \mathbb{E}_t \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) \right] - \mathcal{L}^o(\boldsymbol{\theta}^t) &\leq -\eta \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right) \\ &= -\eta \left(1 - \frac{\eta L}{2} \right) \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right). \end{aligned}$$

Therefore, for a stepsize $\eta \in (0, 1/L]$, we have that

$$\mathbb{E}_t \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) \right] - \mathcal{L}^o(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2} \left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Taking the unconditional expectation (over $\mathcal{K}_o$ and $\mathcal{F}^{t+1}$) on both sides of this inequality, we get:

$$\mathbb{E} \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^o(\boldsymbol{\theta}^t) \right] \leq -\frac{\eta}{2} \cdot \mathbb{E} \left[\left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \right] + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Now, using the law of total expectation, we get that

$$\mathbb{E} \left[\mathbb{E} \left[\mathcal{L}^o(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^o(\boldsymbol{\theta}^t) \mid \mathcal{F}^{t+1} \right] \right] \leq -\frac{\eta}{2} \cdot \mathbb{E} \left[\mathbb{E} \left[\left\| \nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \right\|^2 \mid \mathcal{F}^{t+1} \right] \right] + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Therefore, using the fact that $\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}[\mathcal{L}^o(\boldsymbol{\theta})]$ and Jensen's inequality (B.2), we have

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2} \cdot \mathbb{E} \left\| \mathbb{E} \left[\nabla \mathcal{L}^o(\boldsymbol{\theta}^t) \mid \mathcal{F}^{t+1} \right] \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Using the dominated convergence theorem, we can interchange the expectation and the gradient operators, arriving at the following descent lemma in expectation:

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2} \cdot \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right). \tag{B.7}$$

Taking the average of the inequality above for $t \in \{0, 1, \ldots, T-1\}$, we get that:

$$\frac{\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^T) - \mathcal{L}(\boldsymbol{\theta}^0)}{T} \leq -\frac{\eta}{2T} \sum_{t=0}^{T-1} \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Lastly, rearranging the terms and using the existence and definition of $\mathcal{L}^\star$ (D2), we have that

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 \leq \frac{2(\mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star)}{\eta T} + \eta L \left(\sigma_b^2 + K \cdot \sigma_s^2 \right),$$

thus arriving at the result in (3.9).

B.2.4 Proof of Theorem 4

Following the same steps as in the proof of Theorem 3, we can arrive at the same descent lemma in expectation (B.7),

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) \leq -\frac{\eta}{2} \cdot \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Rearranging the terms and subtracting the infimum on both sides, we get that

$$\mathbb{E}\mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}^\star \leq \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star - \frac{\eta}{2} \cdot \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Now, using the definition of our Lyapunov function, $\delta^t = \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star$, we arrive at the following inequality:

$$\delta^{t+1} \leq \delta^t - \frac{\eta}{2} \cdot \mathbb{E} \left\| \nabla \mathcal{L}(\boldsymbol{\theta}^t) \right\|^2 + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Now, from the PL inequality (D5), we have that:

$$\delta^{t+1} \leq \delta^t - \mu\eta \cdot \mathbb{E} \left[\mathcal{L}(\boldsymbol{\theta}^t) - \mathcal{L}^\star \right] + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Therefore, again using the definition of our Lyapunov function, we arrive at

$$\delta^{t+1} \leq (1 - \mu\eta) \cdot \delta^t + \frac{\eta^2 L}{2} \left(\sigma_b^2 + K \cdot \sigma_s^2 \right).$$

Recursing the inequality above, we get that

$$\delta^T \leq (1-\mu\eta)^T \delta^0 + \frac{\eta^2 L}{2}\left(\sigma_b^2 + K \cdot \sigma_s^2\right) \sum_{t=0}^{T-1} (1-\mu\eta)^t.$$

Finally, using the sum of a geometric series, we arrive at

$$\delta^T \leq (1-\mu\eta)^T \delta^0 + \frac{\eta L}{2\mu}\left(\sigma_b^2 + K \cdot \sigma_s^2\right),$$

which corresponds to the result that we set out to prove.

B.3 Experiment details

Notes on `VFedTrans`. For the `VFedTrans` (Huang et al., 2023) pipeline, we use `FedSVD` (Chai et al., 2022) for federated representation learning and an autoencoder for local representation learning, as these are the best performing methods in Huang et al. (2023). For the local predictor, we use a multilayer perceptron, as in Huang et al. (2023). As seen in Table 3.1, the performance of `VFedTrans` is nearly identical to that of `Local`. However, given that requires a multi-stage pipeline with operations for each pair of clients, its runtime is much longer. In particular, for the Credit dataset, for $p_{\text{miss}} = 0.0$ for training and inference, running `VFedTrans` takes 2843.0 ± 50.6s, while running `Local` takes only 127.8 ± 3.0s.

Notes on `PlugVFL`. We also note that the original `PlugVFL` paper (Sun et al., 2024) **1)** does not address missing training data and **2)** only considers settings with two clients. In our experiments, we extended `PlugVFL` to handle more generic scenarios. To address missing training data fairly, we replaced representations for missing feature blocks with zeros at the fusion model, similarly to the dropout mechanisms in the original paper, but now, for missing data, these representations are zeroed throughout training rather than per iteration. To generalize `PlugVFL` for more than two clients, we assigned a dropout probability to each passive party (clients not holding the fusion model).

Notes on MIMIC-IV. We use ICU data of MIMIC-IV v1.0 (Johnson et al., 2020) and follow the data processing pipeline in Gupta et al. (2022). We focus on the task of predicting in-hospital mortality for patients admitted with chronic kidney disease. We do feature selection with diagnosis data and the selected features of chart events (labs and vitals) used in the "Proof of concept experiments" Gupta et al. (2022) as input features.

Models used. For the HAPT and Credit datasets, we trained simple multilayer perceptrons; for the MIMIC-IV dataset, we trained an LSTM (Hochreiter, 1997); and, for the CIFAR-10 and CIFAR-100 experiments, we use ResNets18 (He et al., 2016) models.

Appendix C

Appendix for Chapter 4

C.1 Supporting lemmas

Throughout the proofs, we slightly abuse notation for the sake of conciseness, denoting $\boldsymbol{\theta}_{c,:}$ as $\boldsymbol{\theta}_c$. (This collides with $\boldsymbol{\theta}_k$ in Chapter 4, but it should still be clear which block we refer to when using each.)

We use the following inequality, which follows from the Cauchy-Schwarz inequality, in our results:

$$\|\boldsymbol{v}_1 + \cdots + \boldsymbol{v}_n\|^2 \leq n \left(\|\boldsymbol{v}_1\|^2 + \cdots + \|\boldsymbol{v}_1\|^2\right). \tag{C.1}$$

C.1.1 Proof of Lemma 4

We start by using the fact that the difference at block c between consecutive iterates, $\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t$, is given by the sum of the gradient-based updates used from $(t, 0)$ to $(t, P-1)$:

$$\begin{aligned}
\mathbb{E}_{t^+}\left[\nabla_c \mathcal{L}(\boldsymbol{\theta}^t)^\top (\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t)\right] &= \mathbb{E}_{t^+}\left[(\boldsymbol{\Pi}_c \nabla \mathcal{L}(\boldsymbol{\theta}^t))^\top (\boldsymbol{\theta}^{t,P}(c) - \boldsymbol{\theta}^{t,0}(c))\right] \\
&= \mathbb{E}_{t^+}\left[\nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_c \left(\sum_{p=0}^{P-1} \boldsymbol{\theta}^{t,p+1}(c) - \boldsymbol{\theta}^{t,p}(c)\right)\right] \\
&= \mathbb{E}_{t^+}\left[\nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_c \left(\sum_{p=0}^{P-1} -\eta \boldsymbol{\Pi}_{c, i_c^{t,p}} \nabla \mathcal{L}_{\mathcal{B}^t}(\boldsymbol{\theta}^{t,p}(c))\right)\right] \\
&= -\eta \sum_{p=0}^{P-1} \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c, i_c^{t,p}} \mathbb{E}_{t^+}\left[\nabla \mathcal{L}_{\mathcal{B}^t}(\boldsymbol{\theta}^{t,p}(c))\right] \\
&\overset{(a)}{=} -\eta \sum_{p=0}^{P-1} \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c, i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)),
\end{aligned}$$

where (a) follows from (D3).

We now separate the $p = 0$ term from the remaining, $p > 0$, terms of the sum, which correspond to updates using stale information, and use a polarization identity on the latter:

$$\begin{aligned}
\mathbb{E}_{t^+}\left[\nabla_c \mathcal{L}(\boldsymbol{\theta}^t)^\top(\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t)\right] &= -\eta \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c,i_c^{t,0}} \nabla \mathcal{L}(\boldsymbol{\theta}^t) - \eta \sum_{p=1}^{P-1} \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c,i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) \\
&\overset{(a)}{=} -\eta \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c,i_c^{t,0}} \nabla \mathcal{L}(\boldsymbol{\theta}^t) - \frac{\eta}{2} \sum_{p=1}^{P-1} \nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \boldsymbol{\Pi}_{c,i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^t) \\
&\quad - \frac{\eta}{2} \sum_{p=1}^{P-1} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c))^\top \boldsymbol{\Pi}_{c,i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) \\
&\quad + \frac{\eta}{2} \sum_{p=1}^{P-1} \|\boldsymbol{\Pi}_{c,i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) - \boldsymbol{\Pi}_{c,i_c^{t,p}} \nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 \\
&\overset{(b)}{=} -\eta \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} - \frac{\eta}{2} \sum_{p=1}^{P-1} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}} \\
&\quad - \frac{\eta}{2} \sum_{p=1}^{P-1} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}} + \frac{\eta}{2} \sum_{p=1}^{P-1} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) - \nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}},
\end{aligned} \tag{C.2}$$

where we use $\boldsymbol{u}^\top \boldsymbol{v} = \frac{1}{2}\|\boldsymbol{u}\|^2 + \frac{1}{2}\|\boldsymbol{v}\|^2 - \frac{1}{2}\|\boldsymbol{u} - \boldsymbol{v}\|^2$ in (a) and $\|\boldsymbol{u}\|^2_{\boldsymbol{A}} = \boldsymbol{u}^\top \boldsymbol{A}\boldsymbol{u}$ in (b). Let us upper bound the offset term:

$$\begin{aligned}
\|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) - \nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}} &\overset{(a)}{\leq} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) - \nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 \\
&\overset{(b)}{\leq} L^2 \|\boldsymbol{\theta}^{t,p}(c) - \boldsymbol{\theta}^{t,0}(c)\|^2 \\
&= L^2 \|-\eta \boldsymbol{\Pi}_{c,i_c^{t,0}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,0}(c)) - \eta \sum_{r}^{p-1} \boldsymbol{\Pi}_{c,i_c^{t,r}} \nabla \mathcal{L}(\boldsymbol{\theta}^{t,r}(c))\|^2,
\end{aligned}$$

where (a) follows from $\boldsymbol{\Pi}_{c,i_c^{t,p}} \preceq \boldsymbol{I}$ and (b) from (D2).

Now, using (C.1) and the fact that $p \leq P-1$, we get that:

$$\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c)) - \nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,p}}} \leq L^2\eta^2 p\Big(\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,0}}} + \sum_r^{p-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,r}(c))\|^2_{\mathbf{\Pi}_{c,i_c^{t,r}}}\Big)$$
$$\leq L^2\eta^2(P-1)\Big(\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,0}}} + \sum_r^{P-2}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,r}(c))\|^2_{\mathbf{\Pi}_{c,i_c^{t,r}}}\Big).$$

Plugging this inequality into (C.2), we arrive at (4.3).

C.1.2 Proof of Lemma 5

From the fact that $||\boldsymbol{\theta}_i|| \leq ||\boldsymbol{\theta}||$ and from (C.1), we have that:

$$\begin{aligned}
\mathbb{E}_{t^+}\|\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t\|^2 &= \mathbb{E}_{t^+}\|\boldsymbol{\theta}_c^{t,P}(c) - \boldsymbol{\theta}_c^{t,0}(c)\|^2 \\
&\leq \mathbb{E}_{t^+}\|\boldsymbol{\theta}^{t,P}(c) - \boldsymbol{\theta}^{t,0}(c)\|^2 \\
&= \mathbb{E}_{t^+}\left\|\sum_{p=0}^{P-1}\boldsymbol{\theta}^{t,p+1}(c) - \boldsymbol{\theta}^{t,p}(c)\right\|^2 \\
&\leq P\cdot\sum_{p=0}^{P-1}\mathbb{E}_{t^+}\left\|\boldsymbol{\theta}^{t,p+1}(c) - \boldsymbol{\theta}^{t,p}(c)\right\|^2.
\end{aligned}$$

Now, from our gradient-based update, we get that:

$$\begin{aligned}
\mathbb{E}_{t^+}\|\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t\|^2 &\leq P\sum_{p=0}^{P-1}\mathbb{E}_{t^+}\left\|-\eta\mathbf{\Pi}_{c,i_c^{t,p}}\nabla\mathcal{L}_{\mathcal{B}^t}(\boldsymbol{\theta}^{t,p}(c))\right\|^2 \\
&= \eta^2 P\sum_{p=0}^{P-1}\mathbb{E}_{t^+}\left\|\nabla\mathcal{L}_{\mathcal{B}^t}(\boldsymbol{\theta}^{t,p}(c))\right\|^2_{\mathbf{\Pi}_{c,i_c^{t,p}}} \\
&\overset{(a)}{\leq} \eta^2 P\sum_{p=0}^{P-1}\left[\left\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\right\|^2_{\mathbf{\Pi}_{c,i_c^{t,p}}} + \frac{\sigma^2}{B}\right] \\
&= \eta^2 P\Big(\frac{\sigma^2 P}{B} + \|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,0}}} + \sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\mathbf{\Pi}_{c,i_c^{t,p}}}\Big),
\end{aligned}$$

where (a) follows from (D3) and (D4). We thus arrive at (4.4).

C.2 Proof of Theorem 5

It follows from the L-smoothness assumption (D2) that

$$\mathcal{L}(\boldsymbol{\theta}^{t+1}) \leq \mathcal{L}(\boldsymbol{\theta}^t) + \nabla\mathcal{L}(\boldsymbol{\theta}^t)^\top(\boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t) + \frac{L}{2}||\boldsymbol{\theta}^{t+1} - \boldsymbol{\theta}^t||^2.$$

We now let $\delta^t \coloneqq \mathcal{L}(\boldsymbol{\theta}^{t+1}) - \mathcal{L}(\boldsymbol{\theta}^t)$, split the upper bound across cluster-specific terms, and take the conditional expectation:

$$\mathbb{E}_{t^+}\delta^t \leq \sum_{c=1}^{C} \mathbb{E}_{t^+}\left[\nabla_c\mathcal{L}(\boldsymbol{\theta}^t)^\top(\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t) + \frac{L}{2}||\boldsymbol{\theta}_c^{t+1} - \boldsymbol{\theta}_c^t||^2\right].$$

Using Lemma 4 and Lemma 5 on the bound above, we get:

$$\begin{aligned}
\mathbb{E}_{t^+}\delta^t \leq\ & -\eta\left(1 - \frac{\eta^2L^2(P-1)^2}{2}\right)\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} - \frac{\eta}{2}\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\boldsymbol{\Pi}_{c,i_c^{t,p}}} \\
& - \frac{\eta}{2}(1-\eta^2L^2(P-1)^2)\cdot\sum_{c=1}^{C}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}} \\
& + \frac{\eta^2P^2\sigma^2L}{2B} + \frac{\eta^2LP}{2}\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} + \frac{\eta^2LP}{2}\cdot\sum_{c=1}^{C}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}}.
\end{aligned}$$

Or, equivalently,

$$\begin{aligned}
\mathbb{E}_{t^+}\delta^t \leq\ & -\eta\left(1 - \frac{\eta PL}{2} - \frac{\eta^2L^2(P-1)^2}{2}\right)\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} \\
& - \frac{\eta}{2}\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\boldsymbol{\Pi}_{c,i_c^{t,p}}} + \frac{\eta^2P^2\sigma^2L}{2B} \\
& - \frac{\eta}{2}(1-\eta LP-\eta^2L^2(P-1)^2)\sum_{c=1}^{C}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}}.
\end{aligned} \tag{C.3}$$

Now, as long as $1-\eta LP-\eta^2L^2(P-1)^2 > 0$, which we can ensure by having $\eta \in \left(0, \frac{1}{2LP}\right]$, we can drop the $\sum_{c=1}^{C}\sum_{p=1}^{P-1}\|\nabla f(\boldsymbol{\theta}^{t,p}(c))\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,p}}}$ term, arriving at:

$$\mathbb{E}_{t^+}\delta^t \leq -\mu(\eta)\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\boldsymbol{\Pi}_{c,i_c^{t,0}}} - \nu(\eta)\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\boldsymbol{\Pi}_{c,i_c^{t,p}}} + \frac{\eta^2P^2\sigma^2L}{2B}.$$

where $\mu(\eta) \coloneqq \eta\left(1 - \frac{\eta PL}{2} - \frac{\eta^2 L^2 (P-1)^2}{2}\right)$ and $\nu(\eta) \coloneqq \frac{\eta}{2}$. Further, let $\widetilde{\tau}(\eta) \coloneqq \min\{\mu(\eta), \nu(\eta)\}$, we get that

$$\mathbb{E}_{t^+}\delta^t \leq -\widetilde{\tau}(\eta)\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,0}}+\cdots+\mathbf{\Pi}_{c,i_c^{t,P-1}}} + \frac{\eta^2 P^2\sigma^2 L}{2B}.$$

We now take the conditional expectation with respect to $\boldsymbol{p}^t$,

$$\mathbb{E}_t\delta^t \leq -\widetilde{\tau}(\eta)\sum_{c=1}^{C}\mathbb{E}_t\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{c,i_c^{t,0}}+\cdots+\mathbf{\Pi}_{c,i_c^{t,P-1}}} + \frac{\eta^2 P^2\sigma^2 L}{2B},$$

and, since $\boldsymbol{\theta}^t$ is fixed when conditioned on $\mathcal{F}^t$, we get that:

$$\mathbb{E}_t\delta^t \leq -\widetilde{\tau}(\eta)\cdot\sum_{c=1}^{C}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\bar{\mathbf{\Pi}}_c^t} + \frac{\eta^2 P^2\sigma^2 L}{2B},$$

where $\bar{\mathbf{\Pi}}_c^t \coloneqq \mathbb{E}\left[\mathbf{\Pi}_{c,i_c^{t,0}} + \cdots + \mathbf{\Pi}_{c,i_c^{t,P-1}} \mid \mathcal{F}^t\right]$. From (A1), we have that $\bar{\mathbf{\Pi}}_c^t \succeq \pi\mathbf{\Pi}_c$. From this, we have that:

$$\begin{aligned}\mathbb{E}_t\delta^t &\leq -\widetilde{\tau}(\eta)\cdot\pi\cdot\sum_{c=1}^{C}\|\nabla f(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_c} + \frac{\eta^2 P^2\sigma^2 L}{2B} \\ &= -\widetilde{\tau}(\eta)\cdot\pi\cdot\|\nabla f(\boldsymbol{\theta}^t)\|^2 + \frac{\eta^2 P^2\sigma^2 L}{2B}.\end{aligned} \tag{C.4}$$

Thus, since $\widetilde{\tau}(\eta) > 0$ for $\eta \in \left(0, \frac{1}{2LP}\right]$, it follows from the classic result by Robbins and Siegmund (1971) that, in the full-batch case, $\nabla\mathcal{L}\left(\boldsymbol{\theta}^t\right) \to 0$ almost surely. Taking the unconditional expectation and letting $\tau(\eta) \coloneqq \widetilde{\tau}(\eta)/\eta$, we get:

$$\mathbb{E}\delta^t \leq -\eta\pi\cdot\tau(\eta)\cdot\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta^2 P^2\sigma^2 L}{2B}.$$

Rearranging the terms and averaging over t, we have that:

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{\mathcal{L}(\boldsymbol{\theta}^0) - \mathbb{E}\mathcal{L}(\boldsymbol{\theta}^T)}{\eta\pi T\cdot\tau(\eta)} + \frac{\eta^2 P^2\sigma^2 L}{2\eta\pi B\cdot\tau(\eta)}.$$

Thus, from the finite infimum assumption in (D2), we get:

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2 \leq \frac{\mathcal{L}(\boldsymbol{\theta}^0) - \mathcal{L}^\star}{\eta\pi T\cdot\tau(\eta)} + \frac{\eta^2 P^2\sigma^2 L}{2\eta\pi B\cdot\tau(\eta)}.$$

Finally, since $\tau(\eta) \geq 1/2$ for $\eta \in \left(0, \frac{1}{2LP}\right]$, we arrive at (4.5).

C.3 Proof of Theorem 6

We now consider the case where tokens roam over overlapping sets of clients. We define $\mathbf{\Pi}_\gamma$ analogously to $\mathbf{\Pi}_c$. That is, the nonzero entries along the diagonal correspond to the blocks that can be visited by token γ. This means that $\mathbf{\Pi}_\gamma = \boldsymbol{I}$, yet we stick to this notation for consistency, and because the $\mathbf{\Pi}_{\gamma, i_\gamma^{t,p}}$ notation will remain useful. In this setting, (A1) holds for a single "cluster": the whole graph. Lemma 4 and Lemma 5 also continue to hold, sufficing to adjust their notation:

$$\begin{aligned}\mathbb{E}_{t^+}\left[\nabla_\gamma \mathcal{L}(\boldsymbol{\theta}^t)^\top (\boldsymbol{\theta}_\gamma^{t+1} - \boldsymbol{\theta}_\gamma^t)\right] \leq & -\eta\left(1 - \frac{\eta^2 L^2 (P-1)^2}{2}\right) \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma, i_\gamma^{t,0}}} \\ & - \frac{\eta}{2} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1} \mathbf{\Pi}_{\gamma, i_c^{t,p}}} \\ & - \frac{\eta}{2}(1 - \eta^2 L^2 (P-1)^2) \sum_{p=1}^{P-1} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(\gamma))\|^2_{\mathbf{\Pi}_{\gamma, i_\gamma^{t,p}}}\end{aligned}$$

and

$$\mathbb{E}_{t^+} ||\boldsymbol{\theta}_\gamma^{t+1} - \boldsymbol{\theta}_\gamma^t||^2 \leq \eta^2 P \left(\frac{\sigma^2 P}{B} + \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma, i_\gamma^{t,0}}} + \sum_{p=1}^{P-1} \|\nabla \mathcal{L}(\boldsymbol{\theta}^{t,p}(\gamma))\|^2_{\mathbf{\Pi}_{\gamma, i_\gamma^{t,p}}}\right),$$

respectively. From (A8), we have that

$$\mathcal{L}(\boldsymbol{\theta}^{t+1}) = \mathcal{L}\left(\frac{1}{\Gamma}\sum_{\gamma=1}^{\Gamma} \boldsymbol{\theta}^{t,P}(\gamma)\right) \leq \frac{1}{\Gamma}\sum_{\gamma=1}^{\Gamma} \mathcal{L}(\boldsymbol{\theta}^{t,P}(\gamma)).$$

Thus, it follows from L-smoothness (D2) that

$$\delta^t \leq \frac{1}{\Gamma}\sum_{\gamma=1}^{\Gamma}\left[\nabla \mathcal{L}(\boldsymbol{\theta}^t)^\top \left(\boldsymbol{\theta}^{t,P}(\gamma) - \boldsymbol{\theta}^t\right) + \frac{L}{2}\left\|\boldsymbol{\theta}^{t,P}(\gamma) - \boldsymbol{\theta}^t\right\|^2\right].$$

Taking the conditional expectation, we get that:

$$\mathbb{E}_{t^+}\delta^t \leq \frac{1}{\Gamma}\sum_{\gamma=1}^{\Gamma} \mathbb{E}_{t^+}\left[\nabla_\gamma \mathcal{L}(\boldsymbol{\theta}^t)^\top (\boldsymbol{\theta}_\gamma^{t+1} - \boldsymbol{\theta}_\gamma^t) + \frac{L}{2}||\boldsymbol{\theta}_\gamma^{t+1} - \boldsymbol{\theta}_\gamma^t||^2\right].$$

We bound the inner product term using Lemma 4 and the norm term using Lemma 5, arriving at:

$$\begin{aligned}\mathbb{E}_{t^+}\delta^t \leq &-\frac{\eta}{\Gamma}\left(1-\frac{\eta^2L^2(P-1)^2}{2}\right)\cdot\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,0}}}-\frac{\eta}{2\Gamma}\cdot\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}\\&-\frac{\eta}{2\Gamma}(1-\eta^2L^2(P-1)^2)\cdot\sum_{\gamma=1}^{\Gamma}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(\gamma))\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}\\&+\frac{\eta^2P^2\sigma^2L}{2B}+\frac{\eta^2LP}{2\Gamma}\cdot\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,0}}}+\frac{\eta^2LP}{2\Gamma}\cdot\sum_{\gamma=1}^{\Gamma}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(\gamma))\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}.\end{aligned}$$

Thus, we have that

$$\begin{aligned}\mathbb{E}_{t^+}\delta^t \leq &-\frac{\eta}{\Gamma}\left(1-\frac{\eta PL}{2}-\frac{\eta^2L^2(P-1)^2}{2}\right)\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,0}}}\\&-\frac{\eta}{2\Gamma}\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}+\frac{\eta^2P^2\sigma^2L}{2B}\\&-\frac{\eta}{2\Gamma}(1-\eta LP-\eta^2L^2(P-1)^2)\sum_{\gamma=1}^{\Gamma}\sum_{p=1}^{P-1}\|\nabla\mathcal{L}(\boldsymbol{\theta}^{t,p}(\gamma))\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}.\end{aligned}$$

Now, if $\eta\in\left(0,\frac{1}{2LP}\right]$, we can drop the last term, and get that:

$$\mathbb{E}_{t^+}\delta^t \leq -\frac{\mu(\eta)}{\Gamma}\cdot\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\mathbf{\Pi}_{\gamma,i_\gamma^{t,0}}}-\frac{\nu(\eta)}{\Gamma}\cdot\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=1}^{P-1}\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}+\frac{\eta^2P^2\sigma^2L}{2B}.$$

Thus, using the fact that $\widetilde{\tau}(\eta)=\min\{\mu(\eta),\nu(\eta)\}$, we have:

$$\mathbb{E}_{t^+}\delta^t \leq -\frac{\widetilde{\tau}(\eta)}{\Gamma}\sum_{\gamma=1}^{\Gamma}\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=0}^{P-1}\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}+\frac{\eta^2P^2\sigma^2L}{2B}.$$

We now take the conditional expectation $\mathbb{E}_t$,

$$\mathbb{E}_t\delta^t \leq -\frac{\widetilde{\tau}(\eta)}{\Gamma}\sum_{\gamma=1}^{\Gamma}\mathbb{E}_t\|\nabla\mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\sum_{p=0}^{P-1}\mathbf{\Pi}_{\gamma,i_\gamma^{t,p}}}+\frac{\eta^2P^2\sigma^2L}{2B},$$

and, since $\boldsymbol{\theta}^t$ is fixed when conditioned on $\mathcal{F}^t$, we get that:

$$\mathbb{E}_t \delta^t \leq -\frac{\widetilde{\tau}(\eta)}{\Gamma} \cdot \sum_{\gamma=1}^{\Gamma} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2_{\bar{\boldsymbol{\Pi}}^t_\gamma} + \frac{\eta^2 P^2 \sigma^2 L}{2B},$$

where $\bar{\boldsymbol{\Pi}}^t_\gamma \coloneqq \mathbb{E}_t \left[\boldsymbol{\Pi}_{\gamma, i^{t,0}_\gamma} + \cdots + \boldsymbol{\Pi}_{\gamma, i^{t,P-1}_\gamma} \right]$. From (A1), we have that $\bar{\boldsymbol{\Pi}}^t_\gamma \succeq \pi \boldsymbol{I}$. This can be achieved through various communication schemes, as discussed below, in Appendix C.4. Therefore

$$\mathbb{E}_t \delta^t \leq -\frac{\widetilde{\tau}(\eta)}{\Gamma} \cdot \pi \cdot \sum_{\gamma=1}^{\Gamma} \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta^2 P^2 \sigma^2 L}{2B} = -\widetilde{\tau}(\eta) \cdot \pi \cdot \|\nabla \mathcal{L}(\boldsymbol{\theta}^t)\|^2 + \frac{\eta^2 P^2 \sigma^2 L}{2B},$$

which matches inequality (C.4) exactly. We can therefore continue the proof as in Appendix C.2, arriving the result in (4.6).

On the derivation of suboptimality results. Note that, assuming only convexity, it is not straightforward to derive a suboptimality result, as our update vector is not an unbiased estimate of the gradient, and thus the standard procedure to derive such results from a descent lemma does not apply. Yet, it would be straightforward to obtain a suboptimality result (and prove linear convergence) if we further assumed the Polyak-Łojasiewicz inequality (Polyak, 1963) to hold.

C.4 Lower bounding π

As mentioned earlier, we can achieve a positive lower bound for $\pi > 0$ through a variety of communication schemes. Some examples of such schemes include:

- If all clients have a probability p_0 of receiving the token directly from the server, then (A1) holds for $\pi = p_0$ for any $P \geq 1$, since, for all $c \in [C]$, $j \in [K_c]$, and $t = 0, \ldots, T-1$:

$$\mathbb{P}\left(\bigcup_{p=0}^{P-1} \{ i_c^{t,p} = j \} \right) \geq \mathbb{P}\left(i_c^{t,0} = j \right) = p_0.$$

- If at least one client i in each cluster c has a probability p_0 of receiving the token directly from the server and S is greater than or equal to the diameter—the greatest distance between any pair of nodes in a graph—of the cluster. Then, it follows from the lazy

random walk of the token that, for all $c \in [C]$, $j \in [K_c]$, and $t = 0, \ldots, T-1$:

$$\mathbb{P}\left(\bigcup_{p=0}^{P-1} \{i_c^{t,p} = j\}\right) \geq p_0 \cdot (1 + d_{\max})^{-S},$$

where $d_{\max}$ is the maximum node degree across all clusters. Thus, we have that (A1) holds for $\pi = p_0 \cdot \frac{1}{(1+d_{\max})^S}$.

- We can also leverage the fact that the token roaming follows a Markov chain to arrive at a tighter bound than the previous one. In particular, if S is greater than the mixing time of the Markov chain, if we make some mild assumptions on the associated transition matrix, we can leverage the results in Paulin (2012) to drop the exponential dependency on S above.

Bibliography

Alghunaim, S. A., Lyu, Q., Yan, M., and Sayed, A. H. (2021). Dual consensus proximal algorithm for multi-agent sharing problems. *IEEE Transactions on Signal Processing*, 69:5568–5579. 1.3, 4.1, 4.5, 4.5.1

Alistarh, D., Grubic, D., Li, J., Tomioka, R., and Vojnovic, M. (2017). QSGD: Communication-efficient SGD via gradient quantization and encoding. *Advances in neural information processing systems*, 30. 1, 2.1, 2.2

Alistarh, D., Hoefler, T., Johansson, M., Konstantinov, N., Khirirat, S., and Renggli, C. (2018). The convergence of sparsified gradient methods. *Advances in Neural Information Processing Systems*, 31. 2.1, 2.2, 2.2.1

Alon, N., Avin, C., Koucky, M., Kozma, G., Lotker, Z., and Tuttle, M. R. (2008). Many random walks are faster than one. In *Proceedings of the twentieth annual symposium on parallelism in algorithms and architectures*, pages 119–128. 1.3

Amiri, M. M. and Gündüz, D. (2020). Machine learning at the wireless edge: Distributed stochastic gradient descent over-the-air. *IEEE Transactions on Signal Processing*, 68:2155–2169. 2.1

Beck, A. and Tetruashvili, L. (2013). On the convergence of block coordinate descent type methods. *SIAM Journal on Optimization*, 23(4):2037–2060. 4.1

Beznosikov, A., Horváth, S., Richtárik, P., and Safaryan, M. (2023). On biased compression for distributed learning. *Journal of Machine Learning Research*, 24(276):1–50. 2.1

Bian, J. and Xu, J. (2022). Mobility improves the convergence of asynchronous federated learning. *arXiv:2206.04742*. 4.1

Caruana, R. (1997). Multitask learning. *Machine learning*, 28:41–75. 3.3.2

Castiglia, T., Wang, S., and Patterson, S. (2024). Flexible vertical federated learning with heterogeneous parties. *IEEE Transactions on Neural Networks and Learning Systems*, 35(12):17878–17892. 2.2.2

Castiglia, T. J., Das, A., Wang, S., and Patterson, S. (2022). Compressed-VFL: Communication-efficient learning with vertically partitioned data. In *International Conference on Machine Learning*, pages 2738–2766. PMLR. 1, 1.1, 1.1, 1.1, 1.3, 2, 2.1, 2.2.2, 2.2.2, 2.4.1, 2.1, 2.5, 2.5.2, 4.1, 4.4.1, 4.4.1

Ceballos, I., Sharma, V., Mugica, E., Singh, A., Roman, A., Vepakomma, P., and Raskar, R. (2020). SplitNN-driven vertical partitioning. *arXiv:2008.04137*. 1, 4.2

Chai, D., Wang, L., Zhang, J., Yang, L., Cai, S., Chen, K., and Yang, Q. (2022). Practical lossless federated singular vector decomposition over billion-scale data. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, pages 46–55. B.3

Chen, H., Ye, Y., Xiao, M., and Skoglund, M. (2022). Asynchronous parallel incremental block-coordinate descent for decentralized machine learning. *arXiv:2202.03263*. 1.3, 4.1

Chen, J. and Sayed, A. H. (2012). Diffusion adaptation strategies for distributed optimization and learning over networks. *IEEE Transactions on Signal Processing*, 60(8):4289–4305. 2.1

Chen, T., Jin, X., Sun, Y., and Yin, W. (2020). VAFL: a method of vertical asynchronous federated learning. *arXiv:2007.06081*. 1, 2.1, 4.1

Cheng, Y., Liu, Y., Chen, T., and Yang, Q. (2020). Federated learning for privacy-preserving AI. *Communications of the ACM*, 63(12):33–36. 1, 1

Diamond, S. and Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5. 4.5.1

Du, Y., Yang, S., and Huang, K. (2020). High-dimensional stochastic gradient quantization for communication-efficient edge learning. *IEEE Transactions on Signal Processing*, 68:2128–2142. 2.1

Duchi, J. C., Agarwal, A., and Wainwright, M. J. (2012). Dual averaging for distributed optimization: Convergence analysis and network scaling. *IEEE Transactions on Automatic Control*, 57(3):592–606. 1.3

Feng, S. (2022). Vertical federated learning-based feature selection with non-overlapping sample utilization. *Expert Systems with Applications*, 208:118097. 3.1, B.1.1

Feng, S., Li, B., Yu, H., Liu, Y., and Yang, Q. (2022). Semi-supervised federated heterogeneous transfer learning. *Knowledge-Based Systems*, 252:109384. 3.1, B.1.1

Feng, S. and Yu, H. (2020). Multi-participant multi-class vertical federated learning. *arXiv preprint arXiv:2001.11154*. 3.1, B.1.1

Fercoq, O. and Richtárik, P. (2015). Accelerated, parallel, and proximal coordinate descent. *SIAM Journal on Optimization*, 25(4):1997–2023. 4.1

Ganguli, S., Zhou, Z., Brinton, C. G., and Inouye, D. I. (2023). Fault tolerant serverless vfl over dynamic device environment. *arXiv preprint arXiv:2312.16638*. 3.1, 3.5, B.1.1

Gao, D., Wan, S., Fan, L., Yao, X., and Yang, Q. (2024). Complementary knowledge distillation for robust and privacy-preserving model serving in vertical federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19832–19839. 1.2, 3.1, B.1.1

Guo, Y., Sun, Y., Hu, R., and Gong, Y. (2021). Hybrid local SGD for federated learning with heterogeneous communications. In *International Conference on Learning Representations*. 4.1

Gupta, M., Gallamoza, B., Cutrona, N., Dhakal, P., Poulain, R., and Beheshti, R. (2022). An Extensive Data Processing Pipeline for MIMIC-IV. In *Proceedings of the 2nd Machine Learning for Health symposium*, volume 193 of *Proceedings of Machine Learning Research*, pages 311–325. PMLR. 3.5, B.3

Guyon, I., Gunn, S., Ben-Hur, A., and Dror, G. (2004). Result analysis of the NIPS 2003 feature selection challenge. *Advances in neural information processing systems*, 17. 4.5.1

Haddadpour, F., Kamani, M. M., Mokhtari, A., and Mahdavi, M. (2021). Federated learning with compression: Unified analysis and sharp guarantees. In *International Conference on Artificial Intelligence and Statistics*, pages 2350–2358. PMLR. 5, 2.4.2

Hard, A., Rao, K., Mathews, R., Ramaswamy, S., Beaufays, F., Augenstein, S., Eichner, H., Kiddon, C., and Ramage, D. (2018). Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*. 1

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2.5.1, 4.5.2, B.3

He, L., Bian, A., and Jaggi, M. (2018). COLA: Decentralized linear learning. *Advances in Neural Information Processing Systems*, 31. 1.3, 4.1

He, Y., Kang, Y., Zhao, X., Luo, J., Fan, L., Han, Y., and Yang, Q. (2024). A hybrid self-supervised learning framework for vertical federated learning. *IEEE Transactions on Big Data*, pages 1–13. 1.2, 3.1, B.1.1

Hendrikx, H. (2023). A principled framework for the design and analysis of token algorithms. In *International Conference on Artificial Intelligence and Statistics*, pages 470–489. PMLR. 1.3

Hochreiter, S. (1997). Long short-term memory. *Neural Computation MIT-Press*. B.3

Hosseinalipour, S., Azam, S. S., Brinton, C. G., Michelusi, N., Aggarwal, V., Love, D. J., and Dai, H. (2022). Multi-stage hybrid federated learning over large-scale D2D-enabled fog networks. *IEEE/ACM Transactions on Networking*, 30(4):1569–1584. 4.1

Hu, Y., Niu, D., Yang, J., and Zhou, S. (2019). Fdml: A collaborative machine learning framework for distributed features. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2232–2240. 2.2.2

Huang, C.-j., Wang, L., and Han, X. (2023). Vertical federated knowledge transfer via representation distillation for healthcare collaboration networks. In *Proceedings of the ACM Web Conference 2023*, pages 4188–4199. 1.2, 3.1, 3.5, B.1.1, B.3

Hui, P., Yoneki, E., Chan, S. Y., and Crowcroft, J. (2007). Distributed community detection in delay tolerant networks. In *Proceedings of 2nd ACM/IEEE international workshop on Mobility in the evolving internet architecture*, pages 1–8. 4.4.1

Inose, H., Yasuda, Y., and Murakami, J. (1962). A telemetering system by code modulation-δ-σmodulation. *IRE Transactions on Space Electronics and Telemetry*, (3):204–209. 2.2.1

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr. 3.3.1

Johnson, A., Bulgarelli, L., Pollard, T., Horng, S., Celi, L. A., and Mark, R. (2020). Mimic-iv. *PhysioNet. Available online at: https://physionet. org/content/mimiciv/1.0/(accessed August 23, 2021)*, pages 49–55. 3.5, B.3

Kang, Y., He, Y., Luo, J., Fan, T., Liu, Y., and Yang, Q. (2024). Privacy-preserving federated adversarial domain adaptation over feature groups for interpretability. *IEEE Transactions on Big Data*, 10(6):879–890. 3.1, B.1.1

Kang, Y., Liu, Y., and Liang, X. (2022). Fedcvt: Semi-supervised vertical federated learning with cross-view training. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 13(4):1–16. 1.2, 3.1, B.1.1

Karimireddy, S. P., Rebjock, Q., Stich, S., and Jaggi, M. (2019). Error feedback fixes signsgd and other gradient compression schemes. In *International Conference on Machine Learning*, pages 3252–3261. PMLR. 2.2.1

Khan, A., ten Thij, M., and Wilbik, A. (2022). Communication-efficient vertical federated learning. *Algorithms*, 15(8):273. 1.1, 2.1

Koloskova, A., Loizou, N., Boreiri, S., Jaggi, M., and Stich, S. (2020). A unified theory of decentralized SGD with changing topology and local updates. In *International Conference on Machine Learning*, pages 5381–5393. PMLR. 1.3

Koloskova, A., Stich, S., and Jaggi, M. (2019). Decentralized stochastic optimization and gossip algorithms with compressed communication. In *International Conference on Machine Learning*, pages 3478–3487. PMLR. 2.1

Krizhevsky, A. and Hinton, G. (2009). Learning multiple layers of features from tiny images. *Master's thesis, Department of Computer Science, University of Toronto*. 2.5.1, 2.5.2, 3.5, 4.5.2

Kurin, V., De Palma, A., Kostrikov, I., Whiteson, S., and Mudigonda, P. K. (2022). In defense of the unitary scalarization for deep multi-task learning. *Advances in Neural Information Processing Systems*, 35:12169–12183. 3.3.2

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324. 2.5.1

Li, B., Cen, S., Chen, Y., and Chi, Y. (2020a). Communication-efficient distributed optimization in networks with gradient tracking and variance reduction. *Journal of Machine Learning Research*, 21(180):1–51. 1.3

Li, B. and Chi, Y. (2025). Convergence and privacy of decentralized nonconvex optimization with gradient clipping and communication compression. *IEEE Journal of Selected Topics in Signal Processing*, 19(1):273–282. 2.1, 2.6

Li, M., Chen, Y., Wang, Y., and Pan, Y. (2020b). Efficient asynchronous vertical federated learning via gradient prediction and double-end sparse compression. In *2020 16th international conference on control, automation, robotics and vision (ICARCV)*, pages 291–296. IEEE. 1.1, 2.1

Li, W., Xia, Q., Cheng, H., Xue, K., and Xia, S.-T. (2023). Vertical semi-federated learning for efficient online advertising. In *Proceedings of the International Workshop on Trustworthy Federated Learning in Conjunction with IJCAI 2023 (FL-IJCAI'23)*. 3.1, B.1.1

Li, W., Xia, Q., Deng, J., Cheng, H., Liu, J., Xue, K., Cheng, Y., and Xia, S.-T. (2022a). Vfed-ssd: Towards practical vertical federated advertising. *arXiv preprint arXiv:2205.15987*. 3.1, B.1.1

Li, Z., Zhao, H., Li, B., and Chi, Y. (2022b). Soteriafl: A unified framework for private federated learning with communication compression. *Advances in Neural Information Processing Systems*, 35:4285–4300. 2.1, 2.6

Lian, X., Zhang, C., Zhang, H., Hsieh, C.-J., Zhang, W., and Liu, J. (2017). Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent. *Advances in Neural Information Processing Systems*, 30. 1, 1.3, 2.1

Lin, F. P.-C., Hosseinalipour, S., Azam, S. S., Brinton, C. G., and Michelusi, N. (2021). Semi-decentralized federated learning with cooperative D2D local model aggregations. *IEEE Journal on Selected Areas in Communications*, 39(12):3851–3869. 1.3, 4.1, 4.4.1

Liu, C., Zhu, L., and Belkin, M. (2022a). Loss landscapes and optimization in over-parameterized non-linear systems and neural networks. *Applied and Computational Harmonic Analysis*, 59:85–116. 1.4.3

Liu, Y., Kang, Y., Xing, C., Chen, T., and Yang, Q. (2020). A secure federated transfer learning framework. *IEEE Intelligent Systems*, 35(4):70–82. 3.1, B.1.1

Liu, Y., Kang, Y., Zou, T., Pu, Y., He, Y., Ye, X., Ouyang, Y., Zhang, Y.-Q., and Yang, Q. (2024). Vertical federated learning: Concepts, advances, and challenges. *IEEE Transactions on Knowledge and Data Engineering*. 1, 1, 2.1, 2.2.2, 2.4.2, 3.1, 4, 4.1, 4.3

Liu, Y., Zhang, X., Kang, Y., Li, L., Chen, T., Hong, M., and Yang, Q. (2022b). FedBCD: A communication-efficient collaborative learning framework for distributed features. *IEEE Transactions on Signal Processing*, 70:4277–4290. 1, 1.3, 2.1, 2.5, 2.5.3, 3.1, 3.5, 4.1, 4.1, 4.4.1, 4.4.1, 4.5

Mao, X., Yuan, K., Hu, Y., Gu, Y., Sayed, A. H., and Yin, W. (2020). Walkman: A communication-efficient random-walk algorithm for decentralized optimization. *IEEE Transactions on Signal Processing*, 68:2513–2528. 4.3

McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR. 1, 1, 1, 2.1

Mishchenko, K., Gorbunov, E., Takáč, M., and Richtárik, P. (2024). Distributed learning with compressed gradient differences. *Optimization Methods and Software*, pages 1–16. 2.2.1

Mota, J. F., Xavier, J. M., Aguiar, P. M., and Püschel, M. (2013). D-ADMM: A communication-efficient distributed algorithm for separable optimization. *IEEE Transactions on Signal processing*, 61(10):2718–2723. 2.1

Nassif, R., Vlaski, S., Carpentiero, M., Matta, V., Antonini, M., and Sayed, A. H. (2023). Quantization for decentralized learning under subspace constraints. *IEEE Transactions on Signal Processing*, 71:2320–2335. 2.1

Nedic, A. and Bertsekas, D. P. (2001). Incremental subgradient methods for nondifferentiable optimization. *SIAM Journal on Optimization*, 12(1):109–138. 1.3

Nedic, A., Olshevsky, A., and Rabbat, M. G. (2018). Network topology and communication-computation tradeoffs in decentralized optimization. *Proceedings of the IEEE*, 106(5):953–976. 1.3

Nedic, A. and Ozdaglar, A. (2009). Distributed subgradient methods for multi-agent optimization. *IEEE Transactions on Automatic Control*, 54(1):48–61. 1.3

Nesterov, Y. (2012). Efficiency of coordinate descent methods on huge-scale optimization problems. *SIAM Journal on Optimization*, 22(2):341–362. 4.1

Nguyen, D. C., Ding, M., Pathirana, P. N., Seneviratne, A., Li, J., and Poor, H. V. (2021). Federated learning for internet of things: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 23(3):1622–1658. 1

Nguyen, Q., Mukkamala, M. C., and Hein, M. (2018). On the loss landscape of a class of deep neural networks with no bad local valleys. *arXiv preprint arXiv:1809.10749*. 1.4.3

Paulin, D. (2012). Concentration inequalities for markov chains by marton couplings and spectral methods. *arXiv preprint arXiv:1212.2015*. C.4

Polyak, B. T. (1963). Gradient methods for the minimisation of functionals. *USSR Computational Mathematics and Mathematical Physics*, 3(4):864–878. 1.4.3, C.3

Ren, Z., Yang, L., and Chen, K. (2022). Improving availability of vertical federated learning: Relaxing inference on non-overlapping data. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 13(4):1–20. 3.1, B.1.1

Reyes-Ortiz, J.-L., Oneto, L., Samà, A., Parra, X., and Anguita, D. (2016). Transition-aware human activity recognition using smartphones. *Neurocomputing*, 171:754–767. 3.5

Richtárik, P., Sokolov, I., and Fatkhullin, I. (2021). EF21: A new, simpler, theoretically better, and practically faster error feedback. *Advances in Neural Information Processing Systems*, 34:4384–4396. 2.1, 2.2.1, A.3.1

Richtárik, P. and Takáč, M. (2012). Iteration complexity of randomized block-coordinate descent methods for minimizing a composite function. *Mathematical Programming*, 144(1-2):1–38. 4.1

Rieke, N., Hancox, J., Li, W., Milletari, F., Roth, H. R., Albarqouni, S., Bakas, S., Galtier, M. N., Landman, B. A., Maier-Hein, K., et al. (2020). The future of digital health with federated learning. *NPJ digital medicine*, 3(1):119. 1

Robbins, H. and Siegmund, D. (1971). A convergence theorem for non negative almost supermartingales and some applications. In *Optimizing methods in statistics*, pages 233–257. Elsevier. 4.4.1, C.2

Sapio, A., Canini, M., Ho, C.-Y., Nelson, J., Kalnis, P., Kim, C., Krishnamurthy, A., Moshref, M., Ports, D., and Richtárik, P. (2021). Scaling distributed machine learning with in-network aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 785–808. 2.1

Seide, F., Fu, H., Droppo, J., Li, G., and Yu, D. (2014). 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech dnns. In *Fifteenth annual conference of the international speech communication association*. 2.1, 2.2.1

Sharma, H., Haque, A., and Blaabjerg, F. (2021). Machine learning in wireless sensor networks for smart cities: a survey. *Electronics*, 10(9):1012. 1

Shlezinger, N., Chen, M., Eldar, Y. C., Poor, H. V., and Cui, S. (2020). Uveqfed: Universal vector quantization for federated learning. *IEEE Transactions on Signal Processing*, 69:500–514. 2.1

Smith, V., Chiang, C.-K., Sanjabi, M., and Talwalkar, A. S. (2017). Federated multi-task learning. *Advances in neural information processing systems*, 30. 1

Smith, V., Forte, S., Ma, C., Takáč, M., Jordan, M. I., and Jaggi, M. (2018). Cocoa: A general framework for communication-efficient distributed optimization. *Journal of Machine Learning Research*, 18(230):1–49. 4.1

Stich, S. U., Cordonnier, J.-B., and Jaggi, M. (2018). Sparsified SGD with memory. *Advances in Neural Information Processing Systems*, 31. 2.1, 2.2

Su, H., Maji, S., Kalogerakis, E., and Learned-Miller, E. (2015). Multi-view convolutional neural networks for 3D shape recognition. In *Proceedings of the IEEE international conference on computer vision*. 2.5.1, 4.5.2

Sun, J., Du, Z., Dai, A., Baghersalimi, S., Amirshahi, A., Atienza, D., and Chen, Y. (2024). Plugvfl: Robust and ip-protecting vertical federated learning against unexpected quitting of parties. In *2024 IEEE International Conference on Big Data (BigData)*, pages 1124–1133. 3.1, 3.5, B.1.1, B.3

Sun, J., Xu, Z., Yang, D., Nath, V., Li, W., Zhao, C., Xu, D., Chen, Y., and Roth, H. R. (2023). Communication-efficient vertical federated learning with limited overlapping samples. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5203–5212. 3.1, B.1.1

Sun, T., Sun, Y., Xu, Y., and Yin, W. (2019). Markov chain block coordinate descent. *Computational Optimization and Applications*, 75(1):35–61. 4.1, 4.3, 1, 12, 12, 12

Sun, W., Chen, Y., Yang, X., Cao, J., and Song, Y. (2021). Fedio: bridge inner-and outer-hospital information for perioperative complications prognostic prediction via federated learning. In *2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 3215–3221. IEEE. 1

Valdeira, P., Chi, Y., Soares, C., and Xavier, J. (2023). A multi-token coordinate descent method for semi-decentralized vertical federated learning. *arXiv:2309.09977*. 4

Valdeira, P., Wang, S., and Chi, Y. (2025a). Vertical federated learning with missing features during training and inference. In *The Thirteenth International Conference on Learning Representations*. 3

Valdeira, P., Xavier, J., Soares, C., and Chi, Y. (2025b). Communication-efficient vertical federated learning via compressed error feedback. *IEEE Transactions on Signal Processing*, pages 1–15. 2, 3.1

Wang, H. and Chi, Y. (2025). Communication-efficient federated optimization over semi-decentralized networks. *IEEE Transactions on Signal and Information Processing over Networks*, 11:147–160. 4.1

Wright, S. J. (2015). Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34. 4.1

Wu, Z., Qin, Z., Hou, J., Zhao, H., Li, Q., He, B., and Fan, L. (2025). Vertical federated learning in practice: The good, the bad, and the ugly. *arXiv preprint arXiv:2502.08160*. 5

Wu, Z., Song, S., Khosla, A., Yu, F., Zhang, L., Tang, X., and Xiao, J. (2015). 3D ShapeNets: A deep representation for volumetric shapes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2.5.1, 4.5.2

Xiao, Y., Li, X., Li, T., Wang, R., Pang, Y., and Wang, G. (2025). A distributed generative adversarial network for data augmentation under vertical federated learning. *IEEE Transactions on Big Data*, 11(1):74–85. 3.1, B.1.1

Xie, C., Koyejo, S., and Gupta, I. (2019). Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*. 1, 2.1

Yang, L., Chai, D., Zhang, J., Jin, Y., Wang, L., Liu, H., Tian, H., Xu, Q., and Chen, K. (2023). A survey on vertical federated learning: From a layered perspective. *arXiv preprint arXiv:2304.01829*. 2.1, 4.1

Yang, T., Huang, Y., Liang, Y., and Chi, Y. (2024). In-context learning with representations: Contextual generalization of trained transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 1.4.3

Yang, Y., Ye, X., and Sakurai, T. (2022). Multi-view federated learning with data collaboration. In *Proceedings of the 2022 14th International Conference on Machine Learning and Computing*, pages 178–183. 3.1, B.1.1

Ye, Y., Chen, H., Ma, Z., and Xiao, M. (2020). Decentralized consensus optimization based on parallel random walk. *IEEE Communications Letters*, 24(2):391–395. 1.3

Yeh, I.-C. and Lien, C.-h. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert systems with applications*, 36(2):2473–2480. 3.5

Yemini, M., Saha, R., Ozfatura, E., Gündüz, D., and Goldsmith, A. J. (2023). Robust semi-decentralized federated learning via collaborative relaying. *IEEE Transactions on Wireless Communications*, 23(7):7520–7536. 4.1

Ying, B., Yuan, K., and Sayed, A. H. (2018). Supervised learning under distributed features. *IEEE Transactions on Signal Processing*, 67(4):977–992. 2.1

Yu, L., Han, M., Li, Y., Lin, C., Zhang, Y., Zhang, M., Liu, Y., Weng, H., Jeon, Y., Chow, K.-H., et al. (2024). A survey of privacy threats and defense in vertical federated learning: From model life cycle perspective. *arXiv preprint arXiv:2402.03688*. 3.1

Zhang, X., Liu, Y., Liu, J., Argyriou, A., and Han, Y. (2021). D2D-assisted federated learning in mobile edge computing networks. In *2021 IEEE Wireless Communications and Networking Conference (WCNC)*. 4.1

Zhang, X., Yin, W., Hong, M., and Chen, T. (2020). Hybrid federated learning: Algorithms and implementation. *arXiv preprint arXiv:2012.12420*. 4.1

Zhao, H., Li, B., Li, Z., Richtárik, P., and Chi, Y. (2022). Beer: Fast $O(1/T)$ rate for decentralized nonconvex optimization with communication compression. *Advances in Neural Information Processing Systems*, 35:31653–31667. 1.3, 2.1

Zhou, Y., Aryal, S., and Bouadjenek, M. R. (2024). Review for handling missing data with special missing mechanism. *arXiv preprint arXiv:2404.04905*. 3.3.2